
УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 11.09.2026, 02:28:54
Файлов исходного кода: 88
Суммарный объём кода: 1.15 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пракса (KMP) [id: string-kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

```
24. src/data/graphContent.ts
25. src/data/quizzes.ts
```

Билеты (учебный контент)

```
26. src/data/tickets/ticket1_5.ts
27. src/data/tickets/ticket14_20.ts
28. src/data/tickets/ticket21_24.ts
29. src/data/tickets/ticket6_13.ts
```

Интерактивные компоненты и симуляторы

```
30. src/components/ArticulationPointsViz.tsx
31. src/components/BellmanFordViz.tsx
32. src/components/BfsViz.tsx
33. src/components/ChapterImage.tsx
34. src/components/ChapterNav.tsx
35. src/components/ChapterView.tsx
36. src/components/CompilerPanelContent.tsx
37. src/components/DfsBridgesSimulator.tsx
38. src/components/DijkstraViz.tsx
39. src/components/DPVisualizer.tsx
40. src/components/EulerSimulator.tsx
41. src/components/ExpressQuiz.tsx
42. src/components/FloydViz.tsx
43. src/components/GraphTraversalViz.tsx
44. src/components/HeapViz.tsx
45. src/components/hints/demoKit.tsx
46. src/components/hints/demosExtra.tsx
47. src/components/hints/demosGraph.tsx
48. src/components/hints/demosStruct.tsx
49. src/components/hints/MiniDemo.tsx
50. src/components/hints/SimulatorModal.tsx
51. src/components/hints/TermPopover.tsx
52. src/components/JohnsonViz.tsx
53. src/components/KosarajuViz.tsx
54. src/components/KruskalSimulator.tsx
55. src/components/MnemonicCards.tsx
56. src/components/MobileToc.tsx
57. src/components/Navbar.tsx
```

CI / автоматизация

87. .github/workflows/deploy-pages.yml

88. .github/workflows/rebuild-pdf.yml

=====

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

=====

ФАЙЛ 1/88: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

A .github/workflows/deploy-pages.yml

M .github/workflows/rebuild-pdf.yml

M index.html

M public/export/full\_code\_all\_pages.pdf

M public/export/full\_code\_all\_pages.txt

A public/favicon.svg

M src/App.tsx

M src/components/Navbar.tsx

A src/components/PythonCompiler.tsx

M vite.config.ts

```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

name: Deploy app to GitHub Pages

on:

push:

```
run: cp dist/index.html dist/404.html

- name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist

deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
```

```
+           path: ./dist
+
+   deploy:
+     name: Publish to GitHub Pages
+     needs: build
+     runs-on: ubuntu-latest
+     environment:
+       name: github-pages
+       url: ${ steps.deployment.outputs.page_url }
+     steps:
+       - name: Deploy
+         id: deployment
+         uses: actions/deploy-pages@v5
+
+
+## `./.github/workflows/rebuild-pdf.yml`
+
+### Полное содержимое после изменений
+
+```yaml
+name: Rebuild code PDF
+
+# Пайплайн пересборки PDF при каждом коммите:
+#
+#   исходный код  →  full_code_all_pages.txt  (script)
+#               |
+#               →  page-0001.png ... page-NNNN.png  (
+#               |
+#               →  full_code_all_page
+#
+# Готовые TXT и PDF коммитаются обратно в ветку (их отда
+# промежуточные PNG сохраняются как artifacts запуска.
+
+on:
+  push:
+    branches:
+      - "*"
+    paths-ignore:
```

```
run: |
    sudo apt-get update -qq
    sudo apt-get install -y --no-install-recommends \
        imagemagick \
        # На Ubuntu политика ImageMagick иногда запрещает вывод в stdout
        # разрешаем то, что нужно пайплайну (text -> pdf)
        for policy in /etc/ImageMagick-6/policy.xml /usr/share/ImageMagick-6/policies.xml; do
            if [ -f "$policy" ]; then
                sudo sed -i 's/rights="none" pattern="\{PDF\}" rights="write"/rights="write"/' $policy
            fi
        done
    convert -version
```

Универсальное пособие • полный исходный код

```
-----  
diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml  
index bdc3c33..ff49c0c 100644
```

```
--- a/.github/workflows/rebuild-pdf.yml
```

```
+++ b/.github/workflows/rebuild-pdf.yml
```

```
@@ -42,15 +42,15 @@ jobs:
```

```
  steps:
```

```
    - name: Checkout
```

```
      - uses: actions/checkout@v4
```

```
+      uses: actions/checkout@v6
```

```
      with:
```

```
        fetch-depth: 0
```

```
        persist-credentials: true
```

```
    - name: Setup Node.js
```

```
      - uses: actions/setup-node@v4
```

```
+      uses: actions/setup-node@v7
```

```
      with:
```

```
        node-version: "22"
```

```
+        node-version: "24"
```

```
        cache: npm
```

```
    - name: Install ImageMagick + DejaVu fonts
```

```
@@ -97,7 +97,7 @@ jobs:
```

```
      } >> "$GITHUB_STEP_SUMMARY"
```

```
    - name: Upload artifacts (PDF + TXT)
```

```
      - uses: actions/upload-artifact@v4
```

```
+      uses: actions/upload-artifact@v7
```

```
      with:
```

```
        name: full-code-export
```

```
        path: |
```

```
@@ -107,7 +107,7 @@ jobs:
```

```
        retention-days: 30
```

```
    - name: Upload artifacts (PNG-страницы)
```

```
      - uses: actions/upload-artifact@v4
```

```
+      uses: actions/upload-artifact@v7
```

```
    </head>
    <body class="bg-slate-950 text-slate-100 min-h-screen">
    ....
```

```
## `public/favicon.svg`
```

```
### Полное содержимое после изменений
```

```
````xml
```

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64">
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
````
```

```
### Patch относительно `main`
```

```
````diff
```

```
diff --git a/public/favicon.svg b/public/favicon.svg
new file mode 100644
index 00000000..3bbbba0
--- /dev/null
+++ b/public/favicon.svg
@@ -0,0 +1,5 @@
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64">
+ <rect width="64" height="64" rx="16" fill="#4f46e5"/>
+ <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
+ <circle cx="47" cy="21" r="4" fill="#34d399"/>
+</svg>
````
```

```
## `src/App.tsx`
```

```
### Полное содержимое после изменений
```

```
````tsx
```

```
import { useCallback, useEffect, useMemo, useState } from 'react';
```



```
return () => window.removeEventListener("keydown",
 }, [goTo, next, prev]));
```

```
const progress = useMemo(() => ((index + 1) / chapters
```

```
return (
```

```
 <div className="min-h-screen bg-slate-950 text-slate
```

```
 <Navbar activeTab={activeTab} setActiveTab={setAc
```

```
 {activeTab === "guide" ? (
```

```
 <>
```

```
 <MobileToc
```

```
 selectedChapterId={activeChapterId}
```

```
 setSelectedChapterId={goTo}
```

```
 currentTitle={activeChapter.title}
```

```
 index={index}
```

```
 total={chapters.length}
```

```
 />
```

```
 <div className="flex flex-col lg:flex-row max
```

```
 {/* Сайдбар только на десктопе – на мобильном
```

```
 <div className="hidden lg:block lg:w-80 shr
```

```
 <Sidebar selectedChapterId={activeChapter
```

```
 </div>
```

```
 <main id="main-content" className="flex-1 m
```

```
 {/* Шапка главы с быстрыми переходами */}
```

```
 <div className="flex items-center justify
```

```
 <div className="min-w-0">
```

```
 <div className="text-[10px] uppercase
```

```
 {activeChapter.category || "Универс
```

```
 </div>
```

```
 <h2 className="text-base sm:text-xl f
```

```
 {activeChapter.title}
```

```
 </h2>
```

```
 </div>
```

```
 <ChapterNav prev={prev} next={next} ind
```

```
 </div>
```

```

 </footer>
 </div>
);
}

```

```
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
- const [activeTab, setActiveTab] = useState<"guide" | "simulator">();
+ const [activeTab, setActiveTab] = useState<"guide" | "simulator">();
 const [activeChapterId, setActiveChapterId] = useState<string>();
 const [modalVizId, setModalVizId] = useState<string>();

@@ -95,7 +96,7 @@ export default function App() {
 </main>
 </div>
 </>
-) : (
+) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
 <SimulatorHub
 onOpen={setModalVizId}
@@ -105,6 +106,8 @@ export default function App() {
 </main>
) : (
 <ChapterView chapterId={activeChapterId} />
)
}
```

```

ap-3 lg:gap-0">
<div className="flex items-center gap-3 w-full" >
 <div className="bg-gradient-to-tr from-indigo-400 to-indigo-600" >
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-slate-800">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-medium">
 Подготовка к экзаменам: Алгоритмы, Структуры данных
 </p>
 </div>
</div>

<div className="flex items-center w-full lg:w-auto" >
 >
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-center justify-center text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <CodeIcon className="w-4 h-4" /> Симулятор
 </button>
</div>

```

```

 <FileText className="w-4 h-4" /> TXT

 <button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center
er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
 </div>
</header>
);
};

```

### Patch относительно `main`

```

````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  import { chapters } from '../data/content';
  import { downloadBookHtml } from '../utils/exportHtml'

  interface NavbarProps {
-   activeTab: 'guide' | 'simulator';
-   setActiveTab: (tab: 'guide' | 'simulator') => void;
+   activeTab: 'guide' | 'simulator' | 'compiler';
+   setActiveTab: (tab: 'guide' | 'simulator' | 'compile

```

Полное содержимое после изменений

```
```tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в браузер
name = "алгоритмы"

for number in range(1, 4):
 print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL)
 .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_MODULE_URL }));
 }
 return runtimePromise;
}
```

```

 } catch (error) {
 const captured = [...stdout, ...stderr].join("");
 setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
 setRuntimeMessage("Не удалось выполнить код – пройдите по ссылке");
 } finally {
 setRunning(false);
 }
 }, [code, stdin, running, runtimeReady]);

```

```

const reset = () => {
 setCode(STARTER_CODE);
 setStdin("");
 setOutput("Нажмите «Запустить», чтобы увидеть результат");
};

```

```

return (
 <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
 <div className="max-w-6xl mx-auto">
 <div className="flex flex-col sm:flex-row sm:items-center">
 <div>
 <div className="inline-flex items-center gap-2">
 <Terminal className="w-4 h-4" /> Боковая панель
 </div>
 <h2 className="text-2xl sm:text-3xl font-extrabold">
 <p className="text-slate-400 mt-2 max-w-2xl">
 Пишите небольшой код и запускайте его прямо в браузере,
 который переводит CPython в WebAssembly.
 </p>
 </div>
 <a
 href="https://pyodide.org/"
 target="_blank"
 rel="noreferrer"
 className="inline-flex items-center gap-1.5"
 >
 Как это работает <ExternalLink className="w-4 h-4" />

 </div>
 </div>
 </div>
 </main>
);

```

```

 }
 }}
 spellCheck={false}
 aria-label="Код Python"
 className="block w-full min-h-[360px] res
 -none focus:ring-2 focus:ring-inset focus:r
 placeholder="Напишите Python-код..."
 />

 <div className="border-t border-slate-800 b
 <label className="block px-4 pt-3 text-[1
 Ввод для input() • по одной строке на к
 </label>
 <textarea
 id="python-stdin"
 value={stdin}
 onChange={(event) => setStdin(event.tar
 spellCheck={false}
 rows={2}
 placeholder={'Например:\nАлиса'}
 className="block w-full resize-y bg-tra
 eholder:text-slate-700"
 />
 </div>
</section>

<section className="rounded-2xl border border
 <div className="flex items-center justify-b
 <div className="flex items-center gap-2 t
 <Terminal className="w-4 h-4 text-emera
 </div>
 <span className={`inline-flex items-cente
 {runtimeReady && <CheckCircle2 classNam
 {runtimeReady ? "готов" : "не загружен"

 </div>
 <pre className="min-h-[360px] max-h-[560px]
 x] leading-6 text-slate-300">

```

```

+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/full/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/full/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python и выполнит код
+name = "алгоритмы"
+
+for number in range(1, 4):
+ print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+ runPythonAsync: (source: string) => Promise<unknown>;
+ setStdout: (options: { batched: (message: string) => void }) => void;
+ setStderr: (options: { batched: (message: string) => void }) => void;
+ setStdin: (options: { stdin: () => string | number | null }) => void;
+};
+
+type PyodideModule = {
+ loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = null;
+
+function getPythonRuntime() {
+ if (!runtimePromise) {
+ runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_URL, {
+ (module as PyodideModule).loadPyodide({ indexURL: PYODIDE_INDEX_URL })
+ });
+ }
+ return runtimePromise;
+}
+
+function errorText(error: unknown) {
+ if (error instanceof Error) return error.message;
+ return String(error);
+}
+
```



```

+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+ setCode(STARTER_CODE);
+ setStdin("");
+ setOutput("Нажмите «Запустить», чтобы увидеть резу.
+ });
+
+ return (
+ <main className="max-w-screen-2xl mx-auto px-4 sm:p-8">
+ <div className="max-w-6xl mx-auto">
+ <div className="flex flex-col sm:flex-row sm:items-center">
+ <div>
+ <div className="inline-flex items-center gap-2">
+ <Terminal className="w-4 h-4" /> Боковая панель
+ </div>
+ <h2 className="text-2xl sm:text-3xl font-extrabold">
+ <p className="text-slate-400 mt-2 max-w-2xl">
+ Пишите небольшой код и запускайте его прямо в браузере,
+ который переводит CPython в WebAssembly.
+ </p>
+ </div>
+ <a
+ href="https://pyodide.org/"
+ target="_blank"
+ rel="noreferrer"
+ className="inline-flex items-center gap-1.5"
+ >
+ Как это работает <ExternalLink className="text-slate-400" />
+
+ </div>
+
+ <div className="grid xl:grid-cols-[minmax(0,1fr),minmax(0,1fr)] gap-2">
+ <section className="rounded-2xl border border-slate-200 p-4">
+ <div className="flex flex-wrap items-center gap-2">
+ <div>
+ <div className="flex items-center gap-2">
+
+

```

```

+ />
+
+ <div className="border-t border-slate-800"
+ <label className="block px-4 pt-3 text-slate-700"
+ Ввод для input() • по одной строке на
+ </label>
+ <textarea
+ id="python-stdin"
+ value={stdin}
+ onChange={(event) => setStdin(event.target.value)}
+ spellCheck={false}
+ rows={2}
+ placeholder={'Например:\nАлиса'}
+ className="block w-full resize-y bg-tran
+placeholder:text-slate-700"
+ />
+ </div>
+ </section>
+
+ <section className="rounded-2xl border border-slate-800"
+ <div className="flex items-center justify-between"
+ <div className="flex items-center gap-2"
+ <Terminal className="w-4 h-4 text-emerald-500"
+ </div>
+ <span className={`inline-flex items-center gap-2`}
+ {runtimeReady && <CheckCircle2 className="text-emerald-500"
+ {runtimeReady ? "готов" : "не загружен"}
+
+ </div>
+ <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300">
+ {output}
+ </pre>
+ <div className="border-t border-slate-800"
+ </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3"

```

```

base: process.env.VITE_BASE || "/",
plugins: [react(), tailwindcss(), viteSingleFile()],
resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
},
server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
},
preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
},
});
\`\`\`

```

### Patch относительно `main`

```

\`\`\`diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],

```



```
 },
 "devDependencies": {
 "@tailwindcss/vite": "4.1.17",
 "@types/node": "22.19.17",
 "@types/pdfkit": "^0.17.6",
 "@types/react": "19.2.7",
 "@types/react-dom": "19.2.3",
 "@vitejs/plugin-react": "5.1.1",
 "esbuild": "^0.28.2",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
 },
 "description": "Интерактивное пособие по алгоритмам и структурам данных"
}
```

---

ФАЙЛ 7/88: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР: 1.2 KB

---

# algv0 – Универсальное пособие по алгоритмам и структурам данных

Интерактивная web-читалка (React + Vite + Tailwind) с поддержкой темной темы и автоматическим экспортом всего исходного кода в PDF.

## Быстрый старт

```
```bash
npm install
npm run dev          # предпросмотр на http://0.0.0.0:5173
npm run build        # сборка в dist/ (single-file)
```
```

## Экспорт кода: код → txt → png → pdf



## Универсальное пособие • полный исходный код

> Критерии:

> \* **Аналогия** — в HTML главы есть блок «Аналогия ...»

> \* **2D** — к главе привязан интерактивный визуализатор

>

> Всего глав в пособии: **25**.

---

**##** Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава | id | Что делать |
|-------|----|------------|
|-------|----|------------|

|     |     |     |
|-----|-----|-----|
| --- | --- | --- |
|-----|-----|-----|

|                    |           |                      |
|--------------------|-----------|----------------------|
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |
|--------------------|-----------|----------------------|

---

**##** Уровень 2. Темы, которых в пособии НЕТ ВООООЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет — и

| Билет | Тема | Статус |
|-------|------|--------|
|-------|------|--------|

|     |     |     |
|-----|-----|-----|
| --- | --- | --- |
|-----|-----|-----|

|          |                                |                               |
|----------|--------------------------------|-------------------------------|
| <b>1</b> | Дерево отрезков (Segment Tree) | Главы нет. Контент вхолостую. |
|----------|--------------------------------|-------------------------------|

|          |                                |                               |
|----------|--------------------------------|-------------------------------|
| <b>7</b> | Планарность и раскраска графов | Номерной главы нет. Аналогия. |
|----------|--------------------------------|-------------------------------|

|           |                |                                        |
|-----------|----------------|----------------------------------------|
| <b>11</b> | Мосты в графах | Номерной главы нет; есть без аналогии. |
|-----------|----------------|----------------------------------------|

|           |                       |                                        |
|-----------|-----------------------|----------------------------------------|
| <b>13</b> | Эйлеровы пути и циклы | Номерной главы нет; есть без аналогии. |
|-----------|-----------------------|----------------------------------------|

Дополнительно — «осиротевшие» визуализаторы без глав (контент есть)

\* `stack-dfs` → `StackViz.tsx` — **Стек и DFS**

\* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **Очередь**

\* `heap-beam-search` → `HeapViz.tsx` — **Куча и лучевой поиск**

\* `segment-trees` → `SegmentTreeVisualizer.tsx` — **Дерево отрезков**

\* `dynamic-programming` → `DPVisualizer.tsx` — **Динамическое программирование**

|    |                                               |   |   |   |   |  |
|----|-----------------------------------------------|---|---|---|---|--|
| 8  | 8. Графы. Компоненты связности                | 1 | – | – | – |  |
| 9  | 9. Графы. Топологическая сортировка           | 1 |   | – |   |  |
| 10 | 10. Графы. Компоненты сильной связности (SCC) |   |   |   |   |  |
| 12 | 12. Графы. Точки сочленения                   | 1 |   | – | – |  |
| 14 | 14. Графы. Кратчайший путь. Дейкстра          | 3 |   |   | – |  |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман      | 1 |   |   |   |  |
| 16 | 16. Графы. Кратчайший путь. Флойд             | – |   | – | – |  |
| 17 | 17. Графы. Алгоритм Джонсона                  | – |   | – | – |  |
| 18 | 18. Графы. Остовное дерево. Краскал           | 1 |   |   | – |  |
| 19 | 19. Графы. Остовное дерево. Прима             | 1 |   | – | – |  |
| 20 | 20. Графы. Остовное дерево. Борувка           | 1 |   |   | – |  |
| 21 | 21. Префикс-функция (π), КМП                  | 4 |   | – | – |  |
| 22 | 22. Z-функция                                 | 4 |   | – | – |  |
| 23 | 23. Алгоритм Ахо–Корасик                      | 2 |   | – | – |  |
| 24 | 24. Классы сложности, сведение задач          | 4 |   | – | – |  |
| –  | Обзор: Кратчайшие пути и Графы                | – |   | – | – |  |
| –  | Алгоритмы на карте                            | – | – | – | – |  |
| –  | Эйлер: Путь ≠ Цикл                            | – |   | – | – |  |
| –  | Мосты в графах: код + визуализация            | 1 |   | – | – |  |

## ## Рекомендуемый порядок работ

1. **\*\*Вернуть потерянные билеты 1, 7, 11, 13\*\*** и подключить ``segment-trees``, ``stack-dfs``, ``queue-bfs``, ``heap-be`` — это 5 готовых компонентов, которые сейчас просто не
2. **\*\*Дописать аналогии\*\*** в 5 главах из «Уровня 3» (по ш
3. **\*\*Добавить 2D\*\*** к 4 главам «Уровня 4» — минимум `inli`
4. **\*\*Решить судьбу `alg-mar`\*\*** — раскрыть или удалить.

ФАЙЛ 9/88: `tsconfig.json`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

{



```

import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локально
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
```

```

 }, []));

// Закрываем боковой компилятор при смене главы/таба
useEffect(() => {
 setSideOpen(false);
}, [activeChapterId, activeTab]);

// Стрелки ← → листают темы (как на обучающих сайтах)
useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLInputElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target)) return;
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 // не мешаем когда открыты модальки/дройеры
 if (sideOpen || modalVizId) return;
 if (e.key === "ArrowRight" && next) goTo(next.id);
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
}, [goTo, next, prev, sideOpen, modalVizId]);

const progress = useMemo(() => ((index + 1) / chapters.length) * 100);

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />
 </>
) : (
 <TableOfContents
 index={index}
 total={chapters.length}
 />
)}
 </Navbar>
 </div>
);

```

```

 onOpenSimulator={setModalVizId}
 onOpenCompiler={() => setSideOpen(true)}
 />
</main>

 {/* 50/50 док-панель – не чат, а панель кода */}
 {sideOpen && (
 <aside className="w-full lg:w-1/2 shrink-0 p-16 lg:h-[calc(100vh-4rem)] flex flex-col" >
 <CompilerPanelContent
 chapterId={activeChapter.id}
 onClose={() => setSideOpen(false)}
 onOpenFull={() => {
 setSideOpen(false);
 setActiveTab("compiler");
 window.scrollTo({ top: 0, behavior: "smooth" }})
 }}
 />
 </aside>
)}
</div>

 {/* Вертикальная вкладка – только когда панель открыта */}
 {!sideOpen && (
 <div className="hidden xl:flex fixed right-0 top-0" >
 <button
 onClick={() => setSideOpen(true)}
 className="writing-vertical bg-emerald-500 text-white p-4 shadow-emerald-900/30 flex flex-col items-center justify-center"
 style={{ writingMode: "vertical-rl", textOrientation: "mixed" }}
 title="Открыть компилятор 50/50 – слева"
 >
 <Terminal className="w-4 h-4 rotate-90" />
 КОМП
 50/50
 </button>
 </div>
)}

```

```
) : (
 <PythonCompiler chapterId={activeChapter.id} />
)}

 <SimulatorModal vizId={modalVizId} onClose={() =>

 <footer className="p-8 text-center text-slate-600"
 © 2026 Universal Educational Guide. Интерактивн
 </footer>
 </div>
);
}
```

---

ФАЙЛ 12/88: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 165 | РАЗМЕР: 7.5 KB

---

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 3;
const MAX_PER_TERM = 10;

const SKIP_SELECTOR = "code, pre, script, style, a, tex

const escapeRe = (s: string) => s.replace(/[.*+?^${}()|

// Проверяем, поддерживает ли движок lookbehind с \p{L}
let supportsLookbehind = true;
```

```

 if (parent.closest(SKIP_SELECTOR)) return NodeFilter.FILTER_SKIP;
 if (!node.nodeValue || node.nodeValue.trim().length === 0) return NodeFilter.FILTER_SKIP;
 return NodeFilter.FILTER_ACCEPT;
 },
 });

const targets: Text[] = [];
let current = walker.nextNode();
while (current) {
 targets.push(current as Text);
 current = walker.nextNode();
}

for (const textNode of targets) {
 const text = textNode.nodeValue ?? "";
 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 // Для фолбэка без lookbehind: группа 1 – раздели
 const hasPrefixGroup = !supportsLookbehind;
 const prefix = hasPrefixGroup ? m[1] ?? "" : "";
 const surfaceRaw = hasPrefixGroup ? m[2] : m[1];
 if (!surfaceRaw) continue;
 const surface = surfaceRaw.toLowerCase();
 const id = labelToId.get(surface);
 if (!id) continue;

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE) continue;
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);
 }
}

```

```

*/
export function useTermHints(ref: React.RefObject<HTMLElement>) {
 const run = () => {
 const el = ref.current;
 if (!el) return;
 try {
 annotateTerms(el);
 } catch {
 /* если браузер не умеет lookbehind – просто остаётся */
 }
 };

 // при смене главы
 // eslint-disable-next-line react-hooks/exhaustive-deps
 useEffect(run, deps);

 // страховка: контент перерисовали, а подсказок в нём нет
 useEffect(() => {
 const el = ref.current;
 if (!el) return;
 // Если подсказок нет, сразу пробуем
 if (!el.querySelector(".term-hint")) run();

 // MutationObserver – ловим перерисовку innerHTML и т.д.
 const obs = new MutationObserver(() => {
 if (!el.querySelector(".term-hint")) run();
 });
 obs.observe(el, { childList: true, subtree: true, characterData: true });
 return () => obs.disconnect();
 });
}

```

```
import { useEffect } from "react";

/**
 * Хелпер для синхронизации визуализации с боковым комп
 * Хардкод i,k,j,n,m – визуализация пушит текущие значе
 * а SideCompilerDrawer их ловит и подсвечивает.
 *
 * Использование в компоненте визуализации:
 * useVizSync(chapterId, { n, m, i, j, k })
 *
 * Или событие: window.dispatchEvent(new CustomEvent('v
 */
export function useVizSync(chapterId: string, vars: Record<string, string | number>) {
 useEffect(() => {
 const clean: Record<string, string | number> = {};
 for (const [k, v] of Object.entries(vars)) {
 if (v !== null && v !== undefined && v !== "") cl
 }
 // also expose via global for SideCompilerDrawer po
 // @ts-ignore
 window.__liveVizVars = clean;
 // @ts-ignore
 if (typeof window !== "undefined" && (window as unknown as { __compilerSyncHook: (v:
 compilerSyncHook) {
 try {
 (window as unknown as { __compilerSyncHook: (v:
 } catch {}
 }
 window.dispatchEvent(
 new CustomEvent("viz:sync", {
 detail: { chapterId, vars: clean },
 })
);
 }, [chapterId, JSON.stringify(vars)]);
}
```

```
 stroke: #fef08a;
 stroke-width: 8;
 filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9));
 }
}

.animate-pulse-gold {
 animation: pulse-gold 1.5s infinite;
}

@keyframes tocIn {
 from {
 transform: translateX(-100%);
 }
 to {
 transform: translateX(0);
 }
}

@keyframes hintIn {
 from {
 opacity: 0;
 transform: translateY(4px);
 }
 to {
 opacity: 1;
 transform: translateY(0);
 }
}

.term-popover {
 animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в Википедии)
.term-hint {
 cursor: help;
 border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
}
```



```
font-size: clamp(11px, 2.9vw, 13px);
}

.chapter-body table {
 display: block;
 width: 100%;
 overflow-x: auto;
 border-collapse: collapse;
 font-size: clamp(11px, 3vw, 14px);
}

.chapter-body summary {
 cursor: pointer;
 padding: 0.35rem 0;
}

@media (max-width: 767px) {
 .chapter-body :where(.grid) {
 grid-template-columns: minmax(0, 1fr) !important;
 }
 .chapter-body :where(h2) {
 font-size: 1.25rem !important;
 line-height: 1.3 !important;
 }
 .chapter-body :where(h3) {
 font-size: 1.05rem !important;
 line-height: 1.35 !important;
 }
 .chapter-body :where(.p-6, .p-8) {
 padding: 0.9rem !important;
 }
 .chapter-body :where(.text-3xl) {
 font-size: 1.4rem !important;
 }
 .chapter-body :where(.text-2xl) {
 font-size: 1.2rem !important;
 }
 .chapter-body :where(.text-xl) {
```

```
import './index.css';
import App from './App';

createRoot(document.getElementById("root")).render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

---

ФАЙЛ 17/88: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html" | "markdown";
 content: string;
 description?: string;
 category?: string;
}
```

---

ФАЙЛ 18/88: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

---

```
/// <reference types="vite/client" />
```

```
declare module '*.jpg' {
 const src: string;
 export default src;
}
declare module '*.jpg?url' {
 const src: string;
```

```

<h3 class="text-xl font-bold text-blue-400">
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ух отрезков: $\min(\text{ST}[L][k], \text{ST}[R - 2^k + 1][k])$
</div>
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Школьные линейки
 </div>
 <p class="text-slate-300 text-sm mb-4">
 зок длины 7. Ты берешь две заготовленные заготовки
 ь отрезок длины 7 (перекрывая друг друга пополам)
 ничего складывать, просто пересекаем куски.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: Ступени
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ужно покрыть отрезок, мы берем две самые большие
 </div>
 </div>
<details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky)][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2 - (1 << kx) + 1][C2 - (1 << ky)][kx][ky]);
int minimum = min(ans1, ans2);</pre>

```

```

 Аналогия 2: Слияние капель (Merge)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 вого). Мы просто смотрим на корни: у кого Y
 как потомок.</p>
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
pair<Node*, Node*> split(Node* t, int x) {
 if (!t) return {nullptr, nullptr};
 if (t->val <= x) {
 auto [L, R] = split(t->right, x);
 t->right = L;
 return {t, R};
 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
}</pre>
 </div>
</details>
</div>
</div>
</section>`,
},
{
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Дерево поиска, которое чинит само себя",
 category: "Продвинутые структуры",

```

```
</div>
```

```
<!-- 1. Зачем -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl border-2 border-emerald-500 shadow-md">
 <h3 class="text-lg font-bold text-white mb-4">Почему это не дерево?
 <p class="text-slate-300 text-sm mb-4">В обиходе мы привыкли думать, что дерево — это структура, в которой каждый элемент имеет одного или нескольких родителей. Но если мы будем добавлять элементы по возрастанию — и дерево вырождается в линейную структуру.
 <pre class="bg-slate-950 p-4 rounded-lg overflow-x-auto font-mono text-slate-300">вставили 10, 20, 30, 40, 50
```

```
10
```

```
 \
```

```
 20
```

```
 \
```

```
 30
```

```
 \
```

```
 40
```

```
 \
```

```
 50
```

высота = 5, поиск 50 = 5 сравнений  
это уже не дерево, а односвязный список

```
</pre>
```

```
<p class="text-slate-300 text-sm mt-4">Есть два типа структур данных: деревья и односвязные списки. В деревьях каждый элемент имеет одного или нескольких родителей, а в односвязных списках каждый элемент имеет только одного соседа. Если мы будем добавлять элементы по возрастанию, то дерево превратится в односвязный список. Это происходит потому, что каждый новый элемент добавляется в конец списка, а не в середину. В результате структура становится линейной, и поиск элемента требует времени O(n).
<ul class="list-disc list-inside text-slate-300">
 Дерево — это структура, в которой каждый элемент имеет одного или нескольких родителей. В деревьях каждый элемент имеет одного или нескольких родителей, а в односвязных списках каждый элемент имеет только одного соседа. Если мы будем добавлять элементы по возрастанию, то дерево превратится в односвязный список. Это происходит потому, что каждый новый элемент добавляется в конец списка, а не в середину. В результате структура становится линейной, и поиск элемента требует времени O(n).
 Односвязный список — это структура, в которой каждый элемент имеет только одного соседа. В односвязных списках каждый элемент имеет только одного соседа, а в деревьях каждый элемент имеет одного или нескольких родителей. Если мы будем добавлять элементы по возрастанию, то односвязный список превратится в дерево. Это происходит потому, что каждый новый элемент добавляется в середину списка, а не в конец. В результате структура становится нелинейной, и поиск элемента требует времени O(1).

```

```
</div>
```

```
<!-- 2. Аналогии -->
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border-2 border-emerald-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-slate-900 border-emerald-500 shadow-md">
 Аналогия 1: стопка бумаг на столе
```

```
</div>
```

```
<p class="text-slate-300 text-sm">У тебя есть стопка бумаг на столе. Если ты будешь добавлять новые листы в конец стопки, то она будет превращаться в односвязный список. Если же ты будешь добавлять новые листы в середину стопки, то она будет превращаться в дерево.
```

одно и то же место, поэтому В просто перев

</div>

</div>

</div>

<!-- 4. Три случая -->

<div class="bg-slate-800/60 p-6 rounded-xl border

<h3 class="text-lg font-bold text-white mb-3

<p class="text-slate-300 text-sm mb-4">Обоз

узел, который поднимаем, <span class="font

>g</span> – дед (родитель родителя). Смотри

<div class="overflow-x-auto -mx-2 px-2">

<table class="w-full text-xs sm:text-sm

<thead>

<tr class="text-left text-slate

<th class="py-2 pr-3 font-b

<th class="py-2 pr-3 font-b

<th class="py-2 pr-3 font-b

<th class="py-2 font-bold">

</tr>

</thead>

<tbody class="text-slate-300">

<tr class="border-b border-slate

<td class="py-3 pr-3 font-b

<td class="py-3 pr-3">деда

<td class="py-3 pr-3">сам х

<td class="py-3">х стал кор

</tr>

<tr class="border-b border-slate

<td class="py-3 pr-3 font-b

<td class="py-3 pr-3">х и п

аво-право)</td>

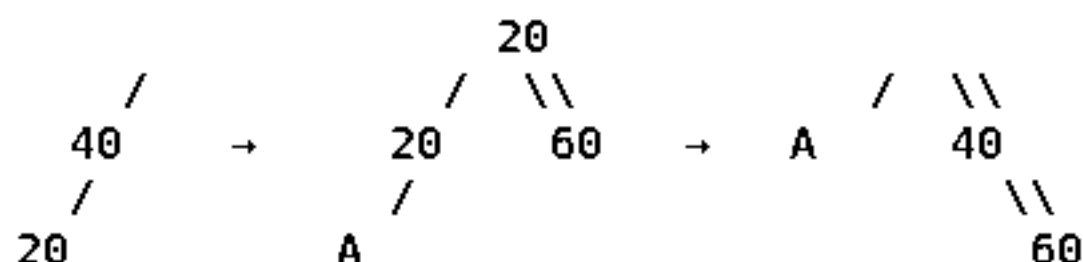
<td class="py-3 pr-3"><span

<td class="py-3">х поднялся

</tr>

<tr>

<td class="py-3 pr-3 font-b



было: линия из 3 узлов

стало: кустик высоты 2

глубина ветки упала вдвое!

```

 <p class="text-slate-400 text-xs mt-4">
 ем.</p>
 </div>
 </div>
 <p class="text-slate-300 text-sm mt-4">
 з, змейка (zig-zag) – крути СНИЗУ вверх.
 </p>
 </div>

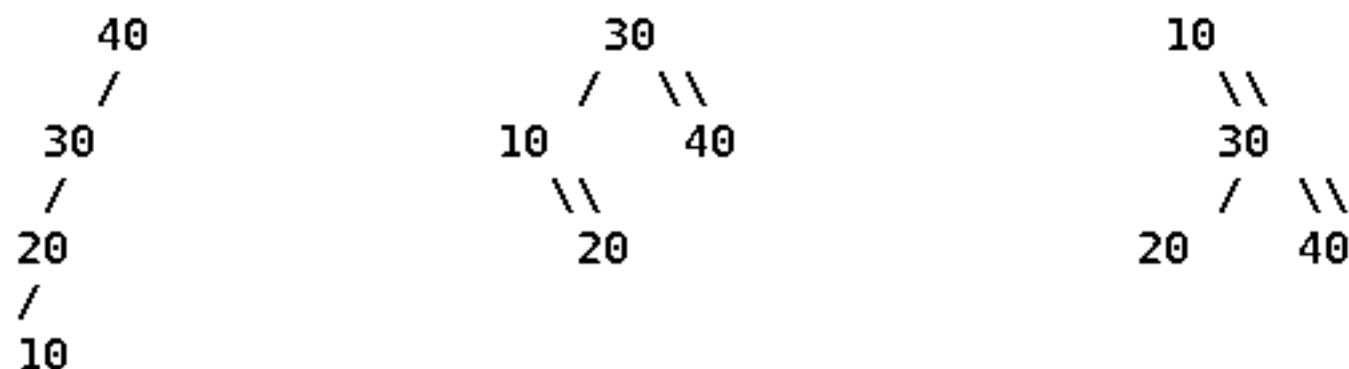
```

<!-- 6. Полный пример -->

```

<div class="bg-slate-800/60 p-6 rounded-xl border" style="border: 1px solid #000000;">
 <h3 class="text-lg font-bold text-white mb-4">Шаг 3
 <p class="text-slate-300 text-sm mb-4">Дерево
 3 шага, а потом делаем Splay(10).</p>
 <pre class="bg-slate-950 p-4 rounded-lg overflow-hidden" style="font-family: monospace; font-size: 0.9em;">
 0 leading-relaxed">старт после Zig-zag
 глубина 3 → 1 10 в корне

```



итог: было 4 узла в линию (высота 4) → стало высота 3,  
а сама десятка теперь в корне: следующий запрос к

```

 <p class="text-slate-300 text-sm mt-4">Обратим
 инили дерево по пути. Все узлы, мимо

```

```

 просто отрезаем ссылки.</div>

```

```

 <div class="bg-slate-900 rounded-lg p-3
k) — sp
/div>

```

```

 <div class="bg-slate-900 rounded-lg p-3
) — Spl
него не будет правого сына — туда и вешаем l
</div>

```

```

</div>

```

```

<!-- 9. Код -->

```

```

<details class="bg-slate-900/40 rounded-lg border
<summary class="p-4 font-bold text-slate-400
open:border-b border-slate-700/50">

```

```

 Код: rotate + splay + find (нажми, что
</summary>

```

```

<div class="p-5 text-sm text-slate-300 space
 <pre class="bg-slate-950 p-4 rounded ov
xed">// узел: { key, left, right, parent }

```

```

// один поворот: x встаёт на место своего родителя

```

```

function rotate(x) {

```

```

 const p = x.parent, g = p.parent;

```

```

 if (p.left === x) { // правый поворот

```

```

 p.left = x.right;

```

```

 if (x.right) x.right.parent = p;

```

```

 x.right = p;

```

```

 } else { // левый поворот (зеркал

```

```

 p.right = x.left;

```

```

 if (x.left) x.left.parent = p;

```

```

 x.left = p;

```

```

 }

```

```

 p.parent = x; // родитель уехал вниз

```

```

 x.parent = g; // x занял его место

```

```

 if (g) { if (g.left === p) g.left = x; else g.right =

```

```

}

```



```
<p class="text-emerald-300 font-bold mb-2">
<ul class="list-disc list-inside text-slate-400">
 Простой код: нет высот, цветов,
 Меньше памяти, чем у AVL и красных чертов
 Сам подстраивается под «горячие» данные
 split / merge получаются почти мгновенно

</div>
<div class="bg-rose-950/30 rounded-xl p-5 border border-rose-500">
 <p class="text-rose-300 font-bold mb-2">Важно!
 <ul class="list-disc list-inside text-slate-400">
 Нет гарантии на отдельную запись
 Не годится для реального времени
 Дерево меняется даже при чтении
 Постоянные записи в память при больших объемах

</div>
</div>

<!-- 11. Вопросы -->
<details class="bg-slate-900/40 rounded-lg border border-slate-700" open:
 <summary class="p-4 font-bold text-slate-400">
 open:border-b border-slate-700/50">
 Вопросы, которые задают на экзамене (иногда)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <div>
 <strong class="text-white block mb-2">Вопрос
 <p class="text-slate-400">Последний элемент (или преемник). Splay делают всегда, даже если
 элемент был в корневом узле, и амортизация сломалась бы.</p>
 </div>
 <div>
 <strong class="text-white block mb-2">Ответ
 <p class="text-slate-400">Потому что они не поворачивают бамбук в другую сторону (см. блок про AVL)
 </div>
 </div>
</div>
```

```

 <div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080; position: relative; margin-bottom: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080; position: absolute; top: -3px; left: 4px; right: 4px; bottom: 4px;">
 ≈ Аналогия 1: Вырезание прямоугольника из него маленький целевой прямоугольник в центральном верхнем углу отрезается дважды! Поэтому
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-mt-12" style="background-color: #f0f0f0; padding: 10px; border-radius: 10px;">
 <div class="flex items-center mb-6 flex-wrap gap-3" style="background-color: #e0e0e0; padding: 5px; border-radius: 10px;">
 7. Графы. Планарные. Покраска
 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400" style="color: #007bff; font-weight: bold; font-size: 1.5em; margin: 0;">7. Графы. Планарные. Покраска
 </div>
 <div class="space-y-8" style="margin-top: 10px;">
 <div class="bg-slate-700/50 p-6 rounded-xl border" style="background-color: #e0e0e0; padding: 10px; border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080;">
 <h3 class="text-xl font-bold text-blue-400" style="color: #007bff; font-weight: bold; font-size: 1.2em; margin-bottom: 10px;">7.1. Планарные графы
 <div class="bg-slate-900 p-4 rounded-lg mb-4" style="background-color: #1a202c; color: white; padding: 10px; border-radius: 10px; margin-bottom: 10px;">
 <p class="text-lg text-blue-300 italic" style="color: #007bff; font-style: italic; font-size: 1.1em; margin-bottom: 5px;">В любой планарный граф можно раскрасить 4 цветами.
 <p class="text-xl font-mono text-white" style="font-family: monospace; font-weight: bold; font-size: 1.2em; margin: 0;">В любой граф можно раскрасить 4 цветами.</p>
 </div>
 <div class="grid grid-cols-1 gap-6" style="margin-top: 10px;">
 <div class="bg-slate-800 p-6 rounded-lg" style="background-color: #1a202c; padding: 10px; border-radius: 10px; box-shadow: 5px 5px 0px #008080; position: relative;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080; position: absolute; top: -3px; left: 4px; right: 4px; bottom: 4px;">
```

```

 еская сортировка — это расписание, которое
 >
 </div>
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb-1">
получить опыт, нужна работа". Алгоритм выяв
 </div>
 </div>
 </div>
 </div>
 </section>`,
 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
 category: "Графы. Продвинутое",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
нвертированному в порядке top-sort.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg"

```

```

 </div>
 <p class="text-slate-300 text-sm mb-6">
 ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТСЯ ПОСТРОИТЬ.
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 description: "Вершины, удаление которых убивает связность графа.",
 category: "Графы. Продвинутое",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">12. Графы. Точки сочленения
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-rose-400">Точки сочленения
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">Вершины, удаление которых убивает связность графа.
 <p class="text-xl font-mono text-white">
 теи > 1).</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Центральный роутер
 </div>
 <p class="text-slate-300 text-sm mb-6">
 правильный роутер - и два сегмента офиса б
 </div>

```

```

 },
 {
 id: "pathfinding-accel",
 title: "17. Графы. Ускорения поиска",
 type: "html",
 description: "",
 category: "Графы. Пути",
 content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 17
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Графы. Пути
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
 <h3 class="text-xl font-bold text-blue-400">Ускорения поиска
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Аналогия: Копаем туннель под землей. Но если начать копать одновременно в 2 РАЗА меньше работы (геометрически: площадь
 <p class="text-xl font-mono text-white">д-длина.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под землей. Но если начать копать одновременно в 2 РАЗА меньше работы (геометрически: площадь
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ней земли. Но если начать копать одновременно в 2 РАЗА меньше работы (геометрически: площадь
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-kruskal",

```

```

 category: "Графы. Остовные",
 content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 order border-emerald-500 shadow-md">
 ✖ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">

```

```

</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-1"
 <p class="text-lg text-blue-300 italic"
 <p class="text-xl font-mono text-white"
i]), который одновременно является её суффикс
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Бумеранг (Префикс
 </div>
 <p class="text-slate-300 text-sm mb-1"
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
мя перепроверок!</p>
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate-300"
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounded"
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
 </details>

```

Код (Скрыто)

```

</summary>
<div class="p-5 text-sm text-slate-300" style="background-color: #f0f0f0; border: 1px solid #ccc; padding: 10px;">
 <pre class="bg-slate-950 p-4 rounded" style="background-color: #1a202c; color: #e2e3e5; padding: 10px;">
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}
 </pre>
</div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 description: "",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 24. Классы сложности, сведение задач

 <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">
 Сведение задач
 </h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-purple-400">
 Сведение (Reduction) A->B: Если я умею решать A, то я умею решать B
 </h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">Пример: Если я умею решать задачу A, то я умею решать задачу B.</p>
 <p class="text-xl font-mono text-white">Сведение (Reduction) A->B: Если я умею решать A, то я умею решать B.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="p-4 rounded">
 <!-- Аналогия 1 -->
 </div>
 </div>
 </div>
 </div>
`
 }
}

```



```

 desc: string;
}

export interface ChapterCompilerMeta {
 title: string;
 vizId?: string;
 /** Какие переменные есть у визуализации – только компилятор */
 vars: Record<string, CompilerVarInfo>;
 /** Доп. пояснение что синхронизируется */
 note: string;
 /** Какой визуализатор связан */
 synchint: string;
}

/**
 * Хардкод синхронизации i,k,j,n,m для каждой страницы.
 * Компилятор НЕ содержит исполняемого кода – только код визуализации.
 * Значения n,m,i,k,j – это то, что показывает визуализатор.
 * Если добавится новая страница – добавь сюда запись, и все.
 */
export const CHAPTER_COMPILER_META: Record<string, ChapterCompilerMeta> = {
 // билет 1
 "segment-trees": {
 title: "Дерево отрезков",
 vizId: "segment-trees",
 vars: {
 n: { label: "n", example: "6", desc: "длина массива" },
 m: { label: "m", example: "8", desc: "размер дерева" },
 i: { label: "i", example: "1", desc: "индекс вершины" },
 k: { label: "k", example: "0", desc: "левая граница" },
 j: { label: "j", example: "5", desc: "правая граница" },
 },
 note: "mid = (k+j)//2 – разбиение; запрос охватывает узлы v с отрезком [l,r]",
 synchint: "Визуализатор подсвечивает узлы v с отрезком [l,r]",
 },
 // билет 2
 "sparse-table": {

```

```

 synchint: "Визуализатор подсвечивает тройку i,k,j и
},
"splay-rotations": {
 title: "Splay повороты",
 vizId: "splay-rotations",
 vars: {
 n: { label: "n", example: "7", desc: "число узлов"
 i: { label: "i", example: "x", desc: "целевой узел"
 k: { label: "k", example: "p", desc: "родитель p"
 j: { label: "j", example: "g", desc: "дед g" },
 m: { label: "m", example: "-", desc: "-" },
 },
 note: "rotate(p) затем rotate(x) vs rotate(x) дважды
 synchint: "Покадровый разбор Zig/ZigZig/ZigZag",
},
// билет 5
"prefix-sums-2d": {
 title: "Префиксные суммы 1D/2D",
 vizId: "prefix-sums-2d",
 vars: {
 n: { label: "n", example: "4", desc: "высота матрицы"
 m: { label: "m", example: "4", desc: "ширина матрицы"
 i: { label: "i", example: "2", desc: "индекс строки"
 j: { label: "j", example: "2", desc: "индекс столбца"
 k: { label: "k", example: "r2,c2", desc: "угол записи"
 },
 note: "P[i][j]=A[i-1][j-1]+P[i-1][j]+P[i][j-1]-P[i-1][j-1]
 synchint: "Демо 1D: P[i]=a[0..i-1]; 2D: подсвечивает
},
// билет 6
"graph-dfs-bfs": {
 title: "DFS / BFS на графе",
 vizId: "graph-dfs-bfs",
 vars: {
 n: { label: "n", example: "6", desc: "число вершин"
 m: { label: "m", example: "7", desc: "число рёбер"
 i: { label: "i", example: "u", desc: "текущая вершина"
 j: { label: "j", example: "v", desc: "сосед v в графе"

```

```

 n: { label: "n", example: "6", desc: "V – вершины"
 m: { label: "m", example: "4", desc: "E – рёбра"
 i: { label: "i", example: "comp", desc: "индекс к
 j: { label: "j", example: "v", desc: "очередная н
 k: { label: "k", example: "-", desc: "—" },
 },
 note: "Пока есть непосещённая j – запускаем DFS/BFS
 synchint: "DFS раскрашивает каждую компоненту своим
},
// билет 9
"top-sort": {
 title: "Топологическая сортировка",
 vizId: "top-sort",
 vars: {
 n: { label: "n", example: "5", desc: "V – вершины"
 m: { label: "m", example: "6", desc: "E – дуги" }
 i: { label: "i", example: "u", desc: "текущая вер
 j: { label: "j", example: "v", desc: "сосед v (ис
 k: { label: "k", example: "order", desc: "позиция
 },
 note: "Kahn (indeg) или DFS-postorder + reverse: k -
 synchint: "Демо одежды: «трусы перед штанами»; подс
},
// билет 10
"scc-kosaraju": {
 title: "SCC – Косарайю",
 vizId: "scc-kosaraju",
 vars: {
 n: { label: "n", example: "6", desc: "V – вершины"
 m: { label: "m", example: "7", desc: "E – дуги" }
 i: { label: "i", example: "order", desc: "порядок
 j: { label: "j", example: "v", desc: "вершина v,
 k: { label: "k", example: "compId", desc: "номер
 },
 note: "Фаза1: DFS и стек order; Фаза2: DFS на GT в
 synchint: "Визуализатор показывает два прохода и ра
},
// билет 11 – мосты

```

```

 note: "cut $\Leftrightarrow$ (p!=-1 && low[to]>=tin[v]) || (p==-1 && low[to]<tin[v])",
 syncHint: "Визуализация подсвечивает центральную С",
},
// билет 13 – эйлер
"euler-path-vs-cycle": {
 title: "Эйлеров путь vs цикл",
 vizId: "euler-path-vs-cycle",
 vars: {
 n: { label: "n", example: "5", desc: "V" },
 m: { label: "m", example: "7", desc: "E" },
 i: { label: "i", example: "v", desc: "текущая вершина" },
 j: { label: "j", example: "deg(v)", desc: "степень вершины" },
 k: { label: "k", example: "cntOdd", desc: "k = количество нечётных степеней" },
 },
 note: "Эйлеров цикл $\Leftrightarrow$ все j чётные; путь $\Leftrightarrow$ k=2 (начало и конец)",
 syncHint: "Симулятор подсвечивает нечётные вершины",
},
"graph-euler": {
 title: "Эйлеров граф",
 vizId: "euler-path-vs-cycle",
 vars: {
 n: { label: "n", example: "5", desc: "V" },
 m: { label: "m", example: "7", desc: "E" },
 i: { label: "i", example: "v", desc: "v" },
 j: { label: "j", example: "deg", desc: "deg" },
 k: { label: "k", example: "odd", desc: "odd count" },
 },
 note: "same",
 syncHint: "Эйлеров обход",
},
// билет 14
dijkstra: {
 title: "Дейкстра",
 vizId: "dijkstra",
 vars: {
 n: { label: "n", example: "6", desc: "V – число вершин" },
 m: { label: "m", example: "9", desc: "E – число рёбер" },
 i: { label: "i", example: "u", desc: "извлечённая вершина" },
 },
 note: "Алгоритм Дейкстры",
 syncHint: "Симулятор подсвечивает вершину u и все вершины, расстояние до которых не больше, чем до u",
},

```

```

 vars: {
 n: { label: "n", example: "5", desc: "V" },
 m: { label: "m", example: "8", desc: "E" },
 i: { label: "i", example: "h", desc: "потенциал h" },
 j: { label: "j", example: "v", desc: "вершина v" },
 k: { label: "k", example: "u", desc: "ребро u→j" },
 },
 note: "Шаг1 Bellman из s → h; Шаг2 перевзвешивание;
 syncHint: "Визуализатор показывает h, перевзвешенную",
 },
 // билет 18
 "mst-kruskal": {
 title: "Краскал",
 vizId: "mst-kruskal",
 vars: {
 n: { label: "n", example: "5", desc: "V – вершины" },
 m: { label: "m", example: "7", desc: "E – рёбра" },
 i: { label: "i", example: "idx", desc: "индекс те" },
 j: { label: "j", example: "v", desc: "конец ребра" },
 k: { label: "k", example: "u", desc: "начало ребра" },
 },
 note: "сортировка $O(m \log m)$ + DSU; лес постепенно
 syncHint: "Подсвечивается очередное ребро i: k→j с",
 },
 // билет 19
 "mst-prim": {
 title: "Прим",
 vizId: "mst-prim",
 vars: {
 n: { label: "n", example: "5", desc: "V" },
 m: { label: "m", example: "7", desc: "E" },
 i: { label: "i", example: "u", desc: "последняя в" },
 j: { label: "j", example: "v", desc: "лучший канд" },
 k: { label: "k", example: "w", desc: " $w = \min \text{edge}$ " },
 },
 note: "дерево растёт как пятно: берём минимальное р
 syncHint: "Граф: дерево подсвечено, heap показывает",
 },

```

```

 },
 note: "окно [k,j] – самый правый блок; быстрый стар
 synchHint: "Лента подсвечивает окно [k..j] и сравнен
},
// билет 23
"aho-corasick": {
 title: "Ахо–Корасик",
 vizId: "aho-corasick",
 vars: {
 n: { label: "n", example: "4", desc: "число слов"},
 m: { label: "m", example: "11", desc: "длина текс"},
 i: { label: "i", example: "v", desc: "вершина бор"},
 j: { label: "j", example: "c", desc: "символ с →"},
 k: { label: "k", example: "link", desc: "суффиксн"},
 },
 note: "строим бор, затем BFS: очереди по k; go по f
 synchHint: "Бор + автомат + анимация водопада символа
},
// билет 24
"complexity-classes": {
 title: "Классы сложности",
 vizId: undefined,
 vars: {
 n: { label: "n", example: "n", desc: "размер вход"},
 m: { label: "m", example: "p(n)", desc: "полином"},
 i: { label: "i", example: "A", desc: "задача A, к"},
 j: { label: "j", example: "B", desc: "задача B, к"},
 k: { label: "k", example: "cert", desc: "сертифик"},
 },
 note: "P – детерминированно за poly; NP – проверка
 synchHint: "Нет граф-симуляции – показываем схему св
},
// вводные
intro: {
 title: "Введение",
 vizId: "intro",
 vars: {
 n: { label: "n", example: "6", desc: "V для превы"}
 }
}

```

```

 vars: {
 n: { label: "n", example: "5", desc: "глубина стека", },
 i: { label: "i", example: "top", desc: "верх стека", },
 j: { label: "j", example: "v", desc: "очередной vertex", },
 k: { label: "k", example: "-", desc: "-" },
 m: { label: "m", example: "-", desc: "-" },
 },
 note: "рекурсия DFS = стек вызовов",
 synchint: "Тарелки-стек и рекурсия DFS",
 },
 "queue-bfs": {
 title: "Очередь → BFS",
 vizId: "queue-bfs",
 vars: {
 n: { label: "n", example: "6", desc: "размер очереди", },
 i: { label: "i", example: "head", desc: "голова очереди", },
 j: { label: "j", example: "tail", desc: "хвост очереди", },
 k: { label: "k", example: "v", desc: "вершина v = queue[i]", },
 m: { label: "m", example: "-", desc: "-" },
 },
 note: "BFS: enqueue(s) → пока очередь не пуста: k=dequeue()",
 synchint: "Очередь в столовой и BFS волна",
 },
 "dynamic-programming": {
 title: "Динамика",
 vizId: "dynamic-programming",
 vars: {
 n: { label: "n", example: "10", desc: "размер DP-таблицы", },
 m: { label: "m", example: "10", desc: "M – второй массив", },
 i: { label: "i", example: "i", desc: "итерация по i", },
 j: { label: "j", example: "j", desc: "итерация по j", },
 k: { label: "k", example: "opt", desc: "k = argmin_{i,j} dp[i][j]", },
 },
 note: "заполняем таблицу по i,j; k – выбор предшествующего элемента",
 synchint: "Таблица DP построчно, подсветка переходов",
 },
};

```

```

type: "html",
content: `<section id="euler-path-vs-cycle" class="
<div class="flex items-center mb-6">
 <span class="bg-indigo-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">He by
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-blue-400
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-blue-300 italic
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
 <p><strong class="text-white">Эйлер
 <div class="p-3 bg-slate-950 rounde
 <p class="text-sm text-blue-300
 <p><strong class="text-white">П
 <p><strong class="text-white">Ц
 </div>
 </div>
 </div>
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Билет 13 (Различие пути и цикла

• <span class="text-green-400 font-
шел-вошёл-вышел).

• <span class="text-yellow-400 font
финиш).
 </p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">

```



```

 <p>
 Задача коммивояжёра (TSP) — это взв
 Доказано, что любой полиномиальный
 В общем, не парься. Просто запомни
 </p>
 </div>
</details>
</div>
</section>`,
 },
 {
 id: "bridges-code",
 title: "Мосты в графах: код + визуализация",
 type: "html",
 content: `<section id="bridges-code" class="mb-20 s
<div class="flex items-center mb-6">
 <span class="bg-emerald-600 text-white px-4 py-
 <h2 class="text-3xl font-bold text-white">Мосты
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

 <div class="bg-slate-900 p-4 rounded-lg mb-1
 <p class="text-lg text-emerald-300 ital
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">tin[v
 <p><strong class="text-white">low[v
 <div class="p-3 bg-slate-950 rounde
 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-40
 </div>
 </div>
 </div>
</div>

 <p class="text-slate-300 text-sm">

```

```
<div class="p-5 text-sm text-slate-300 space-x-4">
 <p>Классическая реализация – <code class="text-slate-300">
 оходов.</p>
</div>
```

```
<div class="bg-slate-950 p-4 rounded-lg">
 <pre class="text-xs font-mono text-slate-300">
```

```
#include <vector>
#include <algorithm>

using namespace std;

int n; // число вершин
vector<vector<int>>> adj; // список смежностей
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++; // устанавливаем время

 for (int to : adj[v]) {
 if (to == p) continue; // не идём назад

 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный вызов
 low[v] = min(low[v], low[to]);

 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
 }
}
```

```

 <p><span class="text-emerald-400 font-
 <p><span class="text-rose-400 font-l
 </div>
</div>

<p class="text-slate-300 text-sm">
 Следствие: Для планарн
ode>.
 Если ребер больше – граф гарантированно
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 K5: Полный граф на 5 вершинах
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У K5: <code class="text-white">V=5,
-white">10 > 9</code> – БАХ, непланарен!
 Все 5 вершин соединены со всеми. На
 Теорема Куратовского: K5 – эталон н
 Если внутри графа спрятан K5 – забудь
 </p>
 </div>

<div class="bg-slate-900 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
 K3,3: 3 дома и 3 колодца
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Три дома хотят соединиться с тремя
 Нарисовать без пересечений невозможно
 У него <code class="text-white">V=6
xt-white">9 ≤ 12</code> – проходит, но он в
 Почему? Потому что у двудольного гр
```

```
<h3 class="text-xl font-bold text-rose-400">
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
<p class="text-lg text-rose-300 italic">
```

```
<div class="text-left font-mono text-sm">
```

```
<p><strong class="text-white">Точка
```

```
рой увеличивает число компонент связности.</p>
```

```
<div class="p-3 bg-slate-950 rounded">
```

```

```

```
<p class="text-xl font-bold tex">
```

```
<p class="text-xs text-slate-40">
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<p class="text-slate-300 text-sm">
```

```
Важно: Это работает то
еются к точкам сочленения. То есть,
ег - это тупик со степенью 1).
```

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap">
```

```
<div class="bg-slate-800 p-6 rounded-lg bor">
```

```
<div class="absolute -top-3 left-4 bg-s
rder-rose-500 shadow-md">
```

Аналогия: Главный перекресток

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

```
Представь город, где два района сое
её перекроют (удалят вершину) – районы буд
>хрупкость системы.
```

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg bor">
```

```
<div class="absolute -top-3 left-4 bg-em
er border-emerald-500 shadow-md">
```

```

 }
 }
 if (p == -1 && children > 1)
 IS_CUTPOINT(v);
}

```

```

 </div>
 </div>
</details>

```

```

<div class="bg-indigo-950/60 p-6 rounded-xl border"
 <h4 class="text-lg font-bold text-indigo-400"
 <p class="text-slate-300 text-sm mb-4">
 Это глубокий дуальный мост между ориентиром
 </p>
 <ul class="list-disc list-inside text-sm text-slate-300">
 <strong class="text-rose-400">Точки
 onг> и показывают физическую уязвимость связности.
 <strong class="text-emerald-400">SCC-зависимые "конгломераты".
 Как точка сочленения рушит связность?
 Косарайю, свернув каждую SCC в одну супер-вершину (сделаем неориентированным), то любая вершина
 о и есть критическая SCC! Если удалить эту вершину, то
 . Одно слабое звено изолирует целые касты SCC.

</div>
</div>
</section>`,
 },
];

```

ФАЙЛ 22/88: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 51 | РАЗМЕР: 1.2 KB

```
import { Chapter } from "../types";
```

```
});
```

```
export const chapters: Chapter[] = finalChapters;
```

```
export const activeVizIds = [
 "euler-path-vs-cycle", "bridges-code", "planarity-euler",
 "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se",
 "intro", "dijkstra", "bellman-ford", "floyd", "aho-corasi",
 "mst-kruskal", "mst-prima", "mst-boruvka", "string-kmp",
 "string-z-func", "segment-trees", "sparse-table", "dynam",
];
```

---

ФАЙЛ 23/88: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРOK: 626 | РАЗМ

---

```
import type { DemoKind } from "../components/hints/MiniI
```

```
/**
```

```
 * Глоссарий «википедийных» подсказок.
```

```
 *
```

```
 * Любое слово из `labels`, встреченное в тексте главы,
```

```
 * в подчёркнутый термин: при наведении (или тапе на мо
```

```
 * с объяснением на пальцах и мини-визуализацией.
```

```
 */
```

```
export interface GlossaryEntry {
```

```
 id: string;
```

```
 /** Варианты написания в тексте (регистр не важен). */
```

```
 labels: string[];
```

```
 title: string;
```

```
 /** Объяснение «на пальцах» – 1–2 фразы. */
```

```
 short: string;
```

```
 /** Строгая формулировка / что важно не забыть. */
```

```
 formal?: string;
```

```
 complexity?: string;
```

```
 /** Какая мини-анимация показывается в карточке. */
```

```

 "Полное бинарное дерево в массиве: дети узла i ле
complexity: "вставка/извлечение $O(\log n)$, «посмотре
demo: "heap",
simulator: "heap-beam-search",
},
{
 id: "stack",
 labels: ["стек", "стека", "стеке", "стеком", "LIFO"],
 title: "Стек (LIFO)",
 short: "Стопка тарелок: кладёшь сверху и берёшь свер
 formal: "push / pop / top за $O(1)$. На стеке живёт р
 demo: "stack",
 simulator: "stack-dfs",
},
{
 id: "queue",
 labels: ["очередь", "очереди", "очередью", "FIFO"],
 title: "Очередь (FIFO)",
 short: "Очередь в столовой: кто первым встал, тот п
 formal: "push_back / pop_front за $O(1)$. Основа BFS
 demo: "queue",
 simulator: "queue-bfs",
},
{
 id: "dsu",
 labels: ["DSU", "CHM", "система непересекающихся мн
 title: "DSU – система непересекающихся множеств",
 short:
 "«Кто твой староста?» Каждый элемент знает своего
 formal:
 "find со сжатием путей + union по рангу/размеру.
 complexity: "почти $O(1)$ – обратная функция Аккерман
 demo: "dsu",
 simulator: "mst-kruskal",
},
{
 id: "trie",
 labels: ["бор", "бора", "боре", "бору", "бором", "t

```

```

 строки.",
 formal: "link(v) = go(link(parent(v)), ch). Считает
demo: "suffix-link",
simulator: "aho-corasick",
},
{
 id: "toposort",
 labels: ["топологическая сортировка", "топсорт", "топосорт"],
 title: "Топологическая сортировка",
 short:
 "Порядок «сначала штаны, потом ботинки»: каждая с
 циклов.",
 formal: "DFS + вершины в обратном порядке выхода, л
 complexity: "O(V + E)",
 demo: "toposort",
 simulator: "top-sort",
},
{
 id: "relaxation",
 labels: ["релаксация", "релаксацию", "релаксации",
 title: "Релаксация ребра",
 short:
 "«А через соседа не быстрее?» Если известный путь
 ,
 formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)
 ан и Флойд.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "prefix-sum",
 labels: ["префиксные суммы", "префиксная сумма", "префикс"],
 title: "Префиксные суммы",
 short:
 "Заранее посчитанные «сколько всего набежало от н
 formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r]-P[l-1]
 complexity: "препроцессинг O(n), запрос O(1)",
 demo: "prefix-sum",

```



```

 simulator: "bridges-code",
 },
 {
 id: "articulation",
 labels: ["точка сочленения", "точки сочленения", "т"],
 title: "Точка сочленения",
 short:
 "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
 formal: "Для не-корня: v — точка сочленения $\Leftrightarrow$ ребро (v, w) —",
 demo: "articulation",
 simulator: "graph-articulation",
 },
 {
 id: "mst",
 labels: [
 "остовное дерево", "остовного дерева", "остовное",
 "минимальное остовное дерево", "MST",
],
 title: "Минимальное остовное дерево (MST)",
 short:
 "Связать все точки проводами так, чтобы суммарная длина была",
 formal: " $V-1$ ребро, никаких циклов. Краскал (жадно) и Прима (не",
 "жадно).",
 demo: "mst",
 simulator: "mst-kruskal",
 },
 {
 id: "amortized",
 labels: [
 "амортизированная", "амортизированный", "амортизирован",
 "амортизированное", "амортизированного", "амортизирован",
],
 title: "Амортизированная сложность",
 short:
 "Средняя цена операции «по абонементу»: одна операция",
 formal: "Метод потенциалов: дорогая операция оплачивается",
 "амортизационно.",
 demo: "amortized",
 },

```

```

 labels: [
 "префикс-функция", "префикс-функции", "префикс-функц",
 "префикс", "префикса", "суффиксом", "суффикс", "с",
 "подстроки", "подстроку", "подстрока",
],
 title: "Префикс-функция π (КМП)",
 short:
 "Для каждой позиции запоминаем: какой кусок начался",
 "от кусок.",
 formal:
 " $\pi[i]$ — длина наибольшего собственного префикса s ",
 "возвращая указатель по тексту.",
 complexity: "O(n + m)",
 demo: "prefix-function",
 simulator: "string-kmp",
},
{
 id: "z-function",
 labels: ["Z-функция", "Z-функции", "Z-функцию", "z-функ",
 title: "Z-функция",
 short:
 "Для каждой позиции: сколько символов подряд, начинающихся",
 "с каждой буквы.",
 formal:
 " $z[i]$ = длина наибольшего общего префикса s и $s[i..n]$ ",
 complexity: "O(n)",
 demo: "prefix-function",
 simulator: "string-z-func",
},
{
 id: "dp",
 labels: [
 "динамическое программирование", "динамики", "динамическое",
 "мемоизация", "мемоизацию", "подзадач", "подзадачу",
],
 title: "Динамическое программирование",
 short:
 "Решаем задачу через ответы на её меньшие копии и"
```

```

 formal:
 "Корректность обычно доказывается «леммой о разре
demo: "mst",
simulator: "mst-kruskal",
},
{
 id: "euler",
 labels: ["эйлеров цикл", "эйлеров путь", "эйлерова
title: "Эйлеров путь и цикл",
short:
 "Пройти по каждому ребру ровно один раз, не отрыв
formal:
 "Связный граф: эйлеров цикл все степени чётные;
simulator: "euler-path-vs-cycle",
},
{
 id: "planarity",
 labels: ["планарный", "планарность", "планарного",
title: "Планарность",
short:
 "Граф планарен, если его можно нарисовать на листе
 ,
formal:
 " $V - E + F = 2$ для связного планарного графа. Кур
 ,
simulator: "planarity-euler-formula",
},
{
 id: "splay",
 labels: [
 "splay", "splay-дерево", "AVL", "AVL-дерево", "вс
],
title: "Splay и повороты",
short:
 "Каждый раз, когда к узлу обратились, его «вытягив
formal:
 "Три случая: zig (родитель – корень), zig-zig (уз
 рацию.",

```

```

 },
 {
 id: "floyd",
 labels: [
 "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уорше",
 "промежуточная вершина", "промежуточную вершину",
],
 title: "Флойд-Уоршелл: разрешаем вершину k",
 short:
 "Внешний цикл – это не «шаг», а «какие пересадки

 шить пересадку в k?",
 formal:
 " $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$. Порядок

 ами.",
 complexity: "O(V³) времени, O(V²) памяти",
 demo: "floyd",
 simulator: "floyd",
 },
 {
 id: "prim",
 labels: ["Прима", "Прим", "Prim"],
 title: "Алгоритм Прима",
 short:
 "Остов растёт как плесень из одной точки: на каждо",
 formal:
 "Множество посещённых + куча рёбер на границе. До",
 усок.",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst-prima",
 },
 {
 id: "boruvka",
 labels: ["Борувка", "Борувки", "Борувку", "Boruvka"],
 title: "Алгоритм Борувки",
 short:
 "Все компоненты одновременно шлют «гонца» к ближай",
 formal:

```

```

],
 title: "Граф",
 short:
 "Точки (вершины) и связи между ними (рёбра). Города и дороги",
 formal:
 "G = (V, E). Ребро бывает ориентированным (улицы с односторонним движением) и неориентированным (улицы с двусторонним движением).",
 demo: "graph",
 simulator: "graph-dfs-bfs",
},
{
 id: "coord-compress",
 labels: ["сжатие координат", "сжатия координат", "сжатие координат"],
 title: "Сжатие координат",
 short:
 "Координаты бывают гигантские и дробные, а нам нужно хранить их в виде целых чисел. Решение: сортировать по порядку и хранить только порядковый номер.",
 formal:
 "sort + unique + binary search: значение → индекс",
 complexity: "O(n log n)",
 demo: "coord-compress",
},
{
 id: "treap",
 labels: ["Treap", "декартово дерево", "декартова д-во"],
 title: "Treap = Tree + Heap",
 short:
 "Одно дерево живёт по двум правилам сразу: по ключу и по приоритету. Случайность и держит его сбалансированным.",
 formal:
 "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2) (соединить два дерева).",
 complexity: "ожидаемо O(log n)",
 demo: "heap",
},
{
 id: "bst",
 labels: ["бинарное дерево поиска", "дерево поиска", "дерево бинарного поиска"],

```

```

 title: "Запрос на отрезке",
 short:
 "Спрашиваем не про один элемент, а про кусок массива и
 предподсчёты.",
 formal:
 "Суммы — префиксные суммы $O(1)$. Минимум без изменений.",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "string",
 labels: ["строки", "строке", "строку", "строка", "строки"],
 title: "Строка как массив символов",
 short:
 "Строковые алгоритмы держатся на трёх вопросах: где встречается
 (Z-функция) и где встречаются слова словаря (бор).",
 formal:
 "Наивный поиск подстроки — $O(N \cdot M)$. Префикс-функция.",
 demo: "prefix-function",
 simulator: "string-kmp",
},
{
 id: "rotation",
 labels: ["поворот", "повороты", "поворота", "повороты"],
 title: "Поворот (rotation) в дереве",
 short:
 "Локальная перевеска двух узлов: ребёнок встаёт на место
 дившемуся месту. Порядок ключей при этом не ломается.",
 formal:
 "Правый поворот вокруг p: $x = p.left; p.left = x$.
 g и Zig-Zag.",
 complexity: " $O(1)$ ",
 demo: "zig",
 simulator: "splay-rotations",
},
{
 id: "zig",
 labels: ["Zig", "Zag", "зиг"],

```

```
e.labels.map((label) => ({ label, id: e.id })))
).sort((a, b) => b.label.length - a.label.length);
```

---

ФАЙЛ 24/88: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 275 | РАЗМ

---

```
import { Chapter } from "../types";
```

```
export const graphChapters: Chapter[] = [
 {
 id: "intro",
 title: "Обзор: Кратчайшие пути и Графы",
 type: "html",
 category: "Графы",
 content: `<section id="intro-content" class="mb-12">
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Обзор
 </div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <p class="text-slate-300 text-sm mb-4">Пред
 оезда). Алгоритмы поиска кратчайшего пути п
 <div class="mt-8 mb-4">
 <div id="slot-mnemonic-cards"></div>
 </div>
 </div>
</div>
</section>`
 },
 {
 id: "dijkstra",
 title: "Билет 14. Дейкстра",
 type: "html",
```

```

 <details class="bg-slate-900/40 rounded-lg border
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Строгое доказательство (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <p>Алгоритм использует приоритетную
 g V). Гарантирует корректность только для n
 </div>
 </details>

 <div id="slot-dijkstra-viz" class="mt-8"></div>
 </div>
 </div>
 </section>`
 },
 {
 id: "bellman-ford",
 title: "Билет 15. Форд-Беллман",
 type: "html",
 category: "Графы",
 content: `<section id="bellman-ford-content" class="
 <div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 rounded
 <h2 class="text-3xl font-bold text-white">Форд-Беллман
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-rose-400">Алгоритм Форда-Беллмана

 <div class="bg-slate-900 p-4 rounded-lg mb-6">
 <p class="text-lg text-rose-300 italic">Алгоритм Форда-Беллмана
 <p class="text-xl font-mono text-white">Алгоритм Форда-Беллмана
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">

```



```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-indigo-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Автомагистрали
 </div>
 <p class="text-slate-300 text-sm mb-4">
 евле долететь из Москвы в Париж, если мы сд
 <div id="slot-chapter-image-floyd"
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Время работы
 </div>
 <p class="text-slate-300 text-sm mb-4">
 три вложенных цикла. Сложность $O(V^3)$. Если
 </div>
 </div>

 <div id="slot-floyd-viz" class="mt-8"></div>
 </div>
 </div>
</section>`
},
{

```

## Аналогия 2: Матёшка (Терминальные)

```

</div>
<p class="text-slate-300 text-sm mb-4">Представ
нашли и "he", потому что оно спрятано внутри
минальные ссылки (term_link) – прямые мос
p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg border b
<summary class="p-4 font-bold text-slate-400 outl
pen:border-slate-700/50">
 Внутренняя структура автомата и код BFS (Скры
</summary>
<div class="p-5 text-sm text-slate-300 space-y-4">
 <p>Каждая вершина имеет таблицу переходов <code>
 ссылку <code>term_link</code> (ближайшая мен
 <div class="bg-black/50 p-4 rounded-lg font-mon
struct Node {
 map<char, int> go; // Предвычисленные пер
 int link = 0; // Суффиксная ссылка: куда
 int term_link = 0; // Терминальная ссылка: матр
 bool is_terminal = false;
 vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные
void buildLinks() {
 queue<int> q;
 // Корни и их дети
 for (auto const& [ch, child] : nodes[0].trie) {
 nodes[child].link = 0;
 q.push(child);
 }

 while (!q.empty()) {

```

```

 ут быть большими дробными числами (долгота
 ву от 0 до N, и уже по ним строим графы.</p>
 </div>
</div>
</section>`
 }
};

```

ФАЙЛ 25/88: src/data/quizzes.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 106 | РАЗМ:

```
export interface QuizQuestion {
 question: string;
 options: string[];
 correctIndex: number;
 explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> =
 "euler-path-vs-cycle": [
 {
 question: "В связном графе ровно 2 вершины с нечётными степенями",
 options: [
 "Есть эйлеров цикл (вернись домой!)",
 "Есть эйлеров путь (из одной нечётной в другую)",
 "Ни пути, ни цикла нет — это полный провал"
],
 correctIndex: 1,
 explanation: "Если нечётных вершин ровно 2 — это эйлеров путь. Если нечётных больше, то ни пути, ни цикла нет."
 },
 {
 question: "Почему гамильтонов цикл — это NP-полная задача?",
 options: [
 "Потому что Гамильтон — болезнь, и лечить её тяжело",
 "Потому что Гамильтон — это NP-полная задача",
 "Потому что Гамильтон — это NP-полная задача"
],
 correctIndex: 1,
 explanation: "Потому что Гамильтон — это NP-полная задача"
 }
]
```

```

 explanation: "low[v] хранит минимальное время входа в вершину v при поиске мостов и точек сочленения.",
 },
 {
 question: "Какова временная сложность алгоритма поиска мостов?",
 options: [
 "O(V^2) – ищем мосты в квадрате",
 "O(V + E) – каждый элемент один раз",
 "O(2^V) – полный перебор"
],
 correctIndex: 1,
 explanation: "DFS обходит каждую вершину и каждое ребро.",
 }
],
"planarity-euler-formula": [
 {
 question: "Планарный граф имеет V=6, E=12. Возможно ли?",
 options: [
 "Да, если это двудольный граф",
 "Нет, нарушается неравенство E ≤ 3V - 6",
 "Да, если нет треугольных граней"
],
 correctIndex: 1,
 explanation: "E ≤ 3V - 6 ⇒ 12 ≤ 12 – на грани возможности. Так что такой граф непланарен.",
 },
 {
 question: "Сколько граней у планарного графа с V=7, E=10?",
 options: [
 "3 грани",
 "5 граней",
 "10 граней"
],
 correctIndex: 1,
 explanation: "F = E - V + 2 = 10 - 7 + 2 = 5. Не 10.",
 },
 {
 question: "Почему K5 непланарен?",
 }
]

```

```
<div id="intro" class="bg-slate-700/50 p-6 rounded-2xl" style="background-color: #2d3748; color: #2d3748; border-radius: 12px; padding: 10px 10px 10px 10px; margin-bottom: 10px; text-align: center;"><h3 class="text-xl font-bold text-blue-400">
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4" style="background-color: #1a202c; padding: 10px 10px 10px 10px; margin-bottom: 10px; border-radius: 8px;"><p class="text-lg text-blue-300 italic">Привет, я пишу вам письмо, чтобы сообщить вам о предстоящем обновлении (lazy).</p></div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2" style="display: grid; grid-template-columns: 1fr 1fr;"><div class="bg-slate-800 p-6 rounded-lg" style="background-color: #2d3748; padding: 10px 10px 10px 10px; border-radius: 8px; margin-bottom: 10px;"><div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 2px solid #2ecc71; border-radius: 4px; margin: -3px -4px 4px -4px; position: relative;"></div>
```

```
Аналогия 1: Корпоративная презентация</div><p class="text-slate-300 text-sm mb-4">Если бы я рассылать 10 000 писем, он пишет одно письмо в день. В счетах сотрудников только тогда, когда сотрудник обновленное обновление).</p></div>
```

```
<div class="bg-slate-900 p-6 rounded-lg" style="background-color: #1a202c; padding: 10px 10px 10px 10px; border-radius: 8px; margin-bottom: 10px;"><div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md" style="border: 2px solid #e91e63; border-radius: 4px; margin: -3px -4px 4px -4px; position: relative;"></div>
```

```
Аналогия 2: Матрешки-коробки</div><p class="text-slate-300 text-sm mb-4">Внутри всё синее!</p></div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg" style="background-color: #1a202c; border-radius: 8px; padding: 10px 10px 10px 10px; margin-bottom: 10px; border: 1px solid #2d3748;"><summary class="p-4 font-bold text-slate-300" style="padding: 10px 10px 10px 10px; border-bottom: 1px solid #2d3748; margin-bottom: 10px; font-weight: bold; color: #a6c1ee;"><b border-slate-700/50">
```

```
Строгое доказательство / Код (Скрытие)</summary>
```

```
<div class="p-5 text-sm text-slate-300" style="padding: 10px 10px 10px 10px; color: #a6c1ee;"></div>
```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 Аналогия 1: Школьные линейки
 </div>
 <p class="text-slate-300 text-sm mb-4" style="color: #a0c0ff; font-size: 0.9em; margin-bottom: 10px;">
 отрезок длины 7. Ты берешь две заготовленные заготовки, берешь отрезок длины 7 (перекрывая друг друга по 1 см), ничего складывать, просто пересекаем куски.
 </p>
</div>
<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 Аналогия 2: Ступени
 </div>
 <p class="text-slate-300 text-sm mb-4" style="color: #a0c0ff; font-size: 0.9em; margin-bottom: 10px;">
 нужно покрыть отрезок, мы берем две самые большие ступени
 </p>
</div>
<details class="bg-slate-900/40 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <summary class="p-4 font-bold text-slate-300" style="color: #a0c0ff; font-weight: bold; padding: 5px 10px;">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300" style="padding: 10px 10px 0 10px; color: #a0c0ff; font-size: 0.9em;">
 <pre class="bg-slate-950 p-4 rounded" style="background-color: #000; color: #fff; padding: 10px; font-family: monospace; font-size: 0.8em;">
int k = log2(R - L + 1);
int minimum = min(st[L][k], st[R - (1 << k) + 1][k]);</pre>
 </div>
 </details>
</div>
</div>
</section>`
},
{
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",

```

Код (Скрыто)

```

</summary>
<div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded-lg">
pair<Node*, Node*> split(Node* t, int x) {
 if (!t) return {nullptr, nullptr};
 if (t->val <= x) {
 auto [L, R] = split(t->right, x);
 t->right = L;
 return {t, R};
 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
}</pre>
</div>
</details>
</div>
</section>`
 },
 {
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Дерево поиска, которое самобалансируется",
 category: "Продвинутые структуры",
 content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">

```

```

 p.right = x.left;
 x.left = p;
 }
} </pre>

 </div>
</details>

<div class="bg-indigo-950/60 p-6 rounded-xl">
 <h4 class="text-lg font-bold text-indigo-400">
 Поиск в бинарном дереве

 <div class="space-y-4 text-sm text-slate-300">
 <div>
 <strong class="text-white block">
 Почему это важно?

 <p>Оно поднимет узел, на котором мы находимся, до корня. За счет сдвига этого листа в корень, дерево становится более сбалансированным. Это улучшает время поиска. /p>
 </div>
 <div>
 <strong class="text-white block">
 Как это работает?

 <p>Если агент постоянно переключается между узлами, находящимися на разных концах дерева, его можно представить как вытянутый бамбук для другого. Постоянное свинг-переключение превратит палочку в более сбалансированную структуру. /p>
 </div>
 <div>
 <strong class="text-white block">
 Почему это важно?

 <p>Хотя один поиск может стоить дорого, деревья поиска обладают прекрасным свойством: они не требуют много памяти по пути. "Палочное" дерево прессуется в сбалансированное дерево. /p>
 </div>
 </div>
</div>
</div>
</div>
</div>

```



---

ФАЙЛ 27/88: src/data/tickets/ticket14\_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 248 | РАЗМ

---

```
import { Chapter } from '../..types';
```

```
export const tickets14to20: Chapter[] = [
```

```
{
```

```
 id: "dijkstra",
```

```
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра
```

```
 type: "html",
```

```
 category: "Графы. Пути",
```

```
 content: `
```

```
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
```

```
 <div class="flex items-center mb-6 flex-wrap gap-3">
```

```

```

```
 <h2 class="text-2xl sm:text-3xl font-bold text-white">
```

```
 </div>
```

```
 <div class="space-y-8">
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border">
```

```
 <h3 class="text-xl font-bold text-emerald-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
 <p class="text-lg text-emerald-300 italic">
```

```
 <p class="text-xl font-mono text-white">
```

```
 </div>
```

```
 <div class="grid grid-cols-1 xl:grid-cols-2">
```

```
 <!-- Аналогия 1 -->
```

```
 <div class="bg-slate-800 p-6 rounded-lg">
```

```
 <div class="absolute -top-3 left-4 border"
```

```
 border-emerald-500 shadow-md">
```

```
 Аналогия 1: Позитивное планирование
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 (нельзя поехать и заработать). Он всегда в
```

```
 </div>
```

```

 </div>
 <p class="text-slate-300 text-sm mb-4">
 еряет ВСЕ возможные дороги V-1 раз подряд.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb-4">
 опали во временную петлю.</p>
 </div>
 </div>
 <div id="slot-bellman-viz" class="mt-8"></div>
</div>
</div>
</section>`
 },
 {
 id: "floyd",
 title: "16. Графы. Поиск кратчайшего пути. Флойд",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
 </div>
 <div class="bg-slate-800 p-6 rounded-lg border">

```

```

<div class="bg-slate-900 p-4 rounded-lg mb-6">
 <p class="text-xl font-mono text-white">
</div>
<div class="grid grid-cols-1 gap-6 mb-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-amber-500 shadow-md">
 ⚡ Суть перевзвешивания
 </div>
 <p class="text-slate-300 text-sm mb-6">
 Дейкстра (Билет 14) быстрая, но работает очень медленно. Мы хотим сделать Джонсона пути остались теми же.
 </p>
 <ol class="text-sm text-slate-300 space-y-2">
 Добавляем фиктивную вершину
 Запускаем от неё медленный поиск «потенциалы» <code class="text-pink-400">
 Меняем старые веса: новое расстояние = старое + потенциал
 Магия! Все веса теперь минимальные (потенциалы сокращаются).
 Теперь смело запускаем быстрый поиск

 </div>
</div>
<div id="slot-johnson-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-1" style="border-color: #000000; border-radius: 10px; background-color: #000000; padding: 10px;">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ✖ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </div>
 </div>
 </div>
</div>
</div>
</section>`
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-1" style="border-color: #000000; border-radius: 10px; background-color: #000000; padding: 10px;">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">

```

```
<h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 сом, оканчивающимся в позиции i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ♂ Аналогия 1: Поиск без отка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Мы идём по строке слева направо. Если строка
 строки СЕЙЧАС закончился под моим пальцем?»
 Представь, ты ищешь в тексте слово
 х</code>. Тупой алгоритм начнет искать заголовок
 </code>, а это начало нашего слова! Не надо
 </p>
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: LLM стриминг
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Для LLM, которая стримит токены
 каждом шаге. Если мы частично совпали с заголовком
 какого места этого слова мы можем продолжить
 </p>
 </div>
 </div>
 </h3>
 </div>
 </div>
</div>
```

```

 pi[i] = j;
 }</pre>
 </div>
</details>
</div>
</div>
</section>`
 },
 {
 id: "string-z-func",
 title: "22. Z-функция — Сдвиг и наложение",
 type: "html",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 са 0) и её суффиксом, начинающимся с i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Линейка-шаблон
 </div>
 <p class="text-slate-300 text-sm mb">
 Тут мы берём всю строку и «прикр
 я начну читать строку отсюда, насколько длин

```

```

 <details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded

int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
 </div>
 </details>
 </div>
</div>
</section>`
 },
 {
 id: "aho-corasick",
 title: "23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `
<section id="aho-corasick" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-indigo-600 text-white px-4 py-1 roun
 <h2 class="text-2xl sm:text-3xl font-bold text-white
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-blue-400 mb-4">1

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">0c
 <p class="text-lg font-mono text-white">Бор (Tr
```

```

 return t[v].link;
 }

 int go(int v, char c) {
 if (t[v].go[c] == -1) {
 if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
 else t[v].go[c] = v == 0 ? 0 : go(get_link(v), c);
 }
 return t[v].go[c];
 }
}

</div>
</details>
</div>
</section>`
},
{
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">
 <p class="text-xl font-mono text-white">
 Сведение (Reduction) A->B: Если я умею решать A, то я умею решать B.
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4">

```



```

category: "Графы. База",
content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">
 </div>
 <div class="space-y-8">
 {/* База представлений */}
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-slate-300">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-slate-300 italic">
 <p class="text-xl font-mono text-white">
 (V + E).</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Матрица = Таблица
 </div>
 <p class="text-slate-300 text-sm mb-4">
 глупо, но проверка "знает ли Вася Петю" мгно
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия: Списки = Контакты
 </div>
 <p class="text-slate-300 text-sm mb-4">
 т. Оптимально для почти всех задач.</p>
 </div>
 </div>
 </div>
 </div>
 {/* DFS vs BFS */}
 <div class="bg-slate-700/50 p-6 rounded-xl border

```

<li>Двунаправленный поиск для у

</ul>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg l

<summary class="p-4 font-bold text-slate

-b border-slate-700/50">

Код обходов C++ (Основа)

</summary>

<div class="grid grid-cols-1 md:grid-co

<pre class="bg-slate-950 p-3 rounde

// DFS (Рекурсия)

vector<bool> vis;

vector<vector<int>>> adj;

void dfs(int v) {

vis[v] = true;

for (int u : adj[v]) {

if (!vis[u]) dfs(u);

}

}</pre>

<pre class="bg-slate-950 p-3 rounde

// BFS (Очередь)

queue<int> q;

q.push(start\_node);

vis[start\_node] = true;

while(!q.empty()) {

int v = q.front(); q.pop();

for (int u : adj[v]) {

if (!vis[u]) {

vis[u] = true;

q.push(u);

}

}

}</pre>

</div>

```

</section>`,
 },
 {
 id: "graph-components",
 title: "8. Графы. Компоненты связности",
 type: "html",
 description: "Нахождение кусков графа",
 category: "Графы. База",
 content: `
<section id="graph-components" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">8. Графы. Компоненты связности
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-emerald-400">Нахождение кусков графа
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Граф — это набор вершин и ребер, соединяющих их. Если граф связный, то из любой вершины можно добраться до любой другой. Если же граф не связный, то он состоит из нескольких частей, каждая из которых является связным графом. Эти части называются компонентами связности.
 <p class="text-xl font-mono text-white">Нахождение компонент связности — это процесс разбиения графа на такие части.
 <p class="text-xl font-mono text-white">Рассмотрим пример. Представим себе карту мира, где страны — это вершины, а океаны — это ребра. Если мы хотим узнать, сколько существует независимых частей (континентов), то нам нужно найти количество компонент связности.
 <p class="text-xl font-mono text-white">В данном случае ответ очевиден: 5 (Африка, Азия, Европа, Северная Америка, Южная Америка).
 <p class="text-xl font-mono text-white">Аналогия 1: Острова
 <p class="text-slate-300 text-sm mb-4">Если представить карту мира, где страны — это вершины, а океаны — это ребра, то количество компонент связности будет равно количеству независимых частей (континентов).
 <p class="text-xl font-mono text-white">В данном случае ответ очевиден: 5 (Африка, Азия, Европа, Северная Америка, Южная Америка).
 <p class="text-xl font-mono text-white">Аналогия 2: Острова
 <p class="text-slate-300 text-sm mb-4">Если представить карту мира, где страны — это вершины, а океаны — это ребра, то количество компонент связности будет равно количеству независимых частей (континентов).
 <p class="text-xl font-mono text-white">В данном случае ответ очевиден: 5 (Африка, Азия, Европа, Северная Америка, Южная Америка).
 </div>
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-emerald-400">Аналогия 1: Острова
 <p class="text-slate-300 text-sm mb-4">Если представить карту мира, где страны — это вершины, а океаны — это ребра, то количество компонент связности будет равно количеству независимых частей (континентов).
 <p class="text-xl font-mono text-white">В данном случае ответ очевиден: 5 (Африка, Азия, Европа, Северная Америка, Южная Америка).
 <p class="text-xl font-mono text-white">Аналогия 2: Острова
 <p class="text-slate-300 text-sm mb-4">Если представить карту мира, где страны — это вершины, а океаны — это ребра, то количество компонент связности будет равно количеству независимых частей (континентов).
 <p class="text-xl font-mono text-white">В данном случае ответ очевиден: 5 (Африка, Азия, Европа, Северная Америка, Южная Америка).
 </div>
 </div>
</section>`,
 },
 {

```

```

 <details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate-300"
 -b border-slate-700/50">
 Душный мод: Код на C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <p class="text-indigo-400 font-bold"
 <div class="bg-slate-950 p-4 rounded"
 <pre class="text-xs font-mono text-slate-300"
vector<int> adj[MAXN];
bool visited[MAXN];
vector<int> ans;

void dfs(int v) {
 visited[v] = true;
 for (int to : adj[v]) {
 if (!visited[to]) {
 dfs(to);
 }
 }
 ans.push_back(v); // Добавляем в момент ВЫХОДА (pos
}

void topological_sort(int n) {
 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
 reverse(ans.begin(), ans.end()); // Разворачиваем с
}</pre>
 </div>
 </div>
</details>
</div>
</div>
</section>`,
 },
 {

```

Аналогия: Интриги в офисе

</div>

<p class="text-slate-300 text-sm mb

Сильная связность – это "клуб с  
ни в одной SCC. Добавить курьера Гошу, котор  
йти наружу (к директору), но обратно в клуб

</p>

</div>

<div class="bg-slate-900 p-6 rounded-lg

<div class="absolute -top-3 left-4  
border-rose-500 shadow-md">

Связь с Билетом 12

</div>

<p class="text-slate-300 text-sm mb

<b>SCC (Билет 10)</b> склеивает

</br><br>

<b>Точки сочленения (Билет 12)</b>  
ость</strong> монолита. Вырвешь точку сочле

</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg

<summary class="p-4 font-bold text-slate  
-b border-slate-700/50">

Душный мод: Код Алгоритм Косарайю

</summary>

<div class="p-5 text-sm text-slate-300

<p class="text-emerald-400 font-bol

<ol class="list-decimal list-inside

<li>Запускаем DFS на исходном г

li>

<li>Разворачиваем этот список (

<li>Переворачиваем все ребра в

<li>Идем по <code>order</code>:

!</li>

</ol>

```

</section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 description: "Вершины, удаление которых убивает связность",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">12. Графы. Точки сочленения
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
 <h3 class="text-xl font-bold text-rose-400">Точки сочленения
 <p class="text-slate-300">Вершины графа, удаление которых увеличивает количество компонент связности, называются точками сочленения. Точка сочленения — это вершина, которая соединяет две или более компонент связности. Если удалить точку сочленения, то граф распадется на две или более компонент связности. Точка сочленения — это вершина, которая соединяет две или более компонент связности. Если удалить точку сочленения, то граф распадется на две или более компонент связности.
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">Пример: Рассмотрим граф, состоящий из двух циклов, соединенных в одной вершине. Эта вершина является точкой сочленения. Если ее удалить, то граф распадется на две компоненты связности.
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Точка сочленения — это вершина, удаление которой увеличивает число компонент связности.
 <div class="p-3 bg-slate-950 rounded">
 Важно:
 <p class="text-xl font-bold text-rose-400">Точка сочленения — это вершина, удаление которой увеличивает число компонент связности.
 <p class="text-xs text-slate-400">Точка сочленения — это вершина, удаление которой увеличивает число компонент связности.
 </div>
 </div>
 </div>
 </div>
 <div class="p-6 rounded-xl border border-slate-700">
 <p class="text-slate-300 text-sm">
 Важно: Это работает только для вершин, которые являются точками сочленения. То есть, Точка сочленения — это вершина, удаление которой увеличивает число компонент связности.
 </p>
 </div>
 </div>
</section>
`
 }
]

```



```
 </div>
 </div>
</section>`,
 },
];
```

---

## РАЗДЕЛ: ИНТЕРАКТИВНЫЕ КОМПОНЕНТЫ И СИМУЛЯТОРЫ

---

---

ФАЙЛ 30/88: src/components/ArticulationPointsViz.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useEffect } from "react";
import { Play, Pause, RotateCcw, Info } from "lucide-react";
```

```
interface Node {
 id: number;
 x: number;
 y: number;
 label: string;
}
```

```
interface Edge {
 u: number;
 v: number;
}
```

```
const nodes: Node[] = [
 { id: 0, x: 250, y: 50, label: "0" },
 { id: 1, x: 100, y: 150, label: "1" },
 { id: 2, x: 250, y: 250, label: "2" },
 { id: 3, x: 400, y: 150, label: "3" },
 { id: 4, x: 550, y: 150, label: "4" },
 { id: 5, x: 700, y: 50, label: "5" },
```



```

useEffect(() => {
 const newSteps: StepInfo[] = [];
 let timer = 0;
 const visited = new Set<number>();
 const ap = new Set<number>();
 const currentTin = Array(nodes.length).fill(-1);
 const currentLow = Array(nodes.length).fill(-1);

 const recordStep = (desc: string, u: number | null) => {
 newSteps.push({
 desc,
 visited: new Set(visited),
 ap: new Set(ap),
 currentNode: u,
 tin: [...currentTin],
 low: [...currentLow],
 });
 };

 recordStep("Начало DFS (Тарьян) для поиска точек со

 const dfs = (v: number, p: number = -1) => {
 visited.add(v);
 currentTin[v] = currentLow[v] = timer++;
 let children = 0;

 recordStep(`Заходим в вершину ${v}. tin[${v}]=${c

 for (const to of adj[v]) {
 if (to === p) continue;

 if (visited.has(to)) {
 currentLow[v] = Math.min(currentLow[v], curren
 recordStep(
 `Обратное ребро ${v}-${to}. Обновляем low[${
 v,
 });
 } else {

```

```

 recordStep("Алгоритм завершен. Найдены все точки со
 setSteps(newSteps);
}, []));

const currentStep = steps[currentStepIndex] || {
 desc: "",
 visited: new Set(),
 ap: new Set(),
 currentNode: null,
 tin: [],
 low: [],
};

useEffect(() => {
 let interval: NodeJS.Timeout;
 if (isPlaying && currentStepIndex < steps.length -
 interval = setInterval(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearInterval(interval);
}, [isPlaying, currentStepIndex, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIndex(0);
};

return (
 <div className="bg-slate-900 rounded-xl border bord
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Info className="w-5 h-5 text-rose-400" />
 Моделирование поиска точек сочленения

```

```

 return (
 <line
 key={"edge-" + i}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isBridge ? "#f43f5e" : "#475568"}
 strokeWidth={isBridge ? "4" : "2"}
 strokeDasharray={isBridge ? "8,4" : "1"}
 />
);
 }
 }
 }

 {/* Nodes */}
 {nodes.map((node) => {
 const isCurrent = currentStep.currentNode === node.id;
 const isVisited = currentStep.visited.has(node.id);
 const isAp = currentStep.ap.has(node.id);

 const fill = isCurrent
 ? "#3b82f6"
 : isAp
 ? "#e11d48"
 : isVisited
 ? "#10b981"
 : "#1e293b";
 const stroke = isCurrent
 ? "#60a5fa"
 : isAp
 ? "#f43f5e"
 : isVisited
 ? "#34d399"
 : "#334155";
 const r = isAp ? 24 : 20;

 return (
 <g key={node.id}>

```

```

 y={node.y + r + 20}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 low:{" "}
 {currentStep.low[node.id] >= 0
 ? currentStep.low[node.id]
 : "-"}
 </text>
 </>
 })
</g>
);
}}
</svg>
</div>

<div className="bg-slate-950 p-6 border-t lg:bo
 <h4 className="text-sm font-bold text-slate-4
 Статус алгоритма
 </h4>

 <div className="bg-slate-900 border border-sl
 <p className="text-white font-medium min-h-
 {currentStep.desc}
 </p>
 </div>

 <div className="space-y-4 flex-1 overflow-y-a
 <div className="mb-4">
 <h5 className="text-xs text-slate-500 fon
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Точка сочленения
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg

```

```
 }}
 />
 </div>
 </div>
</div>
</div>
</div>
);
};
```

---

ФАЙЛ 31/88: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';
import { useVizSync } from '../hooks/useVizSync';
import { useVizControl } from '../hooks/useVizControl';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 iteration: number;
 checkingEdge: { u: string; v: string; w: number } | null;
 distances: { [key: string]: number };
 previous: { [key: string]: string | null };
 log: string;
 hasNegativeCycle?: boolean;
 activeCodeLine: number;
}

export default function BellmanFordViz() {
 const [hasCycle, setHasCycle] = useState(false);
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);
```

```
const codeLines = [
 { line: 1, text: 'function bellman_ford(graph, start' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0' },
 { line: 3, text: ' for i from 1 to V - 1: // |V| - 1 iterations' },
 { line: 4, text: ' for (u, v, weight) in E: // E is an array of edges' },
 { line: 5, text: ' if dist[u] + weight < dist[v]' },
 { line: 6, text: ' dist[v] = dist[u] + weight' },
 { line: 7, text: ' for (u, v, weight) in E: // Проверка на отрицательный цикл' },
 { line: 8, text: ' if dist[u] + weight < dist[v]' },
 { line: 9, text: ' return "ОБНАРУЖЕН ОТРИЦАТЕЛЬНЫЙ ЦИКЛ" },
 { line: 10, text: ' return dist' },
];
```

```
useEffect(() => {
 const simulationSteps: Step[] = [];
 let dist: { [key: string]: number } = { S: 0, A: 999 };
 let prev: { [key: string]: string | null } = { S: null };

 simulationSteps.push({
 iteration: 0, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: ' Фура заведена. Начинаем с Базы (S). Расстояние до А: 999',
 activeCodeLine: 2,
 });
```

```
const vCount = nodes.length;
for (let iter = 1; iter < vCount; iter++) {
 let anyUpdate = false;
 for (const edge of edges) {
 const uDist = dist[edge.u];
 const vDist = dist[edge.v];

 let updated = false;
 if (uDist !== 999 && uDist + edge.w < vDist) {
 dist = { ...dist, [edge.v]: uDist + edge.w };
 prev = { ...prev, [edge.v]: edge.u };
 updated = true;
 anyUpdate = true;
 }
 }
}
```

```

 simulationSteps.push({
 iteration: vCount, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev }
 log: ' Проверка завершена. Ни одно ребро не
 activeCodeLine: 10,
 });
 }
 }

 setSteps(simulationSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
 }, [hasCycle]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;

useVizSync("bellman-ford", {
 n: 7,
 m: edges.length,
 i: currentStep?.iteration ?? "-",
 k: currentStep?.checkingEdge?.u ?? "-",
 j: currentStep?.checkingEdge?.v ?? "-",
});

useVizControl("bellman-ford", {
 onStep: () => { setIsPlaying(false); setCurrentStepIndex(0); },
 onReset: () => { setIsPlaying(false); setCurrentStepIndex(0); },
 onPlay: () => setIsPlaying(true),
 onPause: () => setIsPlaying(false),
});

```

```

<div className="absolute top-3 left-3 bg-blue
 ld shadow-sm">
 Итерация {currentStep.iteration} (Шаг {currentStep.step})
</div>

```

```

{currentStep.hasNegativeCycle && (
 <div className="absolute top-3 right-3 bg-red
 animate-pulse shadow-lg shadow-rose-600/50">
 ⚠ Отрицательный цикл!
 </div>
)}

```

```

<svg className="w-full h-auto max-h-[500px]" >
 <defs>
 <marker id="arrow-bf-normal" markerWidth="10" markerHeight="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
 <marker id="arrow-bf-active" markerWidth="10" markerHeight="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 <marker id="arrow-bf-cycle" markerWidth="10" markerHeight="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
 </defs>

```

```

{edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === edge.u);
 const vNode = nodes.find((n) => n.id === edge.v);
 const isChecking = currentStep.checkingEdges === idx;
 const isNeg = edge.w < 0;
 const isCyclePart = hasCycle && ((edge.u === 'D' || edge.v === 'D'));

```

```

 let strokeColor = '#475569';
 let marker = 'url(#arrow-bf-normal)';
 if (isChecking) { strokeColor = '#f59e0b'; }
 else if (currentStep.hasNegativeCycle && isNeg) { strokeColor = '#e11d48'; }

```

```

 return (
 <g key={idx}>

```





```
{ id: 3, label: '3', x: 75, y: 250 },
{ id: 4, label: '4', x: 225, y: 250 },
{ id: 5, label: '5', x: 375, y: 250 },
{ id: 6, label: '6', x: 525, y: 250 },
];

const EDGES = [
 { from: 0, to: 1 }, { from: 0, to: 2 },
 { from: 1, to: 3 }, { from: 1, to: 4 },
 { from: 2, to: 5 }, { from: 2, to: 6 }
];

const ADJ: { [key: number]: number[] } = {
 0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],
};

interface Step {
 activeLine: number;
 queue: number[];
 visited: number[];
 currentNode: number | null;
 logs: string;
}

const generateBfsSteps = (): Step[] => {
 const steps: Step[] = [];
 const queue: number[] = [];
 const visited = new Set<number>();

 // Init
 steps.push({
 activeLine: 1,
 queue: [],
 visited: [],
 currentNode: null,
 logs: "Инициализация: начинаем с корня (0). "
 });
};
```

```

 });

 visited.add(n);
 queue.push(n);

 steps.push({
 activeLine: 9,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Помечаем ${n} как посещенную и добавляем в очередь.`
 });
 }
}
} else {
 steps.push({
 activeLine: 6,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Соседей нет или все уже посещены.`
 });
}
}

steps.push({
 activeLine: 12,
 queue: [],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

const STEPS = generateBfsSteps();

```



```

 stroke={stroke}
 strokeWidth="3"
 className="transition-colors duration-300"
 />
 <text
 x={n.x} y={n.y}
 textAnchor="middle"
 dy=".3em"
 className="fill-white font-bold text-sm"
 >
 {n.label}
 </text>
 </g>
);
}}}
</svg>
</div>

```

```

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
 <div className="bg-slate-900 border border-slate-800 p-4">
 <div className="bg-slate-800 text-slate-400 p-4">
 bfs(graph, start)
 War {step}
 </div>
 <div className="p-4">
 {[
 { line: 1, code: 'queue = Queue()' },
 { line: 2, code: 'queue.push(start); visited.add(start);' },
 { line: 3, code: '}' },
 { line: 4, code: 'while not queue.isEmpty() {' },
 { line: 5, code: ' node = queue.pop()' },
 { line: 6, code: ' for neighbor in graph[node] {' },
 { line: 7, code: ' if neighbor not in visited {' },
 { line: 8, code: ' visited.add(neighbor);' },
 { line: 9, code: ' queue.push(neighbor);' },
 { line: 10, code: ' }' },
 { line: 11, code: ' }' },
 { line: 12, code: '}' },
]}
 </div>
 </div>

```

```
 });
}
```

---

ФАЙЛ 33/88: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const imageMap: Record<string, { src: string; alt: string }> = {
 dijkstra: {
 src: dijkstraImg,
 alt: 'Добрый Дейкстра',
 caption: 'Добрый — только позитив!',
 borderColor: 'border-blue-500/30',
 captionColor: 'text-blue-400',
 },
 'bellman-ford': {
 src: bellmanImg,
 alt: 'Фура по болоту',
 caption: 'Тяжёлая, но ей пофиг на ямы!',
 borderColor: 'border-rose-500/30',
 captionColor: 'text-rose-400',
 },
 floyd: {
 src: floydImg,
 alt: 'Все ко Всем Флойд',
 caption: 'Все связаны со всеми!',
 borderColor: 'border-emerald-500/30',
 captionColor: 'text-emerald-400',
 },
};
```

```
export default function ChapterImage({ vizType }: { vizType: string }) {
 return (
```

```

return (
 <div className="flex items-center gap-2 text-xs">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 <ChevronLeft className="w-4 h-4" />
 Назад
 </button>

 {index + 1} / {total}

 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 Вперёд
 <ChevronRight className="w-4 h-4" />
 </button>
 </div>
);
}

```

```

return (
 <nav className="grid grid-cols-1 sm:grid-cols-2 gap-2">
 {prev ? (
 <button
 onClick={() => onGo(prev.id)}
 className="group text-left p-4 rounded-xl bg-slate-100
 transition-all"
 >
 <div className="flex items-center gap-1.5 text-sm">

```

```

import { ChapterNav } from "../ChapterNav";
import { downloadChapterHtml } from "../utils/exportHtml";
import { getFallbackMeta } from "../data/compilerVars";

interface Props {
 chapter: Chapter;
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 onOpenSimulator: (vizId: string) => void;
 onOpenCompiler?: () => void;
}

const isTouch = () => typeof window !== "undefined" &&

export const ChapterView: React.FC<Props> = ({ chapter,
 const contentRef = useRef<HTMLDivElement>(null);
 const [anchor, setAnchor] = useState<HintAnchor | null>(null);
 const hideTimer = useRef<number | null>(null);
 const [saving, setSaving] = useState<"idle" | "work">("idle");

 useTermHints(contentRef, [chapter.id]);

 const cancelHide = useCallback(() => {
 if (hideTimer.current) {
 window.clearTimeout(hideTimer.current);
 hideTimer.current = null;
 }
 }, []);

 const scheduleHide = useCallback(() => {
 cancelHide();
 hideTimer.current = window.setTimeout(() => setAnchor(anchor), 1000);
 }, [cancelHide]);

 useEffect(() => {

```



```

const el = (e.target as HTMLElement).closest<HTMLElement>
if (!el) return;
e.preventDefault();
if (anchor && anchor.termId === el.dataset.termId)
else open(el);
};

const handleFocus = (e: React.FocusEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (el) open(el);
};

const viz = getViz(chapter.id);
const meta = getFallbackMeta(chapter.id);

return (
 <>
 <article className="chapter-body">
 /* панель загрузки: сохранить именно эту страницу */
 <div className="flex flex-wrap items-center gap-2">
 <button
 onClick={saveHtml}
 disabled={saving === "work"}
 title="Сохранить эту тему одним автономным файлом"
 className="flex items-center gap-1.5 text-x-small font-medium border-1px border-gray-200 hover:text-white hover:border-emerald-500"
 >
 {saving === "work" ? (
 <Loader2 className="w-3.5 h-3.5 animate-spin" />
) : saving === "done" ? (
 <Check className="w-3.5 h-3.5 text-emerald-500" />
) : (
 <Download className="w-3.5 h-3.5" />
)}
 {saving === "done" ? "Файл сохранён" : "Скачать"}
 </button>
 <button
 onClick={() => window.print()}

```

```

 }}
 </div>

 <div
 ref={contentRef}
 className="prose prose-invert max-w-none"
 onMouseOver={handlePointerOver}
 onMouseOut={handlePointerOut}
 onClick={handleClick}
 onFocus={handleFocus}
 dangerouslySetInnerHTML={{ __html: chapter.content }}
 />

 <p className="mt-6 flex items-start gap-2 text-
 ed-lg px-3 py-2">
 <MousePointerClick className="w-4 h-4 shrink-0" />

 Подчёркнутые термины – интерактивные: наведи
 объяснение на пальцах, мини-анимацию и кнопки

 </p>

 {viz && (
 <section id="chapter-viz" className="mt-8 scroll-mt-16">
 <div className="flex items-center justify-between">
 <h3 className="flex items-center gap-2 text-emerald-300">
 <Sparkles className="w-4 h-4 text-indigo-300" />
 Демонстрация: {viz.title}
 </h3>
 <div className="flex items-center gap-2">
 {onOpenCompiler && (
 <button
 onClick={onOpenCompiler}
 className="flex items-center gap-1.5 text-emerald-300
 700 text-emerald-300 hover:text-white hover:opacity-90"
 title="Открыть боковой компилятор с терминалом" />
 <Terminal className="w-3.5 h-3.5" />
)}
 </div>
 </div>
 </section>
)}

```

```
};
```

---

ФАЙЛ 36/88: src/components/CompilerPanelContent.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useEffect, useState } from "react";
import { Copy, Play, RotateCcw, StepForward, Pause, Terminate } from "lucide-react";
import { getFallbackMeta } from "../data/compilerVars";
```

```
interface Props {
 chapterId: string;
 onOpenFull: () => void;
 onClose?: () => void;
 compact?: boolean;
}
```

```
export function CompilerPanelContent({ chapterId, onOpenFull }: Props) {
 const meta = getFallbackMeta(chapterId);
 const [live, setLive] = useState<Record<string, string>>({});
 const [playing, setPlaying] = useState(false);
```

```
 useEffect(() => {
 const handler = (e: Event) => {
 const d = (e as CustomEvent).detail;
 if (d?.chapterId === chapterId || !d?.chapterId) return;
 };
 window.addEventListener("viz:sync", handler as EventListener);
 return () => window.removeEventListener("viz:sync", handler);
 }, [chapterId]);
```

```
 const sendControl = (action: "step" | "play" | "pause" | "stop") => {
 window.dispatchEvent(new CustomEvent("viz:control", { detail: action }));
 const viz = document.getElementById("chapter-viz");
 if (!viz) return;
 if (action === "step") {
```

```

 </div>
 </div>

 {/* тулбар IDE */}
 <div className="shrink-0 flex items-center gap-1.5" >
 <button onClick={() => sendControl("reset")} cla
 late-700 text-white text-xs border border-sla
 <RotateCcw className="w-3 h-3" /> Сброс
 </button>
 <button onClick={() => sendControl("step")} cla
 igo-500 text-white text-xs font-bold" title="
 <StepForward className="w-3 h-3" /> Шаг
 </button>
 <button onClick={() => sendControl(playing ? "p
 t-white text-xs font-bold ${playing ? "bg-amb
 {playing ? <Pause className="w-3 h-3" /> : <P
 </button>
 <div className="flex-1" />
 <button onClick={onOpenFull} className="inline-
 t-white text-xs font-bold" title="Полный Pyth
 <Terminal className="w-3 h-3" /> Запустить
 </button>
 </div>

 {/* LIVE полоска */}
 <div className="shrink-0 px-3 py-1.5 border-b bor
 no">
 <span className={`w-2 h-2 rounded-full ${live ?
 {live ? Object
 keys(meta.vars).join(" ")}`}
 {live && <span className="ml-auto text-emerald-
 </div>

 {/* редактор с нумерацией */}
 <div className="flex-1 overflow-auto bg-[#080d14]
 <div className="flex">
 <div className="shrink-0 bg-[#0f172a] border-
 ing-6">

```

```

{ line: "", highlight: "normal" },
{ line: " if (visited[to]) {", highlight: "normal" },
{ line: " low[v] = min(low[v], tin[to]);", highlight: "normal" },
{ line: " } else {", highlight: "normal" },
{ line: " dfs(to, v);", highlight: "normal" },
{ line: " low[v] = min(low[v], low[to]);", highlight: "normal" },
{ line: "", highlight: "normal" },
{ line: " if (low[to] > tin[v]) {", highlight: "normal" },
{ line: " // ЭТО МОСТ!", highlight: "normal" },
{ line: " }", highlight: "normal" },
{ line: " }", highlight: "normal" },
{ line: "}", highlight: "normal" },
{ line: "}" },
];

```

```

interface DfsStep {
 currentNode: number | null;
 visitedIndices: number[];
 tin: Record<number, number>;
 low: Record<number, number>;
 stack: number[];
 bridges: string[];
 log: string;
 activeEdge: [number, number] | null;
 backEdge: [number, number] | null;
 activeCodeLine: number[];
}

```

```

const GRAPH_NODES = [
 { id: 0, label: "A", x: 100, y: 120 },
 { id: 1, label: "B", x: 260, y: 120 },
 { id: 2, label: "C", x: 180, y: 200 },
 { id: 3, label: "D", x: 180, y: 300 },
 { id: 4, label: "E", x: 100, y: 370 },
 { id: 5, label: "F", x: 260, y: 370 },
 { id: 6, label: "G", x: 340, y: 200 }
];

```

```

 activeCodeLine: [13,14,15]
},
{
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, sta
 activeEdge: [1,2], backEdge: null,
 log: "Из В идём в С. tin[C]=4, low[C]=4.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, sta
 activeEdge: null, backEdge: [2,0],
 log: "Видим соседа А (уже посещён)! Обратное ребро
 activeCodeLine: [7,8,9]
},
{
 currentNode: 3, visitedIndices: [0,1,6,2,3],
 tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3
 activeEdge: [2,3], backEdge: null,
 log: "Идём C→D. tin[D]=5, low[D]=5.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 4, visitedIndices: [0,1,6,2,3,4],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2
 activeEdge: [3,4], backEdge: null,
 log: "D→E. tin[E]=6, low[E]=6.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
 activeEdge: [4,5], backEdge: null,
 log: "E→F. tin[F]=7, low[F]=7.",
 activeCodeLine: [4,5,7,10,11]
},
{

```

```

 log: "Возврат В→А. DFS завершён! Мосты: (В,G) и (С,D)",
 activeCodeLine: [11,13,16,17,18]
 },
 {
 currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:1,2:2,3:3,4:4,5:5},
 activeEdge: null, backEdge: null,
 log: "Готово! Найдены мосты: (В,G) и (С,D). Алгоритм завершён!",
 activeCodeLine: []
 },
];

```

```

export function DfsBridgesSimulator() {
 const [dfsIdx, setDfsIdx] = useState(0);
 const [dfsAuto, setDfsAuto] = useState(false);
 const dfsTimer = useRef<NodeJS.Timeout | null>(null);

 useEffect(() => {
 if (dfsAuto) {
 dfsTimer.current = setInterval(() => {
 setDfsIdx(prev => {
 if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false);
 return prev + 1;
 });
 }, 2500);
 }
 return () => { if (dfsTimer.current) clearInterval(dfsTimer.current);
 }, [dfsAuto]);

```

```

const step = DFS_STEPS[dfsIdx];

```

```

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">Настройка
 <div className="flex items-center gap-1 bg-slate-800 p-2">
 <button onClick={() => { setDfsAuto(false); setDfsIdx(0);
 disabled={dfsIdx === 0}

```

```

);
 let sc = "#475569", sw = 2, dash = "none"
 if (bridge) { sc = "#eab308"; sw = 4; }
 else if (activeTree) { sc = "#10b981"; sw = 2; }
 else if (activeBack) { sc = "#f43f5e"; sw = 2; }
 else if (step.visitedIndices.includes(e.u)) { sc = "#10b981"; sw = 2; }
 return <line key={e.key} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
 stroke={sc} strokeWidth={sw} strokeDash={dash}
 className="transition-all duration-500" />
)}
 {step.currentNode !== null && step.currentNode.id === n.id ? (
 const n = GRAPH_NODES.find(x => x.id === n.id)
 if (!n) return null;
 return <circle cx={n.x} cy={n.y} r={18} fill={cur} />
) : null}
)}()
 {GRAPH_NODES.map(n => {
 const cur = step.currentNode === n.id;
 const vis = step.visitedIndices.includes(n.id);
 const st = step.stack.includes(n.id);
 return <g key={n.id}>
 <circle cx={n.x} cy={n.y} r={12}
 fill={cur ? "#10b981" : st ? "#064e3b" : "#f43f5e"}
 stroke={cur ? "#fff" : st ? "#34d399" : "#f43f5e"}
 strokeWidth={2.5}
 className="transition-all duration-300" />
 <text x={n.x} y={n.y+3} textAnchor="middle">
 {n.label}</text>
 {vis && (
 <div>
 <rect x={n.x-18} y={n.y-28} width={36} height={10}
 className="transition-all duration-300" />
 <text x={n.x} y={n.y-20} textAnchor="center">
 {step.low[n.id] || "?"}</text>
 </div>
)}
 </g>
)}
 </g>
)}
)}

```



```

 <div className="flex flex-col-reverse gap-1"
 {step.stack.length === 0
 ? <div className="text-[10px] text-slate-500"
 : step.stack.map((id, i) => {
 <div key={id} className={`px-2 py-0
 i === step.stack.length-1
 ? "bg-emerald-600/20 text-emerald-500"
 : "bg-slate-900 text-slate-400"
 }`} >Вершина {GRAPH_NODES.find(n=>n.id === id).name}
 <span className="text-slate-500 m-1"

 </div>
))
 }
 </div>
 </div>

 { /* Log */ }
 <div className="bg-indigo-950/10 border border-indigo-500"
 <p className="text-[11px] text-slate-300 leading-5"
 <div className="mt-2 pt-2 border-t border-slate-700"
 Мосты:
 <span className="font-mono font-bold text-slate-300"
 {step.bridges.length > 0 ? step.bridges.map(b => b.name) : ""}

 </div>
 </div>
 </div>
</div>
);
}

```

```

 { u: 'E', v: 'F', w: 1 },
];

const adjList: { [key: string]: { v: string; w: number } } = {
 ...nodes.forEach(n => (adjList[n.id] = [])),
 ...edges.forEach(e => {
 adjList[e.u].push({ v: e.v, w: e.w });
 });
};

const codeLines = [
 { line: 1, text: 'function dijkstra(graph, start):' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0' },
 { line: 3, text: ' pq = PriorityQueue(); pq.push(start, 0)' },
 { line: 4, text: ' while not pq.empty():' },
 { line: 5, text: ' d, u = pq.pop() // Извлекаем элемент с минимальным расстоянием' },
 { line: 6, text: ' if d > dist[u]: continue' },
 { line: 7, text: ' for v, weight in graph.neighbors(u):' },
 { line: 8, text: ' if dist[u] + weight < dist[v]:' },
 { line: 9, text: ' dist[v] = dist[u] + weight' },
 { line: 10, text: ' pq.push(v, dist[v])' },
 { line: 11, text: ' return dist // Все кратчайшие пути найдены' },
];

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useState(0);
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist: Record<string, number> = { A: 0, B: 999 };
 const visited: Record<string, boolean> = { A: false, B: false };
 const prev: Record<string, string | null> = { A: null, B: null };

 simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ' Дейкстра готов к доставке. Начинаем из Пикет-Пост-Оффиса',
 activeCodeLine: 2,
 });
}, []);

```

```

 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: `    Улучшение (Релаксация)! Новое кратчайшее расстояние: ${dist[v]}!`,
 activeCodeLine: 9,
 });
 }
 }
 }

 simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: `  Дейкстра всё доставил! Все возможные кратчайшие расстояния найдены!`,
 activeCodeLine: 11,
 });

 setSteps(simulationSteps);
}, []);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;

useVizSync("dijkstra", {
 n: 7,
 m: 11,
 i: currentStep?.currentNode ?? "-",

```

[illegible]

```

 }}}

 { /* Вершины */ }
 { nodes.map((node) => {
 const isCurrent = currentStep.currentNode === node.id
 const isVisited = currentStep.visited[node.id]
 const dist = currentStep.distances[node.id]

 return (
 <g key={node.id} className="transition-all duration-300">
 <circle
 cx={node.x} cy={node.y}
 r={isCurrent ? 22 : 18}
 fill={isCurrent ? '#4f46e5' : isVisited ? '#a5b4fc' : '#d1d7e8'}
 stroke={isCurrent ? '#a5b4fc' : isVisited ? '#d1d7e8' : '#4f46e5'}
 strokeWidth={isCurrent ? 4 : 2}
 className="transition-all duration-300"
 />
 <text x={node.x} y={node.y + 5} fill="black">
 {node.id}
 </text>
 <text x={node.x} y={node.y - 28} fill="white">
 {dist === 999 ? '∞' : `d=${dist}`}
 </text>
 <text x={node.x} y={node.y + 35} fill="black">
 {node.name.split(' ')[0]}
 </text>
 </g>
)
 })
 }
</svg>
<div className="w-full bg-slate-950/80 p-4 rounded">
 <p className="font-semibold text-amber-400">Шаг {currentStep.id}</p>
 <p className="min-h-[40px] flex items-center">
 {currentStep.currentNode}
 </p>
</div>
</div>

```

```
</div>
```

```
<div className="flex flex-col lg:flex-row gap-6">
```

```
 { /* Таблица расстояний */ }
```

```
 <div className="w-full lg:w-1/2 bg-rose-950/10">
```

```
 <div>
```

```
 <h4 className="text-lg font-bold text-rose-950">
```

```
 <div className="overflow-x-auto">
```

```
 <table className="w-full text-left border">
```

```
 <thead>
```

```
 <tr className="border-b border-rose-950">
```

```
 <th className="py-2 px-3">Вершина</th>
```

```
 <th className="py-2 px-3">Расстояние</th>
```

```
 <th className="py-2 px-3">Из (пред. шаг)</th>
```

```
 </tr>
```

```
 </thead>
```

```
 <tbody className="text-sm">
```

```
 {nodes.map((n) => {
```

```
 const dist = currentStep.distances[n.id]
```

```
 const prev = currentStep.previous[n.id]
```

```
 const vis = currentStep.visited[n.id]
```

```
 const isCurr = currentStep.currentNode === n.id
```

```
 return (
```

```
 <tr key={n.id} className={`border-rose-950`}
```

```
 {isCurr ? `bg-indigo-950/60 font-medium` : ``}
```

```
 >
```

```
 <td className="py-2.5 px-3 flex items-center">
```

```
 <span className={`w-2 h-2 rounded`}
```

```
 {n.name}
```

```
 </td>
```

```
 <td className="py-2.5 px-3 font-medium">
```

```
 {dist === 999 ? '∞' : dist}
```

```
 </td>
```

```
 <td className="py-2.5 px-3 text-sm">
```

```
 {prev || '—'}
```

```
 </td>
```

```
 </tr>
```

```
import { useState, useEffect } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'react-feather';

interface DPStep {
 id: number;
 n: number;
 action: 'call' | 'memo_hit' | 'base_case' | 'calc' | 'return';
 desc: string;
 codeLine: number;
 memoState: { [key: number]: number };
}

const DP_CODE_LINES = [
 /* 1 */ "function fibMemo(n, memo = {}) {",
 /* 2 */ " if (n in memo) return memo[n];",
 /* 3 */ " if (n <= 2) return 1;",
 /* 4 */ " memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);",
 /* 5 */ " return memo[n];",
 /* 6 */ "}";
];

export function DPVisualizer() {
 const [targetN, setTargetN] = useState<number>(5);
 const [steps, setSteps] = useState<DPStep[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const [isPlaying, setIsPlaying] = useState<boolean>(false);
 const [speed] = useState<number>(800);
 const [activeTab, setActiveTab] = useState<'table' | 'graph'>('table');

 useEffect(() => {
 const generatedSteps: DPStep[] = [];
 let stepId = 0;
 const memoMap: { [key: number]: number } = {};

 function simulateFib(n: number): number {
 generatedSteps.push({
 id: stepId,
 n: n,
 action: 'call',
 desc: `Call fibMemo(${n})`,
 codeLine: 1,
 memoState: memoMap,
 });
 stepId++;
 if (n <= 2) {
 generatedSteps.push({
 id: stepId,
 n: n,
 action: 'base_case',
 desc: `Base case: fibMemo(${n}) = 1`,
 codeLine: 3,
 memoState: memoMap,
 });
 stepId++;
 return 1;
 }
 generatedSteps.push({
 id: stepId,
 n: n,
 action: 'calc',
 desc: `Calculate fibMemo(${n}) = fibMemo(${n-1}) + fibMemo(${n-2})`,
 codeLine: 4,
 memoState: memoMap,
 });
 stepId++;
 const result = simulateFib(n - 1) + simulateFib(n - 2);
 generatedSteps.push({
 id: stepId,
 n: n,
 action: 'return',
 desc: `Return result: fibMemo(${n}) = ${result}`,
 codeLine: 5,
 memoState: memoMap,
 });
 stepId++;
 return result;
 }

 simulateFib(targetN);
 setSteps(generatedSteps);
 }, [targetN]);
}
```

```

 memoState: { ...memoMap }
 });

 const res1 = simulateFib(n - 1);
 const res2 = simulateFib(n - 2);
 memoMap[n] = res1 + res2;

 generatedSteps.push({
 id: stepId++,
 n,
 action: 'calc',
 desc: `Сохраняем в кэш: memo[${n}] = ${res1} + ${res2}`,
 codeLine: 5,
 memoState: { ...memoMap }
 });

 return memoMap[n];
 }

 const finalRes = simulateFib(targetN);

 generatedSteps.push({
 id: stepId++,
 n: targetN,
 action: 'done',
 desc: `Расчет завершен! fibMemo[${targetN}] = ${finalRes}`,
 codeLine: 6,
 memoState: { ...memoMap }
 });

 setSteps(generatedSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
}, [targetN]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex(currentStepIndex + 1);
 }, 1000);
 }
 return () => {
 if (timer) {
 clearTimeout(timer);
 }
 };
}, [isPlaying, currentStepIndex]);

```



```

 <option value={5}>N = 5</option>
 <option value={6}>N = 6</option>
 <option value={7}>N = 7</option>
 </select>
 </div>

 <div className="flex items-center bg-slate-950" style={{width: 1000px, height: 100px}}>
 <button
 onClick={() => { setIsPlaying(false); setSteps([0]); }}
 className="p-2 text-slate-400 hover:text-white"
 title="Сбросить"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 <button
 onClick={() => { setIsPlaying(false); setSteps([0]); }}
 disabled={currentStepIndex === 0}
 className="p-2 text-slate-400 hover:text-white"
 title="Шаг назад"
 >
 <ChevronLeft className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1.5 px-4 py-2
 ${isPlaying
 ? 'bg-amber-500 text-slate-950 hover:bg-amber-600'
 : 'bg-emerald-600 text-white hover:bg-emerald-700'}
 `}
 >
 {isPlaying ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
 {isPlaying ? 'Пауза' : 'Пуск'}
 </button>
 <button
 onClick={() => { setIsPlaying(false); setSteps([0]); }}
 disabled={currentStepIndex === steps.length - 1}
 className="p-2 text-slate-400 hover:text-white"
 title="Шаг вперед"
 >
 <RotateCw className="w-4 h-4" />
 </button>
 </div>
 </div>
</div>

```

```

 <div className="mt-6 pt-4 border-t border-slate-400">
 <div className="text-sm font-medium text-slate-400">

 {currentStep.desc}
 </div>
 <div className="text-xs font-mono bg-slate-400 p-2">
 Текущий N: <strong className="text-emerald-400">{currentStep.n}
 </div>
 </div>
)}
</div>

{/* Tabs */}
<div className="flex items-center gap-2 border-b border-slate-400">
 <button
 onClick={() => setActiveTab('table')}
 className={`flex items-center gap-2 px-6 py-3 border rounded-md
 ${activeTab === 'table'
 ? 'border-emerald-500 text-emerald-400 bg-slate-400'
 : 'border-transparent text-slate-400 hover:text-slate-300'}
 `}
 >
 <Eye className="w-4 h-4" />
 Шаг-бай-шаг состояние
 </button>
 <button
 onClick={() => setActiveTab('code')}
 className={`flex items-center gap-2 px-6 py-3 border rounded-md
 ${activeTab === 'code'
 ? 'border-emerald-500 text-emerald-400 bg-slate-400'
 : 'border-transparent text-slate-400 hover:text-slate-300'}
 `}
 >
 <Code className="w-4 h-4" />
 Интерактивный код
 </button>
</div>

```

```


 {line}
 </div>
);
 }
 }
</pre>
<div className="p-6 bg-slate-900/50 border-slate-800">

 {currentStep?.desc}
 </div>
</div>
)}
</div>

{/* Progress Bar */}
<div className="mt-8 pt-6 border-t border-slate-800">
 <div className="flex items-center justify-between">
 Прогресс выполнения
 Шаг {currentStepIndex + 1} из {steps.length}
 </div>
 <div className="h-2 bg-slate-950 rounded-full overflow-hidden">
 <div
 className="h-full bg-gradient-to-r from-emerald-500 to-emerald-400"
 style={{ width: `${((currentStepIndex + 1) / steps.length) * 100}%` }}
 ></div>
 </div>
</div>
</div>
);
}

```

ФАЙЛ 40/88: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useState } from "react";
```

```

 return;
 }
 const key = `${Math.min(last, vId)}-${Math.max(last, vId)}`;
 if (c1VisitedEdges.includes(key)) {
 setC1Msg("Это ребро уже пройдено! Эйлер не прощает повторений!");
 setC1Done(true); return;
 }
 const nextEdges = [...c1VisitedEdges, key];
 const nextPath = [...c1Path, vId];
 setC1VisitedEdges(nextEdges);
 setC1Path(nextPath);

 if (nextEdges.length === C1_EDGES.length) {
 const startDegree = C1_EDGES.filter(e => e.u === c1Path[0]).length;
 const endDegree = C1_EDGES.filter(e => e.u === c1Path[c1Path.length - 1]).length;
 const isCycle = startDegree % 2 === 0 && endDegree % 2 === 0;
 if (isCycle && nextPath[0] === vId) {
 setC1Msg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, факт!");
 } else {
 setC1Msg(`    ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! Счётчик: ${nextEdges.length}`);
 }
 setC1Done(true);
 } else {
 setC1Msg(`Пройдено рёбер: ${nextEdges.length} / ${C1_EDGES.length}`);
 }
};

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">Решение задачи Эйлера
 <button onClick={resetC1} className="flex items-center justify-center border border-slate-700 transition-all">
 <Undo className="h-3.5 w-3.5" /> Сброс
 </button>
 </div>
 <div className="bg-slate-950 rounded-xl p-4 border">

```

```
 : "bg-slate-900/50 border-slate-700 text-slate-50"
 }`}>
 {clMsg}
 </div>
</div>
);
}
```

---

ФАЙЛ 41/88: src/components/ExpressQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from 'react';
import { CheckCircle2, XCircle, Award, RotateCcw, AlertTriangle } from 'lucide-react';

interface Question {
 id: number;
 question: string;
 options: string[];
 correctAnswer: number;
 explanation: string;
}

const quizQuestions: Question[] = [
 {
 id: 1,
 question: 'Что мы делаем в алгоритме Краскала, если ребро соединяет две вершины, которые уже принадлежат одному компоненту связности (красный) цвет?',
 options: [
 'Мы радуемся и добавляем его в минимальное остовное дерево',
 'Мы перекрашиваем их в синий цвет и делим граф по этому ребру',
 'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе получим цикл',
 'Мы вызываем алгоритм Дейкстры для поиска нового пути'
],
 correctAnswer: 2,
 explanation: 'Верно! Если вершины уже в одном клане'
 }
];
```

```

 correctAnswer: 1,
 explanation: 'Точно! Главный принцип ДП – мемоизации.',
 },
 {
 id: 5,
 question: 'Какая структура данных идеально помогает?',
 options: [
 'СНМ (Система непересекающихся множеств / Disjoint Sets)',
 'Стек (LIFO).',
 'Красно-черное дерево.',
 'Хэш-таблица со списками коллизий.'
],
 correctAnswer: 0,
 explanation: 'Блестяще! СНМ (DSU) позволяет за прак
 .
 }
];

```

```

export const ExpressQuiz: React.FC = () => {
 const [currentQIndex, setCurrentQIndex] = useState<number>(0);
 const [selectedOption, setSelectedOption] = useState<string>('');
 const [isAnswered, setIsAnswered] = useState<boolean>(false);
 const [score, setScore] = useState<number>(0);
 const [showResults, setShowResults] = useState<boolean>(false);

 const currentQ = quizQuestions[currentQIndex];

 const handleSelectOption = (index: number) => {
 if (isAnswered) return;
 setSelectedOption(index);
 setIsAnswered(true);
 if (index === currentQ.correctAnswer) {
 setScore(prev => prev + 1);
 }
 };

 const handleNext = () => {
 if (currentQIndex + 1 < quizQuestions.length) {

```

```

 <button
 onClick={handleRestart}
 className="inline-flex items-center gap-2 px-4 rounded-xl shadow-lg shadow-indigo-600/30"
 >
 <RotateCcw className="w-5 h-5" />
 Пройти тест заново
 </button>
 </div>
);
 }

 return (
 <div className="bg-slate-900/90 border border-slate-800" >
 <div className="flex items-center justify-between" >
 <div className="flex items-center gap-3" >

 Экспресс-тест

 <h3 className="text-xl font-bold text-white">
 Вопрос {currentQIndex + 1} / {quizQuestions.length}
 </h3>
 </div>

 Вопрос {currentQIndex + 1} / {quizQuestions.length}

 </div>

 <div className="mb-8">
 <h4 className="text-xl md:text-2xl font-bold text-white">
 {currentQ.question}
 </h4>

 <div className="space-y-4">
 {currentQ.options.map((option, idx) => {
 const isSelected = selectedOption === idx;
 const isCorrect = idx === currentQ.correctAnswer;
 return (
 <div
 key={idx}
 className={`flex items-center gap-3 p-4 rounded-lg border ${
 isSelected ? 'border-purple-500' : 'border-slate-700'
 }`}
 >
 <input
 type="radio"
 checked={isSelected}
 className="w-4 h-4"
 />
 <div>
 {option}
 {isCorrect ? Правильно : Неправильно}
 </div>
 </div>
);
 })}
 </div>
 </div>
 </div>
);
}

```

```

 {isAnswered && (
 <div className="bg-slate-950 border border-slate-800 p-4">
 <div className="flex items-center gap-2 mb-2">
 <AlertCircle className="w-4 h-4" />
 Объяснение:
 </div>
 <p className="text-slate-300 text-sm leading-relaxed">
 {currentQ.explanation}
 </p>
 </div>
)}

 <div className="flex justify-end pt-4 border-t border-slate-800">
 <button
 onClick={handleNext}
 disabled={!isAnswered}
 className={
 "px-8 py-3.5 font-bold rounded-xl transition-colors"
 (!isAnswered
 ? "bg-slate-800 text-slate-500 border border-slate-700"
 : "bg-indigo-600 hover:bg-indigo-500 active:bg-indigo-700")
 }
 >
 {currentQIndex + 1 < quizQuestions.length ? 'Следующий' : 'Завершить'}
 </button>
 </div>
 </div>
);
};

```

ФАЙЛ 42/88: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import { useState, useEffect } from 'react';
import { useVizSync } from '../hooks/useVizSync';

```



```

3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
4: [{ v: 6, w: 1 }],
5: [{ v: 6, w: 5 }],
6: [{ v: 1, w: 1 }],
];

const codeLines = [
 { line: 1, text: 'function floyd_warshall(matrix, V' },
 { line: 2, text: ' dist = copy(matrix)' },
 { line: 3, text: ' for k from 0 to V - 1: // Пос' },
 { line: 4, text: ' for i from 0 to V - 1:' },
 { line: 5, text: ' for j from 0 to V - 1' },
 { line: 6, text: ' if dist[i][k] + d' },
 { line: 7, text: ' dist[i][j] = ' },
 { line: 8, text: ' return dist' },
];

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist = initialMatrix.map(row => [...row]);

 simulationSteps.push({
 k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: ' Инициализация матрицы прямых путей, ∞ (99',
 activeCodeLine: 2,
 });

 const N = 7;
 for (let k = 0; k < N; k++) {
 simulationSteps.push({
 k, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: `+ Внешний цикл. Разрешаем пути через про
 activeCodeLine: 3,
 });

 for (let i = 0; i < N; i++) {
 for (let j = 0; j < N; j++) {
 if (i === j || i === k || j === k) continue;

```

```

useVizSync("floyd", {
 n: 7,
 m: "-",
 k: currentStep ? (currentStep.k >= 0 ? currentStep.k : 0) : 0,
 i: currentStep ? (currentStep.i >= 0 ? currentStep.i : 0) : 0,
 j: currentStep ? (currentStep.j >= 0 ? currentStep.j : 0) : 0,
});
useVizControl("floyd", {
 onStep: () => { setIsPlaying(false); setCurrentStep(0); },
 onReset: () => { setIsPlaying(false); setCurrentStep(0); },
 onPlay: () => setIsPlaying(true),
 onPause: () => setIsPlaying(false),
});
if (!currentStep) return <div>Загрузка...</div>;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
 <div className="flex flex-wrap items-center justify-center gap-4">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white rounded-lg">

 </div>
 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">Алгоритм Флойда</h3>
 <p className="text-sm text-emerald-300">Полный код алгоритма Флойда</p>
 </div>
 </div>

 <div className="flex items-center gap-2">
 <button onClick={() => { setIsPlaying(false); }}>
 text-sm"> C6poc</button>
 <button onClick={() => setIsPlaying(!isPlaying)}>
 -amber-600' : 'bg-emerald-600'}`}>{isPlaying ? 'Пауза' : 'Играть'}</button>
 <button onClick={() => { setIsPlaying(false); }}>
 ; }} className="px-3 py-2 bg-emerald-600 text-white">Сброс</button>
 </div>
 </div>

```

```

 });
 }}}
 {nodesSvg.map((node) => {
 const isK = currentStep.k === node.id;
 const isI = currentStep.i === node.id;
 const isJ = currentStep.j === node.id;
 let fill = '#1e293b'; let stroke = '#64748f';
 if (isK) { fill = '#4f46e5'; stroke = '#a6c9ec'; }
 else if (isI) { fill = '#b45309'; stroke = '#d9b48f'; }
 else if (isJ) { fill = '#047857'; stroke = '#a6d9b4'; }
 return (
 <g key={node.id} className="transition-colors">
 <circle cx={node.x} cy={node.y} r={isJ ? 4 : 2} className="transition-colors duration-200">
 <text x={node.x} y={node.y + 5} fill="white">{node.k}</text>
 <text x={node.x} y={node.y + 32} fill="white">{node.i}</text>
 </g>
);
 });
 }}}
</svg>

<div className="w-full bg-slate-950/80 p-4 rounded">
 <p className="font-semibold text-amber-400">Список смежности</p>
 <p className="min-h-[40px] flex items-center">
 </div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/80 p-4 rounded">
 <h4 className="text-lg font-bold text-emerald-400">Список смежности</h4>
 <div className="space-y-2 text-sm font-mono">
 {Object.keys(adjList).map((uStr) => {
 const u = parseInt(uStr);
 const isI = currentStep.i === u;
 const isK = currentStep.k === u;
 return (
 <div key={u} className={`flex flex-wrap`}>

```

```

 <tr key={i} className="border-b borde
 <td className={`p-2 bg-rose-950/40
 tStep.k === i ? 'text-amber-400' : ''}`>
 {nodeNames[i].split(' ')[0]} {i}
 </td>
 {row.map((val, j) => {
 const isUpd = currentStep.i === i
 const isVia = currentStep.k !== -
 tep.j === j});
 let bg = 'bg-slate-900/40 text-sl
 if (isUpd) bg = 'bg-emerald-900/8
 else if (isVia) bg = 'bg-indigo-9
 else if (i === j) bg = 'bg-slate-
 return (<td key={j} className={`p
 /td>);
 }}}
 </tr>
 }}}
 </tbody>
 </table>
</div>
</div>

```

```

 {/* Код алгоритма */}
 <div className="w-full lg:w-1/2 bg-slate-900/60
 <div>
 <h4 className="text-lg font-bold text-slate
 <div className="bg-slate-950/80 rounded-lg p
 {codeLines.map(item => {
 <div key={item.line} className={`py-1.5
 e === item.line ? 'bg-blue-900/80 text-ambe
 <span className="text-slate-600 w-5 s
 <span className="whitespace-pre flex-
 </div>
)}}
 </div>
 </div>
 </div>
 </div>
 </div>

```

```

 'E': ['B'],
 'F': ['C'],
 'G': ['C'],
};

type Action = {
 type: 'visit' | 'process' | 'backtrack';
 node: string;
 desc: string;
 queueOrStack?: string[];
 visitedNodes?: string[];
};

function generateDFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const stack: string[] = []; // for visualization, j

 function dfs(v: string) {
 vis.add(v);
 stack.push(v);
 steps.push({ type: 'visit', node: v, desc: `Зависит от`,
 visitedNodes: Array.from(vis) });

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 steps.push({ type: 'process', node: v, desc: `Процесс`,
 queueOrStack: [...stack], visitedNodes: Array.from(vis) });
 dfs(u);
 }
 }

 stack.pop();
 steps.push({ type: 'backtrack', node: v, desc: `Откат`,
 queueOrStack: [...stack], visitedNodes: Array.from(vis) });
 }

 dfs(start);
}

```

```

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);

const steps = useMemo(() => mode === 'dfs' ? genera

useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [mode]);

useEffect(() => {
 if (autoPlay) {
 const t = setInterval(() => {
 setStepIdx(p => {
 if (p >= steps.length - 1) {
 setAutoPlay(false);
 return p;
 }
 return p + 1;
 });
 }, 1200);
 return () => clearInterval(t);
 }
}, [autoPlay, steps.length]);

const step = steps[stepIdx] || steps[0];

const isVisited = (nodeId: string) => step.visitedN
const isCurrent = (nodeId: string) => step.node ===

return (
 <div className="space-y-4">
 <div className="flex bg-slate-900 border b
 <button onClick={() => setMode("dfs")}
 className={`px-4 py-2 text-xs font-l
text-slate-400 hover:text-slate-300`} `}>
 DFS (В глубину)
 </button>

```

```

 strokeColor = mode ===
 textColor = "fill-white"
 }

 if (current && step.type !==
 strokeColor = "stroke-amber"
 textColor = "fill-amber"
 radius = 22;
 }

 return (
 <g key={n.id} className=
 <circle cx={n.x} cy=
 className={`stroke-
 <text x={n.x} y={n.y}
 className={`text-
 {n.label}
 </text>

 {/* Ripple for BFS mode */}
 {current && mode ===
 <circle cx={n.x} cy={n.y}
 className="stroke-amber"
 >>
 </g>
)
)
 }
}
</svg>
</div>

{/* Structure View (Stack / Queue) */}
<div className="bg-slate-900 border border-emerald-500
 r">
 <div className={`absolute -top-3 left-3 right-3 bottom-3
 ${mode === 'dfs' ? 'border-emerald-500' : 'border-emerald-500'}
 >
 {mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
 </div>

```

```
 </button>
 <button onClick={() => {setAutoPlay
g-slate-800 rounded-lg ml-2 border-l border
 <RefreshCw strokeWidth={3} size
 </button>
 </div>

 {/* Description Log */}
 <div className="flex-1 bg-slate-900/50
ow-hidden">
 <div className="text-[10px] text-sl
{steps.length}]</div>
 <div className="text-sm text-slate-:
 {step.desc}
 </div>
 </div>
 </div>
 </div>
);
}
```

---

ФАЙЛ 44/88: src/components/HearViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
```

```
interface Patient {
 id: string;
 name: string;
 emoji: string;
 symptom: string;
 priority: number; // 1 to 99 (Severity)
 animState: 'in' | 'idle' | 'winner' | 'out';
}
```



## Универсальное пособие • полный исходный код

```

function nodePos(index: number): { x: number; y: number } {
 const level = Math.floor(Math.log2(index + 1));
 const posInLevel = index - (Math.pow(2, level) - 1);
 const count = Math.pow(2, level);
 const x = ((posInLevel + 0.5) / count) * 100;
 const y = 10 + level * 24;
 return { x, y };
}
```

```
// Priority presets for realistic ER scenarios
```

```
const PATIENT_PRESETS = [
 { name: 'Иван (Инфаркт)', symptom: ' Болят сердце', emoji: '🫀' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '🦴' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '🌡️' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '🔪' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '🤧' },
];
```

```
const INITIAL_PATIENTS: Patient[] = (() => {
 let arr: Patient[] = [];
 const start = [
 { name: 'Иван (Инфаркт)', symptom: ' Болят сердце', emoji: '🫀' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '🦴' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '🌡️' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '🔪' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '🤧' },
];
 start.forEach((p, i) => {
 arr = heapInsert(arr, { id: `init${i}`, ...p, animS: 0 });
 });
 return arr;
})();
```

```
let uid = 300;
```

```
export const HeapViz: React.FC = () => {
```

```

 setHeap(marked);
 addLog(` Поступил пациент: ${finalName} с тяжестью

 setTimeout(() => {
 setHeap(nextHeap);
 setBusy(false);
 }, 500);
 }, [busy, heap]);

const handleRandomInsert = () => {
 const preset = PATIENT_PRESETS[Math.floor(Math.random() *
 const randomShift = Math.floor(Math.random() * 10)
 const prio = Math.max(10, Math.min(99, preset.priority));
 insertPatient(preset.name, prio);
};

const handleCustomInsert = (e: React.FormEvent) => {
 e.preventDefault();
 if (!customName.trim()) return;
 insertPatient(customName, customPriority);
 setCustomName('');
};

// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
 if (busy || heap.length === 0) {
 setShake(true);
 addLog('⚠ Нет пациентов для экстренной госпитализации');
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);
 const topPatient = heap[0];
 setHealing(topPatient);

 addLog(` СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);

```

```

// --- Render binary tree lines ---
const edges = heap.flatMap(({_, idx}) => {
 const res: React.ReactElement[] = [];
 const p = nodePos(idx);
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < heap.length) {
 const lp = nodePos(l);
 res.push(
 <line key={`el${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 if (r < heap.length) {
 const rp = nodePos(r);
 res.push(
 <line key={`er${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 return res;
});

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-emerald-950/40 border border-emerald-500">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-emerald-400 text-center">
 Queue</h3>
 <p className="text-xs text-slate-300 mt-1">

```

```

 max="99"
 value={customPriority}
 onChange={e => setCustomPriority(Number(e
 className="w-20 accent-emerald-500 cursor
 />
</div>
<button
 type="submit"
 disabled={busy || !customName.trim() || heap
 className="bg-teal-600 hover:bg-teal-500 di
 px-4 rounded-xl shadow-lg active:scale-95 t
>
 Везти
</button>
</form>

{/* Забрать самого тяжелого */}
<button
 onClick={extractMax}
 disabled={busy || heap.length === 0}
 className="flex-1 min-w-[170px] bg-gradient-to
 40 disabled:cursor-not-allowed text-white f
 transition-all cursor-pointer flex items-c
>
 В реанимацию (EXTRACT MAX)
</button>

<button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-8
 r-pointer border border-slate-700"
>
 Сброс
</button>
</div>

{/* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */}
<div className="grid grid-cols-1 lg:grid-cols-12

```

```

 { /* Tooltip */ }
 <div className="absolute bottom-full
 <div className="bg-slate-950 border
0 shadow-xl">
 <div className="font-bold text-wh
 <div className="text-emerald-400"
 </div>
 </div>
 </div>
);
 }}}
</div>

 <div className="w-full h-3 bg-gradient-to-r f
</div>

{ /* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */ }
<div className="lg:col-span-4 bg-slate-900 round
 <div className="absolute top-4 left-4 text-[1
 Операционная койка
 </div>

 <div className="flex-1 flex flex-col justify-
 {healingPatient ? (
 <div className="flex flex-col items-cente
 <div className="relative">
 <span className="text-6xl block anim-l
 <span className="absolute -top-4 -right
 <span className="absolute -bottom-2 -
 </div>
 <div>
 <h4 className="text-white font-black
 Пациент: {healingPatient.name
 </h4>
 <p className="text-emerald-400 text-x
 Состояние: Стабилизация ({healingPa
 </p>

```

ФАЙЛ 45/88: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { createContext, useContext, useEffect, use
```

```
/** Общие примитивы для мини-визуализаций подсказок. */
```

```
export const W = 260;
```

```
export const H = 130;
```

```
/**
```

```
 * Общий множитель скорости. Кадры были слишком короткими
```

```
 * заметить, что именно переехало, и анимация читалась
```

```
 */
```

```
export const SPEED = 2.1;
```

```
/** Длительность переезда узлов. Должна быть заметно меньше
```

```
export const MOVE_MS = 800;
```

```
export const EASE = "cubic-bezier(.34,.01,.2,1)";
```

```
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;
```

```
/** Наведение на карточку ставит анимацию на паузу — мож
```

```
export const DemoPauseContext = createContext(false);
```

```
export function useLoop(steps: number, ms = 900) {
```

```
 const [step, setStep] = useState(0);
```

```
 const paused = useContext(DemoPauseContext);
```

```
 const period = Math.round(ms * SPEED);
```

```
 useEffect(() => {
```

```
 if (steps <= 1 || paused) return;
```

```
 const t = setInterval(() => setStep((s) => (s + 1) %
```

```
 return () => clearInterval(t);
```

```
 }, [steps, period, paused]);
```

```
 return step;
```

```
}
```

```

stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
 <g style={{ transition: MOVE }}>
 <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke}
 {label !== undefined && (
 <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
 {label}
 </text>
)}
 </g>
);

export const Edge: React.FC<{
 a: [number, number];
 b: [number, number];
 color?: string;
 width?: number;
 dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
 <line
 x1={a[0]}
 y1={a[1]}
 x2={b[0]}
 y2={b[1]}
 stroke={color}
 strokeWidth={width}
 strokeDasharray={dashed ? "4 3" : undefined}
 style={{ transition: MOVE }}
 />
);

```

ФАЙЛ 46/88: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```
import React from "react";
```

```

};

/** Компоненты связности: несколько «островов», которые
export const ComponentsDemo: React.FC = () => {
 const comps: [string, string][][] = [
 [["A", "B"], ["B", "C"]],
 [["D", "E"]],
 [["F", "G"], ["G", "H"], ["F", "H"]],
];
 const pos: Record<string, [number, number]> = {
 A: [26, 40], B: [62, 92], C: [26, 92],
 D: [120, 40], E: [120, 92],
 F: [190, 34], G: [232, 78], H: [178, 96],
 };
 const colors = [C.active, C.warn, C.done];
 const step = useLoop(comps.length + 1, 900);
 const compOf = (v: string) => comps.findIndex((e) =>

 return (
 <Svg caption={step === 0 ? "Граф распался на острова"
 {comps.flatMap((edges, ci) =>
 edges.map(([u, v], i) => {
 <Edge key={`_${ci}-${i}`} a={pos[u]} b={pos[v]}
 })
 })}
 {Object.entries(pos).map(([k, [x, y]]) => {
 const ci = compOf(k);
 return <Node key={k} x={x} y={y} r={12} label={k}
 })}
 </Svg>
);
};

```

```

/** Что такое граф: вершины, рёбра, направление и вес.
export const GraphBasicsDemo: React.FC = () => {
 const step = useLoop(3, 1300);
 const pos: Record<string, [number, number]> = { A: [50, 40], B: [150, 40],
 const edges: [string, string, number][] = [["A", "B",

```



/\*\* Сжатие координат: разрежённые числа → плотные индексы

```
export const CoordCompressDemo: React.FC = () => {
 const raw = [1000, 5, 74000, 5, 300];
 const sorted = [5, 300, 1000, 74000];
 const step = useLoop(3, 1200);
 const boxW = 46;
 const startX = (260 - raw.length * (boxW + 4)) / 2;

 return (
 <Svg
 caption={
 step === 0
 ? "Исходные координаты – огромные и с дырами"
 : step === 1
 ? "Сортируем уникальные значения"
 : "Каждое число заменяем его номером → плотные индексы"
 }
 >
 {raw.map((v, i) => {
 <g key={i}>
 <rect x={startX + i * (boxW + 4)} y={20} width={boxW} height={20} />
 <text x={startX + i * (boxW + 4) + boxW / 2} y={35} />
 {v}
 </text>
 </g>
 })}

 {step >= 1 &&
 sorted.map((v, i) => {
 <g key={v}>
 <rect x={30 + i * 52} y={62} width={46} height={20} />
 <text x={53 + i * 52} y={77} textAnchor="middle" />
 {v}
 </text>
 </g>
 })}

 {step >= 2 &&
```

```

 />
)})
 {Object.entries(f.pos).map(([id, [x, y]]) => {
 <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
 <circle r={13} fill={color(id)} stroke={C.idle} />
 <text textAnchor="middle" dominantBaseline="middle">
 {id}
 </text>
 </g>
 })}
 </Svg>
);
};

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => {
 <SplayFrames
 captions={["p – корень, x под ним", "Один поворот: p → x"]}
 frames={[
 { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p": "p", "x": "x"}]},
 { pos: { x: [130, 32], p: [182, 92] }, edges: [{"x": "p", "p": "x"}]}
]}
 />
};

/** Zig-Zig – x и p с одной стороны, крутим сначала верхнюю. */
export const ZigZigDemo: React.FC = () => {
 <SplayFrames
 captions={["Линия g → p → x («бамбук»)", "Поворот В: g → p → x"]}
 frames={[
 { pos: { g: [176, 24], p: [126, 70], x: [76, 112] }, edges: [{"g": "g", "p": "p", "x": "x"}]},
 { pos: { p: [130, 26], x: [78, 78], g: [186, 78] }, edges: [{"p": "p", "x": "x", "g": "g"}]},
 { pos: { x: [102, 24], p: [152, 70], g: [202, 112] }, edges: [{"x": "p", "p": "g", "g": "x"}]}
]}
 />
};

/** Zig-Zag – x и p с разных сторон, крутим сначала нижнюю. */

```

```

 <Node key={k} x={x} y={y} label={k}
 fill={wave.includes(k) ? C.active : visited.h
 stroke={wave.includes(k) ? "#a5b4fc" : undefi
)})
 </Svg>
);
};

```

```

export const DfsDemo: React.FC = () => {
 const path = ["A", "B", "D", "E", "F", "C"];
 const step = useLoop(path.length + 1, 750);
 const visited = path.slice(0, step);
 const head = visited[visited.length - 1];
 return (
 <Svg caption={head ? `Идём вглубь: ${visited.join("
 {GRAPH_EDGES.map(([u, v], i) => {
 const iu = visited.indexOf(u);
 const iv = visited.indexOf(v);
 const onPath = iu >= 0 && iv >= 0 && Math.abs(i
 return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_
 })})
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={k ===
)})
 </Svg>
);
};

```

```

export const ToposortDemo: React.FC = () => {
 const nodes = [
 { id: "труссы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
 { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
];
 const edges: [string, string][] = [
 ["труссы", "штаны"],
 ["носки", "боты"],
 ["штаны", "боты"]
];
 const order = ["труссы", "носки", "штаны", "боты"];
 const step = useLoop(order.length + 1, 800);
 const placed = order.slice(0, step);
 const pos = Object.fromEntries(nodes.map((n) => [n.id, {

```

```

 <text x={130} y={22} textAnchor="middle" fontSize=
 </Svg>
);
};

export const BridgeDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : ""}>
 {edges.map(([u, v], i) => {
 const isBridge = u === "C" && v === "D";
 if (isBridge && cut) return null;
 return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge ? "red" : "black"} />
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "black"} />
))}
 </Svg>
);
};

export const ArticulationDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали вершину C → две отдельные компоненты" : ""}>
 {edges.map(([u, v], i) => (cut && (u === "C" || v === "C") ? null : <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge ? "red" : "black"} />))}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "black"} />
))}
 </Svg>
);
};

```



```

 <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
 </marker>
 </defs>
 <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХОД</text>
 <text x={206} y={44} fontSize={9} fill={C.dim}>ВХОД</text>
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={30 + i * 54} y={56} width={46} height={20} fill={C.dim}></rect>
 <text x={53 + i * 54} y={71} textAnchor="middle">{v}</text>
 </g>
))}
 <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
 </Svg>
);
};

export const DsuDemo: React.FC = () => {
 const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
 const step = useLoop(parents.length, 950);
 const p = parents[step];
 const pos: [number, number][] = [[50, 95], [105, 95], [160, 95], [215, 95]];
 const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#ff7f0e"];
 const root = (i: number): number => (p[i] === i ? i : root(p[i]));
 return (
 <Svg caption="find = «кто твой староста», union = 0" >
 {p.map((par, i) => (
 par !== i ? (
 <path key={i} d={`M ${pos[i][0]} ${pos[i][1]} L ${pos[root(i)][0]} ${pos[root(i)][1]}`
 fill="none" stroke={rootColor[root(i)]} stroke-width={2} />
) : null
))}
 {pos.map(([x, y], i) => (<Node key={i} x={x} y={y} />))}
 </Svg>
);
};

const TRIE_NODES = [

```

```

 });
};

export const SuffixLinkDemo: React.FC = () => {
 const links: [number, number][] = [[1, 0], [2, 0], [3, 0]];
 const step = useLoop(links.length + 1, 900);
 return (
 <Svg caption="Суффиксная ссылка: «куда откатиться, чтобы найти суффикс»" >
 {TRIE_NODES.filter((n) => n.parent !== null).map((n) => {
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x, n.y]} />
 })}
 {links.slice(0, step).map(([from, to], i) => {
 const a = TRIE_NODES[from];
 const b = TRIE_NODES[to];
 return (
 <path key={i} d={`M ${a.x} ${a.y} Q ${(a.x + b.x) / 2} ${(a.y + b.y) / 2} ${b.x} ${b.y}`
 fill="none" stroke={C.warn} strokeWidth={1.5} />
);
 })}
 {TRIE_NODES.map((n) => (
 <Node key={n.id} x={n.x} y={n.y} label={n.ch} fill={C.warn} />
))}
 </Svg>
);
};

```

```

export const SegmentTreeDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const nodes = [
 { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
 { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
 { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
 { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
 { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
 { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
 { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
];
};

```

```

 >
 </g>
)))
 </Svg>
);
};

export const SparseTableDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const len = [1, 2, 4][step];
 const count = 8 - len + 1;
 return (
 <Svg caption={`Уровень j=${step}: готовые ответы дл
 {Array.from({ length: 8 }).map((_, i) => {
 <g key={i}>
 <rect x={12 + i * 30} y={18} width={26} height={18} />
 <text x={25 + i * 30} y={29} textAnchor="middle">
 {i}
 </text>
 </g>
 })}
 {Array.from({ length: count }).map((_, i) => {
 <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 18} width={26} height={18} />
 <text key={i} x={25 + i * 30} y={52 + (i % 4) * 18} textAnchor="middle">
 {i}
 </text>
 })}
 </Svg>
);
};

```

```

export const AmortizedDemo: React.FC = () => {
 const costs = [1, 1, 1, 8, 1, 1, 1, 1];
 const step = useLoop(costs.length + 1, 550);
 const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
 const avg = step ? (total / step).toFixed(2) : "-";
 return (
 <Svg caption={`Сделано ${step} операций, суммарно $
 {costs.map((c, i) => {
 <rect key={i} x={14 + i * 30} y={104 - c * 10} width={26} height={18} />
 <text key={i} x={25 + i * 30} y={104 - c * 10} textAnchor="middle">
 {i}
 </text>
 })}
 </Svg>
);
};

```



```

 fill={idx <= i && step > 0 ? C.warn : C.idle}
 </g>
)))
 <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
 <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
 <text x={130} y={106} textAnchor="middle" fontSize={10}
 π[i] – длина самого длинного «префикс = суффикс»
 </text>
 </Svg>
);
};

export const DpDemo: React.FC = () => {
 const rows = 3, cols = 5;
 const step = useLoop(rows * cols + 1, 320);
 return (
 <Svg caption={`Заполняем таблицу подзадач: готово $
 {Array.from({ length: rows }).map((_, r) =>
 Array.from({ length: cols }).map((_, c) => {
 const idx = r * cols + c;
 const filled = idx < step;
 return (
 <g key={idx}>
 <rect x={40 + c * 38} y={22 + r * 32} width={38} height={32}
 fill={idx === step - 1 ? C.active : filled ? C.warn : C.idle}
 </rect>
 {filled && (
 <text x={57 + c * 38} y={36 + r * 32} textAnchor="middle"
 fontSize={10} fontWeight="700" fill={C.dim}>
 {π[idx]}
 </text>
)}
 </g>
);
 })
)}
 </Svg>
);
};

```

```

 relax: RelaxDemo,
 "prefix-sum": PrefixSumDemo,
 "sparse-table": SparseTableDemo,
 "segment-tree": SegmentTreeDemo,
 bridge: BridgeDemo,
 articulation: ArticulationDemo,
 mst: MstDemo,
 amortized: AmortizedDemo,
 np: NpDemo,
 scc: SccDemo,
 "prefix-function": PrefixFunctionDemo,
 dp: DpDemo,
 floyd: FloydDemo,
 components: ComponentsDemo,
 graph: GraphBasicsDemo,
 "coord-compress": CoordCompressDemo,
 zig: ZigDemo,
 "zig-zig": ZigZigDemo,
 "zig-zag": ZigZagDemo,
};

export const MiniDemo: React.FC<{ kind: DemoKind }> = (
 const [paused, setPaused] = useState(false);
 const Cmp = REGISTRY[kind];
 if (!Cmp) return null;
 return (
 <DemoPauseContext.Provider value={paused}>
 <div
 className="bg-slate-950/70 rounded-lg border bo
 onMouseEnter={() => setPaused(true)}
 onMouseLeave={() => setPaused(false)}
 >
 <Cmp />
 </div>
 </DemoPauseContext.Provider>
);
};

```

```

<div className="relative w-full sm:max-w-5xl max-w-4xl shadow-2xl flex flex-col">
 <div className="flex items-center gap-3 px-4 py-4">
 <div className="min-w-0 flex-1">
 <h3 className="font-bold text-white text-sm">
 {viz.hint && <p className="text-[11px] text-white">
 </p>
 }
 </h3>
 </div>
 <button
 onClick={onClose}
 className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700"
 aria-label="Заккрыть симулятор"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="overflow-y-auto overscroll-contain">
 <Component />
 </div>
</div>
</div>,
document.body
);
};

```

ФАЙЛ 51/88: src/components/hints/TermPopover.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useEffect, useLayoutEffect, useRef, useState } from "react";
import { createPortal } from "react-dom";
import { Play, X, Lightbulb } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";

```

```

export interface HintAnchor {

```

```

 if (top + height > vh - 8) top = Math.max(8, vh -
 } else {
 top = anchor.rect.top - height - GAP;
 if (top < 8) top = anchor.rect.bottom + GAP;
 }

 top = Math.max(8, Math.min(top, vh - height - 8));
 setPos({ top, left, below });
}, [anchor]);

useLayoutEffect(() => {
 if (!anchor) {
 setPos(null);
 return;
 }
 // Measure after paint; use rAF to ensure height is
 const id = requestAnimationFrame(() => {
 recompute();
 // second tick for accurate height after content
 requestAnimationFrame(recompute);
 });
 return () => cancelAnimationFrame(id);
}, [anchor, recompute]);

useEffect(() => {
 if (!anchor) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 const onScroll = () => {
 // Recompute instead of close for better UX; close
 recompute();
 // If anchor element is no longer in viewport, cl
 if (anchor.rect.top < -100 || anchor.rect.top > w
 };
 const onResize = () => recompute();
 window.addEventListener("keydown", onKey);
 window.addEventListener("scroll", onScroll, true);

```

```

 aria-label="Заккрыть подсказку"
 >
 <X className="w-4 h-4" />
 </button>
 </div>

 <div className="p-3 space-y-2.5 max-h-[60vh] overflow-hidden" >
 <p className="text-[13px] leading-relaxed text-slate-500">
 {entry.demo && <MiniDemo kind={entry.demo} />
 {entry.formal && (
 <p className="text-[11.5px] leading-relaxed text-slate-500">
 {entry.formal}
 </p>
)}
 {entry.complexity && (
 <div className="text-[11px] font-mono text-slate-500 flex items-center gap-1.5">

 Сложность: {entry.complexity}
 </div>
)}
 </p>
 {viz && simulatorId && (
 <button
 onClick={() => onOpenSimulator(simulatorId)}
 className="w-full flex items-center justify-center text-slate-500 rounded-lg py-2.5 transition-colors shadow-md"
 >
 <Play className="w-3.5 h-3.5" /> Показать
 </button>
)}
 </div>
 <div className="text-[10px] text-slate-500 text-center">
 Нажми Esc или кликни вне, чтобы закрыть •
 </div>
 </div>
</div>

```

```

const edges = [
 { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 0, labelDx: 0, labelDy: 0 },
 { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 0, labelDx: 0, labelDy: 0 },
 { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 0, labelDx: 0, labelDy: 0 },
];

const isNodeVisible = (id: string) => {
 if (id === 'S' && (step === 0 || step === 7)) return true;
 return false;
};

const getPotential = (id: string) => {
 if (step >= 2 && step < 7) {
 if (id === 'S') return 0;
 if (id === 'A') return 0;
 if (id === 'B') return -2;
 if (id === 'C') return -1;
 }
 return null;
};

const getEdgeColorClass = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "opacity-0";
 if (step === 1) return "stroke-amber-500 stroke-dasharray-4";
 if (step === 2) return "stroke-pink-500 stroke-dasharray-4";
 return "stroke-slate-700 stroke-dasharray-4";
 }

 if (e.id === 'AB') {
 if (step === 4) return "stroke-amber-400 stroke-dasharray-4";
 if (step > 4) return "stroke-emerald-500";
 }

 if (e.id === 'BC') {
 if (step === 5) return "stroke-amber-400 stroke-dasharray-4";
 }
};

```

```

 if (e.id === 'BC') {
 if (step < 5) return e.weight;
 if (step === 5) return `${e.weight} + (${ -2`
 return e.newWeight;
 }
 if (e.id === 'CA') {
 if (step < 6) return e.weight;
 if (step === 6) return `${e.weight} + (${ -1`
 return e.newWeight;
 }
 return e.weight;
 };

const isEdgeTextHighlighted = (e: any) => {
 if (e.id === 'AB' && step === 4) return true;
 if (e.id === 'BC' && step === 5) return true;
 if (e.id === 'CA' && step === 6) return true;
 return false;
};

const isEdgeTextEmerald = (e: any) => {
 if (e.id === 'AB' && step > 4) return true;
 if (e.id === 'BC' && step > 5) return true;
 if (e.id === 'CA' && step > 6) return true;
 return false;
};

const getDesc = () => {
 switch (step) {
 case 0: return (
 <div>
 Исходный граф. Имеется ребро
 Если мы запустим быстрый алгоритм Д
 </div>
);
 case 1: return (
 <div>
 Шаг 1: Точка отсчета.


```

```

 Старый вес: 2.

 Вычисляем: $2 + (-1) - 0 =$ <span cla
 </div>
);
 case 7: return (
 <div className="text-emerald-300">
 Готово! Фиктивную вершину S
 Все новые веса ребер в графе стали
 При этом старые кратчайшие пути оста
шин!
 </div>
);
 default: return "";
 }
};

return (
 <div className="bg-slate-800 p-4 rounded-xl border
 <h4 className="text-sm md:text-base font-bo
 <span className="w-2.5 h-2.5 bg-amber-4
 Визуализация: Перевзвешивание алгоритмом
 </h4>

 <div className="flex flex-col md:flex-row g
 <div className="bg-[#0f172a] flex-1 min
 nter p-4">
 <svg width="400" height="300" class
 <defs>
 <marker id="arrow-slate" vi
 start-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>
 <marker id="arrow-slate-dim
 uto-start-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>
 <marker id="arrow-amber" vi
 start-reverse">

```



```

 dominantBaseline="bottom"
 className={`text-${
 isS ? 'fill-pink-400' : 'fill-white'
 }}
 >
 {getEdgeContent(e)}
 </text>
 </g>
)
 }
 }

 {/** Nodes */}
 {nodes.map(n => {
 if (!isNodeVisible(n.id)) return null
 const pot = getPotential(n.id)

 return (
 <g key={n.id} className={`node-${
 isS ? 'fill-pink-400' : 'fill-white'
 }}
 <circle
 cx={n.x} cy={n.y}
 className={`stroke-${
 isS ? 'stroke-pink-400' : 'stroke-white'
 }}
 />
 <text x={n.x} y={n.y}
 className={`text-${
 isS ? 'fill-pink-400' : 'fill-white'
 }}
 >
 {n.label}
 </text>

 {pot !== null && (
 <g transform={`translate(${n.x}, ${n.y})`}
 fade-in zoom-in duration-500">
 <rect x="-100" y="-100"
 width="200" height="200"
 stroke="black" stroke-width="1" />
 <text x="0" y="0"
 font-size="12"
 font-weight="bold" fill="pink-400">

```

```
 {/* Progress Dots */}
 <div className="flex justify-center"
 {Array.from({length: MAX_STEP +
 <div key={i} className={`w-12 h-12 rounded-full
125' : i < step ? 'bg-indigo-900' : 'bg-slate-500'
)}}
 </div>
 </div>
 </div>
</div>
);
}
```

---

ФАЙЛ 53/88: src/components/KosarajuViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useEffect } from "react";
import {
 Play,
 Pause,
 RotateCcw,
 Layers,
} from "lucide-react";
```

```
interface GNode {
 id: number;
 x: number;
 y: number;
 label: string;
}
```

```
interface GEdge {
 u: number;
 v: number;
```

```

const generatedSteps: AlgoStep[] = [];
const visited = new Set<number>();
const order: number[] = [];
const sccMap: Record<number, number> = {};

const recordStep = (
 phase: AlgoStep["phase"],
 desc: string,
 currentNode: number | null,
 transposed: boolean,
 currentSccId: number,
) => {
 generatedSteps.push({
 phase,
 desc,
 visited: new Set(visited),
 currentNode,
 order: [...order],
 transposed,
 sccMap: { ...sccMap },
 currentSccId,
 activeEdges: transposed
 ? initialEdges.map((e) => ({ u: e.v, v: e.u })
 : [...initialEdges],
));
};

recordStep(
 "init",
 "Инициализация алгоритма Косарайю. Начинаем первый",
 null,
 false,
 -1,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
initialEdges.forEach((e) => adj[e.u].push(e.v));

```

```

 `Первый проход завершен! Стек выхода в топологиче
 ебра.` ,
 null,
 true,
 -1,
);

 // Transpose adj
 const adjT: number[][] = Array.from({ length: 5 },
 initialEdges.forEach((e) => adjT[e.v].push(e.u));

 // Reset visited for phase 2
 visited.clear();
 let sccId = 0;

 const dfs2 = (u: number, currentScc: number) => {
 visited.add(u);
 sccMap[u] = currentScc;
 recordStep(
 "dfs2",
 `DFS 2: заходим в ${u} на транспонированном графе`,
 u,
 true,
 currentScc,
);

 for (const to of adjT[u]) {
 if (!visited.has(to)) {
 dfs2(to, currentScc);
 }
 }
 };

 // Run DFS2 using the order (reversed from back to front)
 for (let i = order.length - 1; i >= 0; i--) {
 const u = order[i];
 if (!visited.has(u)) {
 sccId++;

```

```

 sccMap: {},
 currentSccId: -1,
 activeEdges: initialEdges,
 });

useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1)
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1500);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getSccColor = (id: number) => {
 if (id === 1) return "fill-emerald-600 stroke-emera
 if (id === 2) return "fill-indigo-600 stroke-indigo
 return "fill-slate-800 stroke-slate-500";
};

return (
 <div className="bg-slate-900 rounded-xl border bord
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Layers className="w-5 h-5 text-emerald-400"
 Визуализатор Алгоритма Косарайю (Поиск SCC)
 </h3>
 <div className="flex items-center gap-2 bg-slate

```

```

 <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
 </marker>
 <marker
 id="arrow-active"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#1
 </marker>
 <marker
 id="arrow-transposed"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
 </marker>
 </defs>

```

```

{ /* Edges */ }
{initialEdges.map((edge, idx) => {
 const u = initialNodes.find((n) => n.id ==
 const v = initialNodes.find((n) => n.id ==

 // If transposed, draw reversed coordinates
 const x1 = step.transposed ? v.x : u.x;
 const y1 = step.transposed ? v.y : u.y;
 const x2 = step.transposed ? u.x : v.x;
 const y2 = step.transposed ? u.y : v.y;

 const isSccEdge =

```

```

 }}}

 { /* Nodes */
 {initialNodes.map((node) => {
 const isCurrent = step.currentNode === node
 const isVisited =
 step.visited.has(node.id) ||
 (step.phase === "dfs1" && step.currentNode === node)
 const sccId = step.sccMap[node.id];

 let classes = "fill-slate-800 stroke-slate-800"
 if (sccId) {
 classes = getSccColor(sccId);
 } else if (isCurrent) {
 classes = "fill-amber-600 stroke-amber-600"
 } else if (isVisited && step.phase === "dfs1") {
 classes = "fill-emerald-700 stroke-emerald-700"
 }

 return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r="20"
 className={`transition-all duration-200ms`}
 strokeWidth="3"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="13px"
 fontWeight="bold"
 className="select-none"
 />
 </g>
)
 })}
)}

```

```

 </p>
 </div>

 <div className="flex-1 space-y-4 overflow-y-auto" >
 {/* Top-Sort output / Order stack representation */}
 <div>

 СТЕК ВЫХОДА (DFS 1 ORDER):

 <div className="flex flex-wrap gap-1 bg-slate-900" >
 {step.order.length === 0 ? (

 Пустой стек

) : (
 [...step.order].reverse().map(({val, idx}) => {
 <div
 key={idx}
 className="bg-indigo-950 border border-slate-800 px-2 py-1"
 >
 {val}
 </div>
 })
)}
 </div>
 </div>

 {/* List of actions log */}
 <div>

 ЛОГ ОПЕРАЦИЙ:

 <div className="space-y-1.5 max-h-[150px]" >
 {steps
 .slice(0, currentStepIdx + 1)
 .reverse()
 .slice(0, 5)
 .map((s, idx) => {
 <div

```



ФАЙЛ 54/88: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, CheckCircle } from 'lucide-react';
```

```
interface Vertex {
 id: number;
 label: string;
 x: number;
 y: number;
 color: string; // 'none', 'red', 'blue', 'purple', 'magenta'
}
```

```
interface Edge {
 id: string;
 u: number;
 v: number;
 weight: number;
 status: 'pending' | 'inspecting' | 'included' | 'rejected';
 color: string;
}
```

```
interface StepLog {
 title: string;
 description: string;
 type: 'info' | 'success' | 'warning' | 'error';
}
```

```
// 9 Vertices layout
```

```
const initialVertices: Vertex[] = [
 { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
 { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
 { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
```

```

));

// --- KRUSKAL STEPS ---
const kruskalSteps = [
 // Step 1: Inspect A-B (2)
 () => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 1: Берем самое дешевое ребро A-B (в
 description: 'Оно соединяет две бесцветные верши
 type: 'info'
 }, ...prev]));
 },
 // Step 2: Include A-B
 () => {
 setVertices(prev => prev.map(v => v.id === 0 || v
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро A-B в осто́ве',
 description: 'Вершины A and B окрашены в красны
 type: 'success'
 }, ...prev]));
 },
 // Step 3: Inspect D-E (3)
 () => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
 description: 'Оно соединяет две другие бесцветн
 type: 'info'
 }, ...prev]));
 },
 // Step 4: Include D-E
 () => {
 setVertices(prev => prev.map(v => v.id === 3 || v
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-E в осто́ве',

```

```

 title: 'Слияние кланов: Ребро A-D добавлено',
 description: 'Мы объединили красный и синий клан',
 type: 'success'
 }, ...prev]));
},
// Step 9: Inspect B-E (6) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Берем ребро B-E (вес 6)',
 description: 'НО: это ребро соединяет вершины B',
 type: 'warning'
 }, ...prev]));
},
// Step 10: Reject B-E
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Мы выбрасываем ребро B-E. Иначе п',
 type: 'error'
 }, ...prev]));
},
// Step 11: Inspect E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Берем ребро E-F (вес 7)',
 description: 'Оно соединяет фиолетовую вершину',
 type: 'info'
 }, ...prev]));
},
// Step 12: Include E-F
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро E-F в осто́ве',

```

```

 type: 'error'
 }, ...prev]));
 },
 // Step 17: Inspect G-H (10)
 () => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Берем ребро G-H (вес 10)',
 description: 'Соединяет фиолетовую G и бесцветную H',
 type: 'info'
 }, ...prev]));
 },
 // Step 18: Include G-H
 () => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро G-H в осто́ве',
 description: 'Вершина H перекрашена в фиолетовый',
 type: 'success'
 }, ...prev]));
 },
 // Step 19: Inspect E-H (11) - Cycle!
 () => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Берем ребро E-H (вес 11)',
 description: 'Оба конца E и H уже фиолетовые. Соединяем их!',
 type: 'warning'
 }, ...prev]));
 },
 // Step 20: Reject E-H
 () => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро E-H.',
 type: 'error'
 }, ...prev]));
 },

```

```
// Step 2: Pop D-E (3)
() => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Куча выдает самое дешевое ребро D-E (3)',
 description: 'Плесень тянется по ребру D-E к вершине E',
 type: 'info'
 }, ...prev]));
},
// Step 3: Include D-E, add D's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 3 ? {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 ..e, status: 'in_heap' } : e));
 setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D-F (8)']);
 setLogs(prev => [{
 title: 'Успех: Плесень захватила вершину D',
 description: 'В кучу добавлены новые пути из D: A-D (5), B-E (6), E-F (7), D-F (8)',
 type: 'success'
 }, ...prev]));
},
// Step 4: Pop A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
 description: 'Плесень расширяется в сторону вершины A',
 type: 'info'
 }, ...prev]));
},
// Step 5: Include A-D, add A's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 0 ? {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 ..e, status: 'in_heap' } : e));
 setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D-F (8)']);
 setLogs(prev => [{
 title: 'Успех: Плесень поглотила вершину A',
 description: 'В кучу добавлены новые пути из A: A-B (2), B-E (6), E-F (7), D-F (8)',
 type: 'success'
 }, ...prev]));
},
```

```

 ear' } : e));
 setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (8)']);
 setLogs(prev => [{
 title: 'Успех: Вершина C захвачена',
 description: 'Ребро C-F(9) добавлено в кучу.',
 type: 'success'
 }, ...prev]));
},
// Step 10: Pop B-E (6) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Куча выдает ребро B-E (вес 6)',
 description: 'ВНИМАНИЕ! Плесень проверяет вершину B',
 type: 'warning'
 }, ...prev]));
},
// Step 11: Reject B-E
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'B-E (6)']);
 setLogs(prev => [{
 title: 'Отказ: Игнорируем внутреннее ребро',
 description: 'Ребро B-E отброшено. Плесень не р',
 type: 'error'
 }, ...prev]));
},
// Step 12: Pop E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
 description: 'Плесень из центра Токио (E) тянет',
 type: 'info'
 }, ...prev]));
},
// Step 13: Include E-F, add F's edges
() => {

```

```

 }, ...prev]));
},
// Step 17: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
 setLogs(prev => [{
 title: 'Отказ: Игнорируем ребро C-F',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]));
},
// Step 18: Pop G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
 description: 'Плесень ползет к вершине H.',
 type: 'info'
 }, ...prev]));
},
// Step 19: Include G-H, add H's edges
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 eap' } : e));
 setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I (13)'])
 setLogs(prev => [{
 title: 'Успех: Вершина H захвачена',
 description: 'В кучу добавлено H-I(12).',
 type: 'success'
 }, ...prev]));
},
// Step 20: Pop E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Куча выдает E-H (вес 11)',

```

```

const boruvkaSteps = [
 // Step 1: Five-Year Plan 1 (Inspecting outgoing)
 () => {
 // Highlight the cheapest outgoing edges for EACH
 // Node 0(A) -> A-B(2)
 // Node 1(B) -> A-B(2)
 // Node 2(C) -> B-C(4)
 // Node 3(D) -> D-E(3)
 // Node 4(E) -> D-E(3)
 // Node 5(F) -> E-F(7)
 // Node 6(G) -> D-G(8)
 // Node 7(H) -> G-H(10)
 // Node 8(I) -> H-I(12)
 setEdges(prev => prev.map(e =>
 ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8'
]));
 setLogs(prev => [{
 title: 'Первая пятилетка: Каждая деревня выбирает',
 description: 'Каждая из 9 деревень одновременно выбирает',
 text: 'A выбирает A-B(2), D выбирает D-E(3), а G выбирает D-G(8).',
 type: 'info'
 }, ...prev]);
 },
 // Step 2: Concurrently merge into Kolkhozes (Components)
 () => {
 // Concurrently build chosen roads. Merge into:
 // Component 1: {A, B, C} (colored red)
 // Component 2: {D, E, F, G, H, I} (colored gold/blue)
 setVertices(prev => prev.map(v =>
 [0, 1, 2].includes(v.id) ? { ...v, color: 'red' } : v
));
 setEdges(prev => prev.map(e => {
 if (['0-1', '1-2'].includes(e.id)) {
 return { ...e, status: 'included', color: 'red' };
 }
 if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
 return { ...e, status: 'included', color: 'blue' };
 }
 return e;
 }));
 }
];

```





```

 if (color === 'red') return '#ef4444';
 if (color === 'blue') return '#3b82f6';
 if (color === 'purple') return '#a855f7';
 if (color === 'mold') return '#10b981';
 }
 return '#475569';
};

return (
 <div className="space-y-12">

 {/* 3-in-1 MST Simulator Box */}
 <div className="bg-slate-900/90 border border-slate-800" >
 <div className="flex flex-col lg:flex-row justify-between" >
 <div>
 <div className="flex items-center gap-2 mb-2" >

 Сунер-Интерактив 3 в 1

 <h3 className="text-2xl font-bold text-white" >
 MST
 </h3>
 </div>
 <p className="text-slate-400 text-sm" >
 Наблюдай за работой Краскала (Раскраска),
 </p>
 </div>

 {/* Algorithm Tabs */}
 <div className="flex flex-wrap items-center gap-2" >
 <div className="flex items-center bg-slate-800/50" >
 <button
 onClick={() => handleReset('kruskal')}
 className={
 "flex items-center gap-1.5 px-3 py-2"
 +
 (activeAlgo === 'kruskal'
 ? 'bg-indigo-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-300')
 }
 >
 Краскала
 </button>
 </div>
 </div>
 </div>
 </div>
)
)

```

```

 <RotateCcw className="w-3.5 h-3.5" />
 Сбросить
 </button>

 <button
 onClick={handleNextStep}
 disabled={stepIndex >= currentSteps.length}
 className={
 "flex items-center justify-center gap-2
initial cursor-pointer " +
 (stepIndex >= currentSteps.length
 ? "bg-slate-800 text-slate-500 border
: activeAlgo === 'kruskal'
 ? "bg-indigo-600 hover:bg-indigo-500
: activeAlgo === 'prim'
 ? "bg-emerald-600 hover:bg-emerald-500
 : "bg-rose-600 hover:bg-rose-500 text
)
 >
 <SkipForward className="w-4 h-4" />
 {stepIndex === 0 ? 'Начать' : stepIndex}
 </button>
 </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3"
 /* Graph Canvas */
 <div className="lg:col-span-2 bg-slate-950 rounded
 active min-h-[480px] overflow-hidden">

 /* Legend */
 <div className="absolute top-4 left-4 bg-slate-900
 -center gap-3 text-xs">
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-
 </div>
 {activeAlgo === 'kruskal' ? (
 <>

```

```

 { /* SVG Edges */ }
 <svg className="absolute inset-0 w-full h-full"
 spectRatio="none">
 {edges.map(edge => {
 const u = vertices[edge.u];
 const v = vertices[edge.v];
 const strokeColor = getEdgeStroke(edge);
 const isInspecting = edge.status === 'inspect';
 const isRejected = edge.status === 'rejected';
 const isInHeap = edge.status === 'in_heap';

 return (
 <g key={edge.id}>
 <line
 x1={u.x + "%"}
 y1={u.y + "%"}
 x2={v.x + "%"}
 y2={v.y + "%"}
 stroke={strokeColor}
 strokeWidth={isInspecting ? 4.5 : 1}
 strokeDasharray={isRejected ? '4,4' : ''}
 className={"transition-all duration-300"}
 />
 <text
 x={({u.x + v.x} / 2 + "%")}
 y={({u.y + v.y} / 2 + "%")}
 fill={isInspecting ? '#f59e0b' : '#94a3b8'}
 : '#94a3b8'}
 fontSize="4.5"
 fontFamily="monospace"
 fontWeight="bold"
 textAnchor="middle"
 dominantBaseline="middle"
 className="select-none filter drop-shadow"
 dy="-1.5"
 >
 {edge.weight}
 </text>
 </g>
);
 })}
 </svg>
);
}

```

```

 { /* Logs / Heap */ }
 <div className="bg-slate-950 rounded-2xl border-2 border-b
 <div className="p-4 bg-slate-900 border-b border-2 border-
 <h4 className="font-bold text-slate-200 text-sm">
 Ход выполнения
 </h4>
 </div>

 {activeAlgo === 'prim' && {
 <div className="p-3 bg-slate-900/60 border-2 border-
 <p className="text-[11px] font-bold text-slate-200">
 ✂ Приоритетная очередь (Min-Heap):
 </p>
 <div className="flex flex-wrap gap-1.5">
 {heapQueue.length === 0 ? (

 Нет элементов

) : (
 heapQueue.map((item, idx) => (
 <span key={idx} className={"px-2 py-1 text-xs font-mono" +
 " shadow-md animate-pulse" : "bg-slate-800 text-slate-200"}>
 {item}

))
)}
 </div>
 </div>
)}

 <div className="flex-1 p-4 overflow-y-auto">
 {logs.map((log, idx) => (
 <div
 key={idx}
 className={
 "p-4 rounded-xl border transition-all duration-200" +
 (log.type === 'success'
 ? 'bg-emerald-950/40 border-emerald-500/40'
 : log.type === 'warning'
 ? 'bg-amber-950/40 border-amber-500/40'
 : 'bg-slate-950/40 border-slate-500/40')
 >

```

```

 className={"px-3 py-1.5 rounded text-xs font-medium"}
 == 'prim' ? "bg-emerald-600 text-white" : "bg-white text-slate-700"
 >
 Прима (Билет 19)
 </button>
 <button
 onClick={() => setActiveCheatTab('boruvka')}
 className={"px-3 py-1.5 rounded text-xs font-medium"}
 == 'boruvka' ? "bg-rose-600 text-white" : "bg-white text-slate-700"
 >
 Боровка (Билет 20)
 </button>
 </div>
 </div>

```

```

{activeCheatTab === 'kruskal' && {
 <div className="grid grid-cols-1 lg:grid-cols-2" >
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Clock className="w-3.5 h-3.5 text-inherit" />
 </p>
 <p className="text-2xl font-black text-white">Крускал</p>
 <p className="text-slate-400 text-xs font-medium">Минимальное
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Award className="w-3.5 h-3.5 text-inherit" />
 </p>
 <p className="text-2xl font-black text-white">Авард</p>
 <p className="text-slate-400 text-xs font-medium">Максимальное
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 ожении:</p>
 <p className="text-xs text-slate-300 leading-relaxed">
 ди и красим вершины в один цвет, бракуя циклы.
 </p>
 </div>
 </div>
 </div>

```









```


 <List className="w-4 h-4" />

 Содержание • {index + 1} из {total}

 {children}

 </button>
 </div>

 {open && (
 <div className="lg:hidden fixed inset-0 z-[90]" style={{background: 'black', color: 'white'}}>
 <div className="absolute inset-0 bg-slate-950" />
 <div className="absolute inset-y-0 left-0 w-[100px] flex flex-direction column items-center justify-center text-white">
 <div className="flex items-center justify-between">
 Закреть
 <button
 onClick={() => setOpen(false)}
 className="p-2 -mr-2 text-slate-400 hover:text-white"
 aria-label="Закреть содержание"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 </div>
 <div className="flex-1 min-h-0">
 <Sidebar
 selectedChapterId={selectedChapterId}
 setSelectedChapterId={setSelectedChapterId}
 onNavigate={() => setOpen(false)}
 />
 </div>
 </div>
)}
</>

```

```
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-slate-800">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-extrabold">
 Подготовка к экзаменам: Алгоритмы, Структуры
 </p>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-auto">
 >
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Cpu className="w-4 h-4" /> Тренажёры
 </button>
```

```
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center
er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
</div>
</header>
);
};
```

---

ФАЙЛ 58/88: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```
export function PlanarityDemo() {
 return (
 <div className="space-y-4">
 <h4 className="text-base font-bold text-white">K5
 <p className="text-xs text-slate-400">Просто смотр

 <div className="grid grid-cols-1 md:grid-cols-2 g
 { /* K5 */ }
 <div className="bg-slate-950 rounded-xl p-4 bor
 <div className="absolute top-2 left-2 bg-slate
 00/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0
 {(() => {
 const pts = [
 {id:0,x:160,y:40},
 {id:1,x:270,y:100},
 {id:2,x:230,y:230},
 {id:3,x:90,y:230},
```

```

 <text x={p.x} y={p.y+3} textAnchor="m
 </g>);
 }
 return <>{lines}{circles}</>;
 })()}
</svg>
<div className="absolute bottom-2 left-2 right
 z-20">
 V=5, E=10 {'>'} 3V-6 = 9 {'-->'} {'\u274C'}
</div>
</div>

{/* K3,3 */}
<div className="bg-slate-950 rounded-xl p-4 bor
 <div className="absolute top-2 left-2 bg-slate
 00/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0
 {(() => {
 const pts = [
 {id:0,x:60,y:60},{id:1,x:160,y:60},{id:2,x:260,y:60},
 {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:260,y:220},
];
 const edges = [
 [0,3],[0,4],[0,5],
 [1,3],[1,4],[1,5],
 [2,3],[2,4],[2,5]
];
 function findPt(id:number) { return pts.find(p => p.id === id); }
 function intersect(a:number,b:number,c:number,d:number) {
 if (a===c||a===d||b===c||b===d) return true;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 if (!p1||!p2||!p3||!p4) return false;
 function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
 }
 })}
 </svg>
</div>

```

---

ФАЙЛ 59/88: src/components/PythonCompiler.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useCallback, useEffect, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";
import { getFallbackMeta } from "../data/compilerVars";
```

```
const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
```

```
const STARTER_CODE = `# Первый запуск загрузит Python в интерпретатор
name = "алгоритмы"
`;
```

```
for number in range(1, 4):
 print(f"{number}. Учим {name}!")
`;
```

```
type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};
```

```
type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};
```

```
let runtimePromise: Promise<PyodideRuntime> | null = null;
```

```
function getPythonRuntime() {
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL);
 }
}
```

```

const hook = (vars: Record<string, string>) => setLiveVars(
 (globalThis as unknown as { __compilerSyncHook?: typeof onSync })
 .__compilerSyncHook ?? onSync
);
const onSync = (e: Event) => {
 const d = (e as CustomEvent).detail;
 if (!d?.vars) return;
 if (chapterId && d.chapterId && d.chapterId !== chapterId) return;
 setLiveVars(d.vars);
};
window.addEventListener("viz:sync", onSync as EventListener);
return () => window.removeEventListener("viz:sync", onSync);
}, [chapterId]);

// обновить код при смене главы
useEffect(() => {
 setCode(syncedStarter);
 setLiveVars(null);
}, [chapterId]);

const runCode = useCallback(async () => {
 if (running) return;
 setRunning(true);
 setOutput("");
 setRuntimeMessage(runtimeReady ? "Выполняю код..." : "Запускаю код...");
 const stdout: string[] = [];
 const stderr: string[] = [];
 const inputLines = stdin.split(/\r?\n/);
 try {
 const runtime = await getPythonRuntime();
 setRuntimeReady(true);
 setRuntimeMessage("Python готов – код выполняется");
 runtime.setStdout({ batched: (message) => stdout.push(message) });
 runtime.setStderr({ batched: (message) => stderr.push(message) });
 runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
 await runtime.runPythonAsync(code);
 const text = [...stdout, ...stderr].join("");
 setOutput(text || "Готово: программа ничего не вы输");
 } catch (error) {
 const captured = [...stdout, ...stderr].join("");
 setOutput(captured || "Ошибка: программа завершилась с ошибкой");
 }
}, [code, stdin]);

```

```


 })
 </p>
 </div>
 <a href="https://pyodide.org/" target="_blank"
 00 hover:text-white transition-colors">
 Как это работает <ExternalLink className="w

</div>

{liveVars && (
 <div className="mb-4 rounded-xl border border
 <span className="inline-flex items-center g
 <span className="w-2 h-2 rounded-full bg-

 {Object.entries(liveVars).map(([k, v]) => (
 <span key={k} className="text-xs font-mon
 {k}={String(v)}

))}
 <button onClick={() => setLiveVars(null)} c
 скрыть
 </button>
 </div>
)}

<div className="grid xl:grid-cols-[minmax(0,1.33
 <section className="rounded-2xl border border
 <div className="flex flex-wrap items-center
 <div className="flex items-center gap-2">
 <span className="w-2.5 h-2.5 rounded-fu
 <span className="text-sm font-bold text
 <span className="text-[11px] text-slate
 {meta && <span className="text-[10px] b
 </div>
 <div className="flex items-center gap-2">
 {meta && (
 <button type="button" onClick={insert

```



```
 <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300">{output}</pre>
 <div className="border-t border-slate-800 p-4">
 </section>
 </div>

 <div className="mt-5 grid md:grid-cols-3 gap-3">
 <div className="flex gap-2 rounded-xl border border-slate-800 p-4 k-0 text-indigo-400 mt-0.5" />Ctrl/Cmd + C
 <div className="flex gap-2 rounded-xl border border-slate-800 p-4 k-0 text-amber-400 mt-0.5" />Первый элемент
 <div className="flex gap-2 rounded-xl border border-slate-800 p-4 k-0 text-emerald-400 mt-0.5" />Код в файле
 </div>
 </div>
 </main>
);
}
```

---

ФАЙЛ 60/88: src/components/QueueViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';
```

```
interface Customer {
 id: string;
 name: string;
 emoji: string;
 color: string;
 bubbleText: string;
 animState: 'in' | 'idle' | 'eating' | 'out';
}
```

```
const PRESETS = [
```

```

 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PRESETS[counter % PRESETS.length];
 counter++;

 const newGuest: Customer = {
 id: Date.now().toString(),
 ...preset,
 animState: 'in',
 };

 setQueue(prev => [...prev, newGuest]);
 addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoji}`);

 setTimeout(() => {
 setQueue(prev => prev.map(c => c.id === newGuest.id ? newGuest : c));
 }, 450);
}, [queue, isServing]);

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
 if (isServing) return;
 if (queue.length === 0) {
 setShake(true);
 addLog(`⚠ DEQUEUE: В очереди никого нет! Повар скучает`);
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsServing(true);
 const headGuest = queue[0];

 addLog(`- DEQUEUE: Повар наливает суп первому – ${headGuest.name} ${headGuest.emoji}`);

 // Step 1: Head guest gets eating animation

```

```

<div className="text-3xl"> </div>
<div>
 <h3 className="font-black text-indigo-400 tex
 <p className="text-xs text-slate-300 mt-1 lea
 В очереди за супом всё честно. Кто первый в
 Новые гости встают строго в конец оч
 Здесь нет обхода и блата – только строгий п
 </p>
</div>
</div>

{/* УПРАВЛЕНИЕ */}
<div className="flex items-center gap-3 flex-wrap
 <button
 onClick={enqueue}
 disabled={isServing}
 className="flex-1 min-w-[150px] bg-gradient-t
 d:opacity-40 disabled:cursor-not-allowed te
 ive:scale-95 transition-all cursor-pointer
 >
 Встать в очередь (ENQUEUE)
 </button>

 <button
 onClick={dequeue}
 disabled={isServing || queue.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-t
 opacity-40 disabled:cursor-not-allowed text
 :scale-95 transition-all cursor-pointer fle
 >
 Налить суп первому (DEQUEUE)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-8
 r-pointer border border-slate-700"
 >

```

```

 <span className="text-[8px] font-mono t
 </div>

 {/* СПИСОК ОЧЕРЕДИ */}
 <div className="flex-1 flex items-end gap
 {queue.length === 0 ? {
 <div className="flex-1 flex flex-col
 </
 <p className="text-xs font-bold fon
 <p className="text-[10px]">Все сыты
 </div>
 } : {
 queue.map((customer, idx) => {
 const isHead = idx === 0;
 const isTail = idx === queue.length
 return (
 <div
 key={customer.id}
 className={`
 flex flex-col items-center ga
 ${customer.animState === 'in'
 ${customer.animState === 'eat
 ${customer.animState === 'out
 `}
 >
 {/* Облачко мыслей */}
 <div className={`
 absolute -top-16 left-1/2 -tr
 8 text-center text-slate-200 leading-tight
 ${isHead ? 'opacity-100 borde
 `}>
 <div className="font-bold tex
 {customer.bubbleText}
 <div className="absolute -bot
 it rotate-45"></div>
 </div>

 {/* Персонаж */}

```

```

 order-emerald-800/80 text-[9px] font-mono">
 <Sparkles className="w-3.5 h-3.5 animate-pu
 СЫТЫ
 </div>

 <div className="flex-1 flex flex-col justify-
 {servedHistory.length === 0 ? (
 <div className="flex-1 flex flex-col item

 <p className="text-xs font-bold font-mo
 <p className="text-[10px] mt-1">Здесь 6
 </div>
) : (
 <div className="flex flex-col gap-1.5 w-f
 {servedHistory.map((item, idx) => (
 <div
 key={idx}
 className="w-full py-1.5 px-3 round
 0 flex items-center gap-2 text-[11px] text-
 >
 {item}
 </div>
))}
 </div>
)}
 </div>

 {/* Пол */}
 <div className="w-full h-3 bg-gradient-to-r f
 </div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
 <div className="text-[10px] font-mono text-slate
 {log.map((entry, idx) => (
 <div
 key={idx}

```

```

const [simulationStep, setSimulationStep] = useState<number>(0);
const [bfsQueue, setBfsQueue] = useState<number[]>([]);
const [currentNode, setCurrentNode] = useState<number>(0);
const [speechText, setSpeechText] = useState<string>('Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и буль!');
const [isTalking, setIsTalking] = useState<boolean>(false);
const [isBlinking, setIsBlinking] = useState<boolean>(false);

// Регулярное моргание рыбы
useEffect(() => {
 const blinkInterval = setInterval(() => {
 setIsBlinking(true);
 setTimeout(() => setIsBlinking(false), 200);
 }, 4000);
 return () => clearInterval(blinkInterval);
}, []);

// Эффект разговора при смене текста
useEffect(() => {
 setIsTalking(true);
 const timer = setTimeout(() => setIsTalking(false), 1000);
 return () => clearTimeout(timer);
}, [speechText]);

// Построение начального бора (Trie) без fail-ссылок
const buildInitialTrie = (wordList: string[]) => {
 const newNodes: { [key: number]: TrieNode } = {
 0: { id: 0, char: 'ROOT', parent: 0, children: {} }
 };
 let count = 0;

 wordList.forEach((word) => {
 let curr = 0;
 for (let i = 0; i < word.length; i++) {
 const c = word[i].toUpperCase();
 if (newNodes[curr].children[c] === undefined) {
 newNodes[curr].children[c] = {
 id: count,
 char: c,
 parent: curr,
 children: {}
 };
 count++;
 }
 curr = newNodes[curr].children[c].id;
 }
 });
 return newNodes;
};

```

```

 if (leftX === -1) leftX = newNodes[v].x!;
 rightX = newNodes[v].x!;
 });
 newNodes[u].x = leftX === -1 ? 40 + currentX * 2 : rightX;
 if (leftX === -1) currentX++;
 }
};

layoutDFS(0, 0);

setNodes(newNodes);
setSimulationStep(0);
setBfsQueue([]);
setCurrentNode(null);
setSpeechText(
 `Отлично! Я построил базовое префиксное дерево (БПД)
 S, чтобы расставить fail-ссылки!`
);
};

useEffect(() => {
 buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {
 e.preventDefault();
 const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
 if (!cleanWord) return;
 if (words.includes(cleanWord)) {
 setSpeechText(`Слово «${cleanWord}» уже есть в слове S`);
 return;
 }
 const updated = [...words, cleanWord];
 setWords(updated);
 setNewWord('');
 buildInitialTrie(updated);
};

```

```

 return;
}

const r = bfsQueue[0];
const newQueue = bfsQueue.slice(1);
const updatedNodes = { ...nodes };
const rNode = updatedNodes[r];
const childrenIds = Object.values(rNode.children);

let logs = `Рассматриваем вершину #${r} (${rNode.char}`;

for (const [char, u] of Object.entries(rNode.children)) {
 newQueue.push(u);
 let v = rNode.fail;
 while (v !== 0 && updatedNodes[v].children[char] !== undefined) {
 v = updatedNodes[v].fail;
 }
 const failTarget = updatedNodes[v].children[char];
 updatedNodes[u].fail = failTarget;

 // Рассчитываем терминальную ссылку (term_link)
 if (updatedNodes[failTarget].is_terminal) {
 updatedNodes[u].term_link = failTarget;
 } else {
 updatedNodes[u].term_link = updatedNodes[failTarget].term_link;
 }

 logs += `Для #${u} ('${char}') fail = #${failTarget}`;
}

if (childrenIds.length === 0) {
 logs += 'У этой вершины нет детей, просто переходим к следующему';
}

setNodes(updatedNodes);
setBfsQueue(newQueue);
setCurrentNode(r);
setSpeechText(logs);

```



```

 className={`w-36 h-36 transition-transform
 isTalking ? 'scale-105 drop-shadow-[0_0_10px]_black'
 : ''
 }` }
 >
 { /* Тело лосося */ }
 <ellipse cx="100" cy="100" rx="70" ry="45" fill="#e67e22" />
 <path d="M 40 85 Q 100 55 160 85 Q 100 115 40 85" fill="#e67e22" />

 { /* Хвост */ }
 <path
 d="M 35 100 L 5 70 L 15 100 L 5 130 Z"
 fill="#be123c"
 className="origin-[35px_100px] animate-pulse"
 />

 { /* Плавники */ }
 <path d="M 90 55 L 70 30 L 110 55 Z" fill="#000000" />
 <path d="M 90 145 L 70 170 L 110 145 Z" fill="#000000" />
 <path
 d="M 120 115 L 100 135 L 130 130 Z"
 fill="#f87171"
 className="origin-[120px_115px] animate-pulse"
 />

 { /* Глаз (с морганием) */ }
 <circle cx="145" cy="85" r="12" fill="#fff" />
 { isBlinking ? (
 <line x1="133" y1="85" x2="157" y2="85" stroke="black" />
) : (
 <circle cx="148" cy="85" r="6" fill="#fff" />
 <circle cx="151" cy="82" r="2" fill="#fff" />
) }
 </>
 </>

 { /* Рот (анимация разговора) */ }
 { isTalking ? (
 <path d="M 165 105 Q 155 120 165 125 Q 175 105 165 100" fill="#fff" />
) : (
 <path d="M 165 105 Q 155 120 165 125 Q 175 105 165 100" fill="#000" />
) }

```

```

 оен!'}
 </div>

 <div className="flex gap-2">
 {simulationStep === 0 && (
 <button
 onClick={startBFS}
 className="bg-emerald-600 hover:bg-emerald-500 text-white px-4 py-2 shadow-lg shadow-emerald-900/40 transition-colors"
 >
 <Play className="w-4 h-4" /> Запустить
 </button>
)}
 {simulationStep === 1 && (
 <button
 onClick={stepBFS}
 className="bg-blue-600 hover:bg-blue-500 text-white px-4 py-2 shadow-lg shadow-blue-900/40 transition-colors"
 >
 Шаг BFS

 {currentStep}

 </button>
)}
 {simulationStep === 2 && (
 <button
 onClick={() => buildInitialTrie(words)}
 className="bg-indigo-600 hover:bg-indigo-500 text-white px-4 py-2 shadow-lg shadow-indigo-900/40 transition-transform active:scale-95"
 >
 <RotateCcw className="w-4 h-4" /> Перезагрузить
 </button>
)}
 </div>
 </div>
</div>
</div>

{/* SVG Canvas for Automaton Tree */}

```

```

 y1={parent.y + 16}
 x2={node.x}
 y2={node.y - 16}
 stroke={isCurrent ? '#60a5fa' : '#ccc'}
 strokeWidth="2"
 markerEnd="url(#arrowhead)"
 />
 <text
 x={({parent.x + node.x) / 2 - 10}
 y={({parent.y + node.y) / 2}
 fill="#94a3b8"
 fontSize="12"
 fontWeight="bold"
 >
 {node.char}
 </text>
</g>
);
 })}

 {/* Draw Fail Links */}
 {simulationStep > 0 && Object.values(nodes).length > 0} {
 if (node.id === 0 || node.fail === 0) return;
 // n draw all! But maybe wait, we can just draw the first one
 // Wait, all nodes get fail link. Let's draw them all!
 if (step >= 1 and computed it) {
 // To avoid too many lines, let's draw only the first one
 const target = nodes[node.fail];
 if (!target || !node.x || !node.y || !target.x || !target.y) return;

 // Simple bezier curve for fail link
 const isCurrent = currentNode === node.id;

 return (
 <path
 key={`fail-${node.id}`}
 d={`M ${node.x} ${node.y - 10} Q ${node.x + 10} ${node.y - 16} `}

```

```

 strokeWidth="2"
 />
 <text
 x={node.x}
 y={node.y + 4}
 fill="white"
 fontSize={isRoot ? "10" : "12"}
 fontWeight="bold"
 textAnchor="middle"
 >
 {isRoot ? 'R' : node.char}
 </text>
 {!isRoot && {
 <text
 x={node.x + 12}
 y={node.y - 12}
 fill="#94a3b8"
 fontSize="9"
 >
 {node.id}
 </text>
 }}
 </g>
);
 }}}
</svg>
);
}}({})
</div>

```

```

{/* Управление словарем */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-4" >
 <div className="bg-slate-900/50 p-5 rounded-xl" >
 <div>
 <h5 className="text-white font-bold mb-3" >
 Словарь (Паттерны)
 </h5>
 <div className="flex flex-wrap gap-2 mb-4">

```

```

 { /* Таблица Вершин и суффиксных ссылок */ }
 <div className="lg:col-span-2 bg-slate-900/50 p-4">
 <h5 className="text-white font-bold mb-3 flex items-center">

 Таблица переходов и fail-ссылок

 Всего вершин: {Object.keys(nodes).length}

 </h5>

 <div className="max-h-[220px] overflow-y-auto border border-slate-700">
 <table className="w-full text-left border-collapse">
 <thead>
 <tr className="border-b border-slate-700">
 <th className="py-2 px-3">ID Вершины</th>
 <th className="py-2 px-3">Символ</th>
 <th className="py-2 px-3">Родитель</th>
 <th className="py-2 px-3">fail-ссылка</th>
 <th className="py-2 px-3">term_link</th>
 <th className="py-2 px-3">Дети</th>
 <th className="py-2 px-3">Слова (dict)</th>
 </tr>
 </thead>
 <tbody className="divide-y divide-slate-800">
 {Object.values(nodes).map((node) => {
 const isCurrent = currentNode === node.id
 const isInQueue = bfsQueue.includes(node.id)

 return (
 <tr>
 <td>{node.id}</td>
 <td>{node.symbol}</td>
 <td>{node.parent}</td>
 <td>{node.fail}</td>
 <td>{node.term_link}</td>
 <td>{node.children}</td>
 <td>{node.words}</td>
 </tr>
)
 })}
 </tbody>
 </table>
 </div>
 </div>
)
}

```

```

) : {
 <span className="bg-rose-950
0 transition-colors cursor-help" title={`Ск
 #{node.term_link}

 }}
 </td>
 <td className="py-2.5 px-3 text-s
 {Object.keys(node.children).len
 ? '0'
 : Object.entries(node.children
 .map([c, id]) => `${c}→#
 .join(', '))}
 </td>
 <td className="py-2.5 px-3">
 {node.words.length === 0 ? {
 <span className="text-slate-6
 } : {
 <span className="bg-emerald-9
 {node.words.join(', ')}

 }}
 </td>
 </tr>
</div>
);
 }}}
</tbody>
</table>
</div>
</div>
</div>
</div>
);
};

```

```

const steps: Step[] = [];
let id = 0;
const tree: Record<number, number> = {};

const CODE = [
 "build(v, l, r) {", // 1
 " if (l === r) {", // 2
 " tree[v] = A[l]; return", // 3
 " }", // 4
 " mid = (l + r) >> 1", // 5
 " build(2v, l, mid)", // 6
 " build(2v+1, mid+1, r)", // 7
 " tree[v] = tree[2v]+tree[2v+1]", // 8
 "}", // 9
];
void CODE;

function go(v: number, l: number, r: number) {
 if (l === r) {
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
 , highlightNodes: [v], arrayHighlight: [l, r] });
 tree[v] = arr[l];
 steps.push({ id: id++, desc: `tree[${v}] = ${arr[l]}`,
 , highlightNodes: [v], arrayHighlight: [l, r] });
 return;
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
 { ...tree }, highlightNodes: [v], arrayHighlight: [l, r] });
 go(2 * v, l, m);
 steps.push({ id: id++, desc: `Вернулись в build(${v}, ${l}, ${r}`,
 , highlightNodes: [v], arrayHighlight: [l, r] });
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 steps.push({ id: id++, desc: `Объединяем: tree[${v}] = tree[2 * v] + tree[2 * v + 1]`,
 , highlightNodes: [v], mergeChildren: { ...tree }, arrayHighlight: [l, r] });
}
go(1, 0, n - 1);

```

```
// 3. Bottom-Up Query steps (iterative on implicit tree)
function genQueryBottomUpSteps(arr: number[], qL: number, qR: number) {
 const n = arr.length;
 // Build iterative tree: leaves at [n..2n-1]
 const tree: Record<number, number> = {};
 for (let i = 0; i < n; i++) tree[n + i] = arr[i];
 for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]) / 2;

 const steps: Step[] = [];
 let id = 0;
 let l = qL + n;
 let r = qR + n + 1; // half-open [l, r)
 let res = 0;

 steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL + n}, r = qR + n + 1 = ${qR + n + 1}`, codeLine: 3, treeState: tree, highlightNodes: [l, r] });

 while (l < r) {
 const highlights: number[] = [];
 if (l & 1) {
 res += tree[l] ?? 0;
 steps.push({ id: id++, desc: `l=${l} нечётный → l += 1. Сумма = ${res}. l++`, codeLine: 5, treeState: tree, highlightNodes: [l], arrayHighlightNodes: [l] }, { id: id++, desc: `l=${l} нечётный → l += 1. Сумма = ${res}. l++`, codeLine: 5, treeState: tree, highlightNodes: [l], arrayHighlightNodes: [l] });
 highlights.push(l);
 l++;
 }
 if (r & 1) {
 r--;
 res += tree[r] ?? 0;
 steps.push({ id: id++, desc: `r=${r + 1} нечётный → r -= 1. Сумма = ${res}. r--`, codeLine: 5, treeState: tree, highlightNodes: [r], arrayHighlightNodes: [r] }, { id: id++, desc: `r=${r + 1} нечётный → r -= 1. Сумма = ${res}. r--`, codeLine: 5, treeState: tree, highlightNodes: [r], arrayHighlightNodes: [r] });
 highlights.push(r);
 }
 l >>= 1;
 r >>= 1;
 if (l < r) {
 steps.push({ id: id++, desc: `Поднимаемся: l = (l + r) / 2, r = (l + r) / 2 + 1`, codeLine: 6, treeState: tree, highlightNodes: [l, r], arrayHighlightNodes: [l, r] });
 }
 }
}
```



```

 " return query(left) + query(right);",
 "}"
];
const QUERY_BU_CODE = [
 "queryBU(ql, qr) {",
 " res = 0;",
 " l = ql + n; r = qr + n + 1;",
 " // -----",
 " while (l < r) {",
 " if (l & 1) res += tree[l++];",
 " // -----",
 " if (r & 1) res += tree[--r];",
 " l >>= 1; r >>= 1;",
 " }",
 " return res; }"
];

// ===== PLAYER COMPONENT =====
function Player({ steps, color }: { steps: Step[]; color: string }) {
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [speed, setSpeed] = useState(700);

 useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);

 useEffect(() => {
 if (!playing || idx >= steps.length - 1) { if (idx < steps.length - 1) {
 const t = setTimeout(() => setIdx(i => i + 1), speed);
 return () => clearTimeout(t);
 }
 }, [playing, idx, steps, speed]);

 const step = steps[idx] || steps[0];
 if (!step) return null;

 const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-indigo-500 border-indigo-500 bg-indigo-600', childRing: 'ring-indigo-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-emerald-500 border-emerald-500 bg-emerald-600', childRing: 'ring-emerald-900' }, amber: { btn: 'bg-amber-600 hover:bg-amber-500 border-amber-500 bg-amber-600', childRing: 'ring-amber-900' } };

```

```

 {nd.left && renderNode(nd.left)}
 {nd.right && renderNode(nd.right)}
 </div>
 </div>
)}
</div>
);
}, [step, clr]);

return (
 <div className={`rounded-2xl border overflow-hidden
 ? 'border-emerald-500/30' : 'border-amber-500/30'}
 {/* Header */}
 <div className={`px-4 py-2.5 flex items-center justify-between
 -emerald-950/50' : 'bg-amber-950/50'} border-b border-white
 <h4 className="font-bold text-sm text-white flex-1"
 <span className="text-[10px] font-mono text-slate-500"
 </div>

 {/* Array strip */}
 <div className="px-3 py-2 bg-slate-950 border-b border-white"
 {arr.map((v, i) => {
 const inRange = step.arrayHighlight && i >= step.start
 return (
 <div key={i} className={`flex flex-col items-center justify-between
 go' ? 'bg-indigo-950 border-indigo-500' : 'bg-emerald-950 border-emerald-500'}
 -translate-y-0.5` : 'bg-slate-900 border-slate-500'}
 <span className="text-[7px] text-slate-500"
 <span className="text-[11px] font-bold font-mono text-white"
 </div>
);
 })}
 </div>

 {/* Tree */}
 <div className="flex-1 p-2 overflow-x-auto bg-slate-950"
 <div className="scale-[0.85] origin-top min-w-max"
 </div>

```

```

 white transition-all ${playing ? 'bg-rose-500' : 'bg-slate-400'}
 {playing ? <Pause className="w-3 h-3" /> : <Play className="w-3 h-3" />}
 </button>
 <button onClick={() => { setPlaying(false); setSpeed(1200); }}
 .5 text-slate-400 hover:text-white hover:bg-rose-500
 >
 Pause
 </button>
 </div>
 <select value={speed} onChange={e => setSpeed(Number(e.target.value))}
 text-[9px] font-bold rounded px-1.5 py-1 outline-1
 >
 <option value={1200}>Медл.</option>
 <option value={700}>Норма</option>
 <option value={350}>Быстро</option>
 <option value={120}>Турбо</option>
 </select>
 </div>

 { /* Progress bar */ }
 <div className="h-1 bg-slate-950"><div className="bg-rose-500"
 style={{ width: ((current + 1) / total) * 100 : 0}%` }} /></div>
</div>
);
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
 const [arr, setArr] = useState<number[]>([5, 8, 3, 12]);
 const [customInput, setCustomInput] = useState('5, 8, 3, 12');
 const [qL, setQL] = useState(1);
 const [qR, setQR] = useState(4);
 const [view, setView] = useState<'build' | 'queries'>('build');

 // Clamp query range when array changes
 useEffect(() => {
 setQL(prev => Math.min(prev, arr.length - 1));
 setQR(prev => Math.min(prev, arr.length - 1));
 }, [arr]);
}

```

```

 onChange={e => setCustomInput(e.target.value)}
 className="flex-1 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 placeholder="5, 8, 3, 12, 7, 2"
 />
 <button type="submit" className="bg-indigo-600 text-white px-4 py-2 rounded-md transition-colors">Применить</button>
 </div>
 <div className="flex flex-wrap gap-1.5 pt-1">
 {arr.map((v, i) => (

 A[{i}]

))}
 Сумма
 </div>
 </form>

 {/** Query range */}
 <div className="bg-slate-950 p-4 rounded-xl border border-indigo-500">
 <label className="text-[10px] font-bold uppercase">Запрос</label>
 <div className="flex items-center gap-2">
 <div className="flex items-center gap-1">
 От
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qL}
 onChange={e => setQL(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 />
 </div>
 <div className="flex items-center gap-1">
 До
 <input
 type="number"

```

```

 key={`build-${arr.join('-')}`}
 title="Построение дерева (рекурсия)"
 icon=" "
 steps={buildSteps}
 code={BUILD_CODE}
 color="indigo"
 arr={arr}
 />
)}

{view === 'queries' && (
 <div className="grid grid-cols-1 xl:grid-cols-2"
 <SimPanel
 key={`qtd-${arr.join('-')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Сверху`
 icon="↑ "
 steps={queryTDSteps}
 code={QUERY_TD_CODE}
 color="emerald"
 arr={arr}
 />
 <SimPanel
 key={`qbu-${arr.join('-')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Снизу`
 icon="↑ "
 steps={queryBUSteps}
 code={QUERY_BU_CODE}
 color="amber"
 arr={arr}
 />
 </div>
)}

{view === 'theory' && (
 <div className="space-y-5">
 {/* Why it splits this way */}
 <div className="bg-slate-950 p-5 rounded-xl b
 <h4 className="text-base font-bold text-whi

```



```

 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск по темам.."
 className="w-full bg-slate-800/70 border border-indigo-500 focus:outline-none focus:border-indigo-500"
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -translate-y-1/2 text-indigo-500 hover:text-indigo-400"
 aria-label="Очистить поиск"
 >
 <X className="w-3.5 h-3.5" />
 </button>
)}
 </div>
 </div>

 <div className="flex-1 overflow-y-auto px-3 pb-4" >
 {groups.length === 0 && <p className="text-sm text-slate-500">Нет результатов</p>}

 {groups.map(([category, items]) => (
 <div key={category}>
 <div className="text-[10px] font-bold uppercase text-slate-500">{category}</div>
 <div className="space-y-1">
 {items.map((chapter) => {
 const isSelected = chapter.id === selectedChapterId
 const hasViz = Boolean(VIZ_REGISTRY[chapter.id])
 return (
 <button
 key={chapter.id}
 onClick={() => {
 setSelectedChapterId(chapter.id);
 onNavigate?.();
 window.scrollTo({ top: 0, behavior: 'smooth' });
 }}
 className={`w-full text-left px-3 py-2 ${
 isSelected ? 'bg-indigo-500 text-white' : 'text-slate-800'
 }`}
 >{chapter.title}</button>
)
 })}
 </div>
 </div>
))}
 </div>

```

```
};
```

---

ФАЙЛ 64/88: src/components/SideCompilerDrawer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { createPortal } from "react-dom";
import { CompilerPanelContent } from "../CompilerPanelContent";
```

```
interface Props {
 chapterId: string;
 open: boolean;
 onClose: () => void;
 onOpenFull: () => void;
}
```

```
/**
```

```
 * Мобильный drawer – на десктопе используется inline п...
 * здесь – только оверлей для телефонов (lg:hidden в App...
 * Контент переиспользует CompilerPanelContent – минима...
 */
```

```
export function SideCompilerDrawer({ chapterId, open, onClose }) {
 if (!open) return null;
 return createPortal(
 <div className="fixed inset-0 z-[90] flex justify-end">
 <div className="absolute inset-0 bg-black/40 backdrop-blur-sm">
 <div className="relative w-[92vw] max-w-[400px] h-[400px]">
 <CompilerPanelContent chapterId={chapterId} onClose={onClose} />
 </div>
 </div>,
 document.body
);
}
```

---



```

 { id: '1', name: 'Студент Вася', icon: ' ' },
 { id: '2', name: 'Голодный кодер', icon: ' ' },
 { id: '3', name: 'Кот Борис', icon: ' ' },
]);
const [dequeueMessage, setDequeueMessage] = useState<

// Heap state
const [heap, setHeap] = useState<Hypothesis[]>([
 { id: '1', text: 'Кандидат: "Привет, я искусственный"', icon: ' ' },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный"', icon: ' ' },
 { id: '3', text: 'Кандидат: "Привет, я испанский ин"', icon: ' ' },
]);
const [newCustomText, setNewCustomText] = useState('');
const [newCustomProb, setNewCustomProb] = useState(0.5);
const [heapMessage, setHeapMessage] = useState<string>('');

// Stack handlers
const addPlate = () => {
 const presets = [
 { name: 'Сковорода от омлета', icon: ' ' },
 { name: 'Кастрюля от супа', icon: ' ' },
 { name: 'Чашка от кофе', icon: '☑' },
 { name: 'Миска с остатками салата', icon: ' ' },
 { name: 'Тарелка от тако', icon: ' ' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPlate = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setStack(prev => [...prev, newPlate]);
 setPopMessage(`Положили поверх стопки: ${newPlate.name}`);
};

const popPlate = () => {
 if (stack.length === 0) {
 setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
 }
};

```

```

 }
 const firstPerson = queue[0];
 setQueue(prev => prev.slice(1));
 setDequeueMessage(` Выдан суп первому в очереди (F
 });

const resetQueue = () => {
 setQueue([
 { id: '1', name: 'Студент Вася', icon: ' ' },
 { id: '2', name: 'Голодный кодер', icon: ' ' },
 { id: '3', name: 'Кот Борис', icon: ' ' },
]);
 setDequeueMessage(" Очередь восстановлена.");
};

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
 e.preventDefault();
 if (!newCustomText.trim()) return;
 const item: Hypothesis = {
 id: Date.now().toString(),
 text: `Кандидат: "${newCustomText}"`,
 probability: Number(newCustomProb)
 };
 setHeap(prev => [...prev, item]);
 setNewCustomText('');
 setHeapMessage(` Добавлена гипотеза с вероятностью
};

const popHeap = () => {
 if (heap.length === 0) {
 setHeapMessage("⚠ Куча пуста! Нет кандидатов для
 return;
 }
 // Find highest probability
 let maxIdx = 0;
 for (let i = 1; i < heap.length; i++) {
 if (heap[i].probability > heap[maxIdx].probability

```

```

 ? 'bg-blue-600/20 border-blue-500 text-blue-50'
 : 'bg-slate-800/60 border-slate-700/60 text-slate-200'
 }`}
 >
 <div className="flex items-center gap-3">
 <Layers className={`w-5 h-5 ${activeTab === 'lifo' ? 'bg-blue-600/20 border-blue-500 text-blue-50' : 'bg-slate-800/60 border-slate-700/60 text-slate-200'} `}>
 <div className="text-left">
 <div className="font-bold text-sm text-white">LIFO
 <div className="text-xs opacity-80">LIFO
 </div>
 </div>

 DFS

 </button>

 <button
 onClick={() => setActiveTab('queue')}
 className={`flex items-center justify-between w-5 h-5 ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500 text-indigo-50' : 'bg-slate-800/60 border-slate-700/60 text-slate-200'} `}>
 >
 <div className="flex items-center gap-3">
 <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500 text-indigo-50' : 'bg-slate-800/60 border-slate-700/60 text-slate-200'} `}>
 <div className="text-left">
 <div className="font-bold text-sm text-white">FIFO
 <div className="text-xs opacity-80">FIFO
 </div>
 </div>

 BFS

 </button>

 <button
 onClick={() => setActiveTab('heap')}

```

```

 >
 <Plus className="w-4 h-4" /> Поставить
 </button>
 <button
 onClick={popPlate}
 className="flex-1 md:flex-none flex items-center justify-center
border-blue-500/40 px-4 py-2.5 rounded-xl"
 >
 Помыть верхнюю
 </button>
 <button
 onClick={resetStack}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-slate-700 text-white
tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
</div>
</div>

{popMessage && {
 <div className="bg-blue-950/60 border border-blue-900/60 p-4
imate-fade-in">
 <span className="inline-block w-2 h-2 rounded-full bg-white
{popMessage}
 </div>
}}

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2xl" >
 >
 {stack.length === 0 ? (
 <div className="text-center py-12 text-slate-400">
 <div className="text-5xl mb-3">0</div>
 <p className="text-base font-bold">Сторожка пуста
 <p className="text-xs mt-1">Добавьте элемент
 </div>
) : (

```

```

 ПОЛОЖИЛ ПОСЛЕДНЕЙ → ПОМЫЛ ПЕРВОЙ (LIFO) •
 </div>
 </div>
 </div>
})

{/* QUEUE TAB */}
{activeTab === 'queue' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Симуляция очереди за супом
 <span className="text-xs font-mono bg-i
 /span>
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Кто первым пришел, тот первым получил с
 </p>
 </div>
 <div className="flex flex-wrap items-center
 <button
 onClick={addPerson}
 className="flex-1 md:flex-none flex ite
 -2.5 rounded-xl text-sm font-bold shadow-lg
 >
 <Plus className="w-4 h-4" /> Встать в о
 </button>
 <button
 onClick={servePerson}
 className="flex-1 md:flex-none flex ite
 er border-indigo-500/40 px-4 py-2.5 rounded
 >
 Налить суп первому
 </button>
 <button
 onClick={resetQueue}

```

```

 {queue.map((person, index) => {
 const isFirst = index === 0;
 return (
 <div
 key={person.id}
 className={`flex flex-col items-center
 ${isFirst
 ? 'bg-gradient-to-b from-indigo-500 to-white
 relative'
 : 'bg-slate-900/90 border-slate-800'}
 `}
 >
 {isFirst && (
 <span className="absolute -top-10 left-0 right-0
 se tracking-wider shadow">
 Первый (Head)

)}
 {
 <span className={`font-bold text-2xl
 {person.name}
 `}>

 <span className={`text-[10px] font-mono
 isFirst ? 'bg-indigo-500/30 text-white' : ''
 `}>
 {isFirst ? 'Получает сун' : `В ${person.age} лет`}

 </div>
);
)}
)}

 <div className="flex flex-col items-center justify-center
 -600 min-w-[120px] h-[120px]">
 <Plus className="w-6 h-6 mb-1" />

 </div>
 </div>
)}

```

```
</div>
</div>

{/* Add Form */}
<form onSubmit={addHypothesis} className="bg-slate-900 p-6 gap-4 items-end">
 <div className="md:col-span-6">
 <label className="block text-xs font-bold">Напишите гипотезу:</label>
 <input
 type="text"
 value={newCustomText}
 onChange={e => setNewCustomText(e.target.value)}
 placeholder='например: "Привет, я лучший!"'
 className="w-full bg-slate-900 border border-emerald-500 transition-colors"
 />
 </div>
 <div className="md:col-span-3">
 <label className="block text-xs font-bold">Вероятность:</label>
 <input
 type="range"
 min="0.01"
 max="0.99"
 step="0.01"
 value={newCustomProb}
 onChange={e => setNewCustomProb(Number(e.target.value))}
 className="w-full accent-emerald-500 cursor-pointer"
 />
 </div>
 <div className="md:col-span-3">
 <button
 type="submit"
 disabled={!newCustomText.trim()}
 className="w-full flex items-center justify-center gap-2 bg-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed"
 >
 Добавить
 </button>
 </div>
</form>
```

```

 ? 'bg-gradient-to-r from-emerald-500 to-slate-900/10 scale-[1.01]'
 : 'bg-slate-900/80 border-slate-900'
 }` }
 }
 }

```

```

 <div className="flex items-center justify-between gap-4">

 {isTop ? 'bg-emerald-500 text-white' : 'bg-slate-900 text-white'}

 <div>
 <div className={`font-bold text-white`} >
 {item.text}
 </div>
 {isTop && (

 Вершина Кучи (Root) • Будущее

)}
 </div>
 </div>

```

```

 <div className="flex items-center justify-between gap-4">
 <div className="hidden sm:block">
 <div
 className={`h-full rounded-full`}
 style={{ width: `${percent}%` }}
 ></div>
 </div>

 {isTop ? 'text-emerald-400 text-white' : 'text-white'}
 {percent}%

 </div>

```



```

*/
export const SimulatorHub: React.FC<Props> = ({ onOpen,
 const [query, setQuery] = useState("");

 const items = useMemo(() => {
 const titleById = new Map(chapters.map((c) => [c.id,
 const all = Object.entries(VIZ_REGISTRY).map(([id,
 id,
 entry,
 chapterTitle: titleById.get(id),
]));
 const q = query.trim().toLowerCase();
 if (!q) return all;
 return all.filter(
 (i) =>
 i.entry.title.toLowerCase().includes(q) ||
 (i.entry.hint ?? "").toLowerCase().includes(q)
 (i.chapterTitle ?? "").toLowerCase().includes(q)
);
 }, [query]);

 return (
 <div>
 <div className="mb-6">
 <h2 className="text-xl sm:text-2xl font-extrabo
 <p className="text-sm text-slate-400 leading-re
 Все интерактивные демонстрации в одном месте.
 здесь их можно открыть напрямую.
 </p>
 </div>

 <div className="relative mb-6 max-w-md">
 <Search className="w-4 h-4 text-slate-500 absol
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск тренажёра..."
 className="w-full bg-slate-900 border border-

```

```
 0 hover:text-white transition-colors min-w-[100px]
 >
 <BookOpen className="w-3.5 h-3.5 shrink-0" />

 </button>
)}
 </div>
</div>
)}}
</div>
</div>
);
};
```

---

ФАЙЛ 67/88: src/components/SplayRotationSandbox.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useCallback, useMemo, useState } from "react";
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, Undo } from "lucide-react";

/**
 * Песочница поворотов.
 *
 * Здесь ничего не подсказывается: дано настоящее дерево,
 * поднять отмеченный узел в корень. Клик по узлу = оди
 * Порядок поворотов выбираешь сам; правильный ли он по
 * только после того, как узел окажется наверху (или по
 */

export interface TNode {
 key: number;
 left: TNode | null;
 right: TNode | null;
}
```

```

 target: 5,
 build: () => {
 const n5 = leaf(5);
 const n6: TNode = { key: 6, left: n5, right: leaf(7) };
 const n4: TNode = { key: 4, left: leaf(3), right: n6 };
 return { key: 8, left: n4, right: leaf(9) };
 },
 },
];

/* ----- операции над деревом -----

export const findPath = (root: TNode | null, key: number): TNode[] => {
 const path: TNode[] = [];
 let cur = root;
 while (cur) {
 path.push(cur);
 if (key === cur.key) return path;
 cur = key < cur.key ? cur.left : cur.right;
 }
 return [];
};

/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
 const path = findPath(root, key);
 if (path.length < 2) return root;

 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const gg = path.length >= 3 ? path[path.length - 3] : null;

 if (p.left === x) {
 p.left = x.right;
 x.right = p;
 } else {
 p.right = x.left;
 x.left = p;
 }

 if (gg) {
 gg.left = x;
 x.left = gg;
 }

 return root;
}

```

```

const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 nodes.push({ key: n.key, x: order++, y: depth, depth });
 if (n.left) edges.push([n.key, n.left.key]);
 if (n.right) edges.push([n.key, n.right.key]);
 walk(n.right, depth + 1);
};
walk(root, 0);

const cols = Math.max(1, order);
const rows = Math.max(1, Math.max(...nodes.map((n) => n.depth)));
const colW = 46;
const rowH = 62;
return {
 nodes: nodes.map((n) => ({ ...n, x: n.x * colW + colW / 2, y: n.y * rowH + rowH / 2 })),
 edges,
 width: cols * colW,
 height: rows * rowH + 20,
};
}

/* ----- КОМПОНЕНТ ----- */

export const SplayRotationSandbox: React.FC = () => {
 const [presetIdx, setPresetIdx] = useState(0);
 const preset = PRESETS[presetIdx];

 const [tree, setTree] = useState<TNode>(() => preset.root);
 const [history, setHistory] = useState<TNode[]>([]);
 const [moves, setMoves] = useState<{ node: number; count: number }>({ node: 0, count: 0 });
 const [showAnswer, setShowAnswer] = useState(false);

 const target = preset.target;
 const solved = tree.key === target;

 const reset = useCallback(
 (idx = presetIdx) => {
 setPresetIdx(idx);
 setTree(preset.root);
 setHistory([]);
 setMoves({ node: 0, count: 0 });
 setShowAnswer(false);
 },
 [presetIdx]
);

```

```

 <h4 className="text-sm sm:text-base font-bold">
 <p className="text-[12px] text-slate-400 lead">
 Клик по узлу = один поворот с его родителем

 поворотов выбираешь сам, разбор покажем в к
 </p>
 </div>
 <button
 onClick={() => reset((presetIdx + 1) % PRESETS.length)}
 className="flex items-center gap-1.5 px-3 py-1.5 text-sm font-bold transition-colors shrink-0"
 >
 <Dice className="w-3.5 h-3.5" /> Другое дерево
 </button>
 </div>

 <div className="flex gap-2 mb-3 overflow-x-auto">
 {PRESETS.map((p, i) => {
 <button
 key={p.name}
 onClick={() => reset(i)}
 className={`px-3 py-1.5 rounded-lg text-[11px] ${
 i === presetIdx ? "bg-indigo-600 text-white" : "bg-white text-slate-700"
 }`}
 >
 {p.name}
 </button>
 })}
 </div>

 <div className="bg-slate-900 rounded-xl border border-slate-800">
 <svg
 viewBox={`0 0 ${width} ${height}`}
 className="h-auto block mx-auto"
 style={{ minWidth: Math.min(width * 1.5, 420) }}
 role="img"
 >
 {edges.map(([a, b], i) => {

```

```

 strokeWidth={2}
 />
 <text textAnchor="middle" dominantBaseline="middle">
 {n.key}
 </text>
 </g>
);
}
</svg>
</div>

<div className="flex flex-wrap items-center gap-2">
 <button
 onClick={undo}
 disabled={history.length === 0}
 className="flex items-center gap-1.5 px-3 py-1.5 text-white disabled:opacity-40 text-xs font-bold">
 <Undo2 className="w-3.5 h-3.5" /> Отменить
 </button>
 <button
 onClick={() => reset()}
 className="flex items-center gap-1.5 px-3 py-1.5 text-white font-bold transition-colors">
 <RotateCcw className="w-3.5 h-3.5" /> Заново
 </button>
 <button
 onClick={() => setShowAnswer((s) => !s)}
 className={`flex items-center gap-1.5 px-3 py-1.5 text-white border
 ${showAnswer ? "bg-amber-600/20 border-amber-500 text-amber-500" : "bg-slate-800 border-slate-700 text-slate-200"}
 `}>
 {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : "Скрыть разбор"}
 </button>

```

```


)}
 {wrong === 0 && moves.length > minimal && (
 <div className="opacity-80">Порядок верный,
)}
 </div>
)}
</div>
);
};

```

ФАЙЛ 68/88: src/components/SplayRotationsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useEffect, useMemo, useState } from "react";
import { Gauge, Pause, Play, RotateCcw } from "lucide-react";

```

```

/**
 * Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag.
 *
 * Что сделано ради понятности:
 * • у узлов настоящие числа (20 / 40 / 60), а не абстракции;
 * сразу видно, что дерево остаётся деревом поиска;
 * • под деревом печатается обход слева направо: он ОДНОУРОВНЕВНЫЙ;
 * это и есть доказательство, что поворот ничего не ломает;
 * • кадр «прицел» ничего не двигает, только подсвечивает;
 * • на кадре «результат» остаются призраки – пунктирные линии в
 * местах и стрелки «откуда → куда»;
 * • по умолчанию анимация НЕ крутится сама: пользователь
 * идёт в своём темпе. Автопрокрутка включается кнопкой.
 */

```

```

type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;

```

```

const ZIG: Case = {
 key: "zig",
 label: "Zig",
 when: "родитель x — уже корень, деда нет",
 rotations: "1 поворот",
 keys: { x: 20, p: 40 },
 inorder: ["A", "20", "B", "40", "C"],
 ranges: "A < 20 · B между 20 и 40 · C > 40",
 frames: [
 {
 ...ZIG_BEFORE,
 kind: "start",
 title: "Было: 20 висит под корнем 40",
 text: "Нам нужен ключ 20, а он на глубине 1. Деду",
 ms: RESULT_MS,
 },
 {
 ...ZIG_BEFORE,
 kind: "aim",
 title: "Прицел: крутим ребро 40 — 20",
 text: "Ничего пока не двигается. Смотри на подсве",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [160, 50], A: [80, 130], p: [245, 130],
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
 kind: "result",
 title: "Стало: 20 — корень",
 text: "40 стало правым сыном двадцатки. Поддерев
 ева от 40 — это одно и то же место.",
 moved: ["x", "p", "B"],
 ms: RESULT_MS,
 },
],
};

```



```

 title: "Поворот 1: 40 поднялось над 60",
 text: "40 встало в корень, 60 опустилось к нему п
 убине 1, а не 2.",
 moved: ["p", "g", "C"],
 ms: RESULT_MS,
},
{
 ...ZZ_S1,
 kind: "aim",
 title: "Прицел 2: ребро 40 – 20",
 text: "А теперь обычный одиночный поворот – тот ж
 focus: ["p", "x"],
 ms: AIM_MS,
},
{
 pos: { x: [100, 45], A: [45, 118], p: [188, 118],
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
 kind: "result",
 title: "Стало: 20 в корне, бамбук стал кустом",
 text: "Сравни с первым кадром: А и В были на глуб
 дерево, а не только достаёт ключ.",
 moved: ["x", "p", "A", "B"],
 ms: RESULT_MS,
},
],
};

```

/\* ----- Zig-Zag -----

```

const ZG_S0 = {
 pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
 edges: [["g", "p"], ["g", "D"], ["p", "A"], ["p", "x"
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZG_S1 = {
 pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
 edges: [["g", "x"], ["g", "D"], ["x", "p"], ["x", "C"
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

 focus: ["g", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [180, 50], p: [90, 130], g: [275, 130],
 edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
 kind: "result",
 title: "Стало: 20 и 60 — два сына сорока",
 text: "Дерево стало идеально симметричным: все че
 очку». ",
 moved: ["x", "p", "g", "C"],
 ms: RESULT_MS,
 },
],
};

const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];

const COLOR: Record<string, { fill: string; stroke: str
 x: { fill: "#6366f1", stroke: "#a5b4fc" },
 p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
 g: { fill: "#f59e0b", stroke: "#fcd34d" },
 sub: { fill: "#1e293b", stroke: "#475569" },
};

const ROLE_RU: Partial<Record<NodeId, string>> = {
 x: "x — ищем его",
 p: "p — родитель",
 g: "g — дед",
};

const isSubtree = (id: NodeId) => id === "A" || id ===

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
 const { pos, edges, focus, moved } = frame;
 const dim = (id: NodeId) => (focus ? !focus.includes(
 const ghosts = frame.kind === "result" ? (moved ?? [])

```

```

 const pb = pos[b];
 if (!pa || !pb) return null;
 const hot = Boolean(focus && focus.includes(a));
 return (
 <line
 key={` ${a} - ${b} - ${i} `}
 x1={pa[0]}
 y1={pa[1]}
 x2={pb[0]}
 y2={pb[1]}
 stroke={hot ? "#fbbf24" : "#475569"}
 strokeWidth={hot ? 5 : 2}
 opacity={focus && !hot ? 0.25 : 1}
 style={{ transition: "all 1200ms cubic-bezier(0.16, 0.66, 0.0, 0.4)" }}
 />
);
 }
}

```

```

{Object.keys(pos) as NodeId[]}.map((id) => {
 const p = pos[id];
 if (!p) return null;
 const sub = isSubtree(id);
 const c = COLOR[sub ? "sub" : id];
 const r = sub ? 15 : 20;
 const hot = Boolean(focus && focus.includes(id));
 return (
 <g
 key={id}
 style={{
 transform: `translate(${p[0]}px, ${p[1]}px)`,
 transition: MOVE,
 opacity: dim(id) ? 0.28 : 1,
 }}
 >
 {hot && <circle r={r + 7} fill="none" stroke="red" />}
 <circle r={r} fill={c.fill} stroke={c.stroke} />
 <text
 textAnchor="middle"

```

```

 if (!playing) return;
 const t = setTimeout(() => setFrame((f) => (f + 1) % CASES.length), duration);
 return () => clearTimeout(t);
 }, [playing, frame, duration, total]);

return (
 <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
 <div className="flex gap-2 mb-4 overflow-x-auto no-scrollbar">
 {CASES.map((c, i) => (
 <button
 key={c.key}
 onClick={() => setCaseIdx(i)}
 className={`px-3 sm:px-4 py-2 rounded-lg text-white transition-colors duration-200
 ${i === caseIdx ? "bg-indigo-600 text-white" : "bg-slate-800 text-slate-300"}
 `}
 >
 {c.label}
 </button>
))}
 </div>

 <div className="mb-3 text-xs sm:text-[13px] text-slate-400">
 Корпус
 ({active.rotation}°)
 </div>

 {/* шаг + пояснение стоят НАД картинкой: сначала шаг, потом пояснение */}
 <div className="mb-3 rounded-xl border border-slate-800">
 <div className="flex items-center gap-2 mb-1 flex-wrap">
 <span
 className={`text-[10px] font-bold uppercase transition-colors duration-200
 ${current.kind === "aim"
 ? "bg-amber-500/20 text-amber-300"
 : current.kind === "start"
 ? "bg-slate-700 text-slate-300"
 : "bg-indigo-500/20 text-indigo-300"}
 `}
 >

```

```

 playing
 ? { animation: `demoProgress ${duration}ms`
 : { width: "100%", background: "#334155" }
 }
 />
 </div>

 <div className="flex flex-wrap items-center gap-2">
 <button
 onClick={() => {
 setPlaying(false);
 setFrame((f) => (f - 1 + total) % total);
 }}
 disabled={idx === 0}
 className="px-3 py-2 rounded-lg bg-slate-800 text-white
 transition-colors duration-200 ease-in-out
 dark:hover:text-slate-300 text-xs font-bold transition"
 >
 ← Назад
 </button>
 <button
 onClick={() => {
 setPlaying(false);
 setFrame((f) => (f + 1) % total);
 }}
 className="px-4 py-2 rounded-lg bg-indigo-600 text-white
 transition-colors duration-200 ease-in-out
 w-indigo-900/40"
 >
 {last ? "⏮ Сначала" : "Дальше →"}
 </button>
 <button
 onClick={() => setPlaying((p) => !p)}
 className="flex items-center gap-1.5 px-3 py-2 text-white
 text-xs font-bold transition-colors"
 >
 {playing ? <Pause className="w-3.5 h-3.5" /> : <Play />}
 {playing ? "Стоп" : "Авто"}
 </button>
 <button

```

---

ФАЙЛ 69/88: src/components/SplayTreeViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from "react";
import {
 RotateCcw,
 HelpCircle,
 BookOpen,
} from "lucide-react";
```

```
interface SplayNode {
 key: number;
 left: SplayNode | null;
 right: SplayNode | null;
 x?: number;
 y?: number;
}
```

```
// Simple BST insertion to build initial tree
const insertBST = (root: SplayNode | null, key: number) => {
 if (!root) return { key, left: null, right: null };
 if (key < root.key) {
 root.left = insertBST(root.left, key);
 } else if (key > root.key) {
 root.right = insertBST(root.right, key);
 }
 return root;
};
```

```
// Right Rotation (Zig / Right Rotate)
const rightRotate = (x: SplayNode): SplayNode => {
 const y = x.left;
 if (!y) return x;
 x.left = y.right;
```

```

const [inputValue, setInputValue] = useState<string>('');
const [log, setLog] = useState<string[]>([
 "Дерево инициализировано. Попробуйте кликнуть на узел",
]);
const [highlightedNode, setHighlightedNode] = useState<SplayNode | null>(null);

// Splay operation that tracks steps!
const splayStepByStep = (
 root: SplayNode | null,
 key: number,
): { newRoot: SplayNode | null; steps: string[] } => {
 const steps: string[] = [];
 if (!root) return { newRoot: null, steps: ["Дерево пустое"] };

 // We implement the classic top-down or bottom-up splay
 // For visualization logs, we do a bottom-up explanation
 let path: { node: SplayNode; dir: "left" | "right" } | null = null;
 let current: SplayNode | null = root;

 // Find the element or the element where search failed
 let lastNode: SplayNode = root;
 while (current) {
 lastNode = current;
 if (key === current.key) {
 path.push({ node: current, dir: null });
 break;
 } else if (key < current.key) {
 path.push({ node: current, dir: "left" });
 current = current.left;
 } else {
 path.push({ node: current, dir: "right" });
 current = current.right;
 }
 }

 const targetKey = current ? key : lastNode.key;
 const isFound = !!current;

```

```

 steps.push(
 `Финальный поворот Zig (Правое вращение корня)
);
 return rightRotate(node);
 } else {
 if (!node.right) return node;

 if (k < node.right.key) {
 // Zag-Zig (Right-Left)
 node.right.left = splayAction(node.right.left, target);
 if (node.right.left) {
 node.right = rightRotate(node.right);
 steps.push(`Компонент Zig-Zag: правый поворот
);
 } else if (k > node.right.key) {
 // Zag-Zag (Right-Right)
 node.right.right = splayAction(node.right.right, target);
 node = leftRotate(node);
 steps.push(
 `Вращение Zag-Zag (Левый поворот родителя д
);
 }

 if (!node.right) return node;
 steps.push(
 `Финальный поворот Zag (Левое вращение корня)
);
 return leftRotate(node);
 }
 };

 const finalRoot = splayAction(cloneTree(root), target);
 return { newRoot: finalRoot, steps };
 };

 const handleSplay = (key: number) => {
 setHighlightedNode(key);
 const { newRoot, steps } = splayStepByStep(tree, key);

```



```

// Assign coordinate positions using a simple recursion
const computeCoordinates = (
 node: SplayNode | null,
 x: number,
 y: number,
 levelWidth: number,
): void => {
 if (!node) return;
 node.x = x;
 node.y = y;
 if (node.left) {
 computeCoordinates(node.left, x - levelWidth, y + 1);
 }
 if (node.right) {
 computeCoordinates(node.right, x + levelWidth, y + 1);
 }
};

const drawTree = cloneTree(tree);
computeCoordinates(drawTree, 300, 40, 140);

// Render lines and nodes
const renderLines = (node: SplayNode | null): React.ReactNode[] => {
 if (!node) return [];
 let lines: React.ReactNode[] = [];
 if (node.left && node.left.x && node.left.y) {
 lines.push(
 <line
 key={`l-${node.key}-${node.left.key}`}
 x1={node.x}
 y1={node.y}
 x2={node.left.x}
 y2={node.left.y}
 stroke="#475569"
 strokeWidth="2"
 />,
);
 }
 lines = lines.concat(renderLines(node.left));
};

```

```

 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="12px"
 fontWeight="bold"
 className="select-none pointer-events-none"
 >
 {node.key}
 </text>
 </g>,
);
 circles = circles.concat(renderNodes(node.left));
 circles = circles.concat(renderNodes(node.right));
 return circles;
 };

 return (
 <div className="bg-slate-900 rounded-xl border border-
 {/* Header */}
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <BookOpen className="w-5 h-5 text-indigo-400"
 Интерактивное Splay-дерево
 </h3>
 <p className="text-xs text-slate-400">
 Кликните на вершины дерева, чтобы «вытащить»
 Splay.
 </p>
 </div>

 <div className="grid grid-cols-1 lg:grid-cols-12
 {/* Playfield */}
 <div className="lg:col-span-7 bg-[#0f172a] p-4
 <svg

```

```

 }
 }}
 className="bg-indigo-950 hover:bg-slate-900"
 >
 Вставить
 </button>
 </form>

 <button
 onClick={resetTree}
 className="flex items-center gap-1.5 px-3"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Сброс
 </button>
</div>
</div>

{/* Logs */}
<div className="lg:col-span-5 bg-slate-950 border-1 border-indigo-500 rounded trans
to overflow-y-auto custom-scrollbar">
 <h4 className="text-xs font-bold text-slate-400">
 Ход вращения (Splay-логи)
 </h4>
 <div className="flex-1 space-y-2 mb-4 overflow-y-auto">
 {log.map((step, idx) => (
 <div
 key={idx}
 className={`text-xs p-2.5 rounded trans
 idx === log.length - 1
 ? "border-l-2 border-indigo-500 bg-slate-900"
 : "text-slate-400 bg-slate-900/50"
 }`}
 >
 {step}
 </div>
))}
 </div>
</div>

```

```

 поднимет в корень{" "}
 <strong className="text-white">
 последний успешно просмотренный узел

 , на котором он споткнулся! Это оставля
 сверху.
 </p>
</div>
<div className="mt-3 text-[10px] text-indigo-500">
 Поиск настраивает локальный кэш под реали
</div>
</div>

<div className="bg-slate-900/60 p-4 rounded-l
 <div>
 <p className="font-bold text-slate-300 mb-2">
 2. Эффект качелей (Swing)
 </p>
 <p className="text-slate-400 leading-relaxed">
 Если агент запрашивает по очереди{" "}
 <code className="text-rose-400">[10]</code>
 <code className="text-rose-400">[60]</code>
 углы), дерево будет превращаться в палку
 сторону. Первая операция Splay(60) стои
 <strong className="text-white">0(n)
 глубину n. Вторая Splay(10) заставляет
 Постоянный свинг убивает производительн
 сложности <strong className="text-white">O(1)
 </p>
 </div>
 <div className="mt-3 text-[10px] text-rose-400">
 Кэшировать свингующие пары на концах — фа
 </div>
</div>

<div className="bg-slate-900/60 p-4 rounded-l
 <div>
 <p className="font-bold text-slate-300 mb-2">

```

```

 animState: 'in' | 'idle' | 'washing' | 'clean';
}

const PLATE_PRESETS = [
 { name: 'Тарелка из-под спагетти', emoji: ' ', color: 'from-p'},
 { name: 'Тарелка из-под пиццы', emoji: ' ', color: 'from-p'},
 { name: 'Блюдец от торта', emoji: ' ', color: 'from-p'},
 { name: 'Тарелка от тако', emoji: ' ', color: 'from-y'},
 { name: 'Тарелка из-под суши', emoji: ' ', color: 'from-f'},
 { name: 'Сковорода от глазуньи', emoji: ' ', color: 'from-f'}
];

let counter = 0;

export const StackViz: React.FC = () => {
 const [dirtyStack, setDirtyStack] = useState<Plate[]>([
 { id: 'p1', name: 'Тарелка из-под спагетти', emoji: ' ', color: 'from-p', state: 'idle' },
 { id: 'p2', name: 'Тарелка из-под пиццы', emoji: ' ', color: 'from-p', state: 'idle' },
 { id: 'p3', name: 'Блюдец от торта', emoji: ' ', color: 'from-p', state: 'idle' },
]));

 const [cleanRack, setCleanRack] = useState<Plate[]>([]);
 const [isWashing, setIsWashing] = useState(false);
 const [showSoap, setShowSoap] = useState(false);
 const [shakeEmpty, setShake] = useState(false);
 const [log, setLog] = useState<string[]>([]);

 const addLog = (msg: string) => setLog(prev => [...prev, msg]);

 // PUSH: Положить грязную тарелку сверху стопки
 const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {
 addLog('⚠ Стопка слишком высокая, тарелки могут р...');
 setShake(true);
 }
 }, [isWashing, dirtyStack, addLog, setShake]);

```

```

 setDirtyStack(prev => prev.map(p => p.id === topPlate.id) ? prev : [...prev, topPlate]);
 setShowSoap(true);

 setTimeout(() => {
 // Step 2: Wash complete. Plate becomes clean and is moved to the clean rack
 const cleanPlate: Plate = {
 ...topPlate,
 isDirty: false,
 animState: 'clean',
 };

 setCleanRack(prev => [cleanPlate, ...prev].slice(0, cleanRack.length));
 setDirtyStack(prev => prev.slice(0, prev.length - 1));
 setShowSoap(false);
 setIsWashing(false);
 addLog(` Готово! Чистая тарелка ${topPlate.emoji}`, 850);
 }, [dirtyStack, isWashing]);

 const reset = () => {
 counter = 0;
 setDirtyStack([
 { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', state: 'in' },
 { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', state: 'in' },
 { id: 'r3', name: 'Блюдец от торта', emoji: '🍰', state: 'in' },
]);
 setCleanRack([]);
 setIsWashing(false);
 setShowSoap(false);
 setLog([' Стол сброшен. Посуда снова грязная!']);
 };

 const topIndex = dirtyStack.length - 1;

 return (

```

```

 className="px-5 py-3.5 rounded-2xl bg-slate-800
 r-pointer border border-slate-700"
 >
 Сброс
 </button>
 </div>

 { /* ИНТЕРАКТИВНАЯ КУХНЯ */ }
 <div className="grid grid-cols-1 md:grid-cols-12" >

 { /* ЛЕВАЯ КОЛОНКА: СТОПКА ГРЯЗНЫХ ТАРЕЛОК */ }
 <div className="md:col-span-7 bg-slate-900 rounded-2xl
 [380px]" >
 <div className="absolute top-4 left-4 text-[10px] font-mono" >
 Грязная стопка (Стек вызовов / LIFO)
 </div>

 { /* Визуализация раковины */ }
 <div className="absolute top-4 right-4 flex items-center
 r-slate-800/80 text-[10px] font-mono" >

 РАКОВИНА
 </div>

 { /* Мыльные пузыри при мытье */ }
 { showSoap && (
 <div className="absolute inset-0 pointer-events-none" >
 <div className="absolute w-6 h-6 rounded-full
 " ></div>
 <div className="absolute w-4 h-4 rounded-full
 " ></div>
 <div className="absolute w-5 h-5 rounded-full
 " ></div>
 <div className="absolute text-4xl anim-crash" >
 </div>
 </div>
) }

 { dirtyStack.length === 0 ? (

```

```

 <span className="text-slate-800 text-[9px] font-mono" style={{
 isTop && (
 <span className="text-[9px] bg-black" style={{
 LIFO
 }} />
)
 }} />
 </div>
);
 }
}
</div>
})

{/* Столешница раковины */}
<div className="w-full h-4 bg-gradient-to-r from-slate-900 to-slate-800" />

{/* ПРАВАЯ КОЛОНКА: СУШИЛКА ДЛЯ ЧИСТЫХ ТАРЕЛОК */}
<div className="md:col-span-5 bg-slate-900 rounded-2xl p-4" style={{
 <div className="absolute top-4 left-4 text-[10px] font-mono" style={{
 Сушилка для чистой посуды
 }} />

 <div className="absolute top-4 right-4 flex items-center justify-center gap-2" style={{
 order-emerald-800/80 text-[9px] font-mono" >
 <Sparkles className="w-3.5 h-3.5 animate-pulse" />
 ЧИСТО
 }} />

 <div className="flex-1 flex flex-col justify-between p-4" style={{
 {cleanRack.length === 0 ? (
 <div className="flex-1 flex flex-col items-center justify-center gap-2" style={{

 <p className="text-xs font-bold font-mono" />
 <p className="text-[10px] mt-1">Здесь 6 тарелок
 }} />
) : (
 <div className="flex flex-col gap-2 w-full" style={{
 {cleanRack.map(plate => (

```



```
import { useState, useEffect, useRef, useMemo } from "react";
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";

function generateKmpSteps(pattern: string, text: string) {
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const pi = new Array(n).fill(0);
 const steps: any[] = [];

 steps.push({ i: 0, j: 0, pi: [...pi], desc: `Начало` });

 for (let i = 1; i < n; i++) {
 let j = pi[i - 1];

 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });

 while (j > 0 && s[i] !== s[j]) {
 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
 j = pi[j - 1];
 }

 if (s[i] === s[j]) {
 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
 j++;
 } else {
 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадений нет` });
 }

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записываем` });
 }
}
```

```

 [i]]}' разные. Стоп.` }));
 } else {
 steps.push({ i, l, r, z: [...z], desc: `Упёр` });
 }

 steps.push({ i, l, r, z: [...z], desc: `Фиксирыв` });

 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 steps.push({ i, l, r, z: [...z], desc: `0` });
 }

 if (z[i] === pattern.length) {
 steps.push({ i, l, r, z: [...z], found: true });
 }
}
return { s, steps, patternLength: pattern.length };
}

export function StringAlgorithmsViz({ defaultMode = "kmp" }) {
 const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
 const [pattern, setPattern] = useState("aba");
 const [text, setText] = useState("abacaba");

 // Ensure inputs are valid dynamically
 const safePattern = pattern || " ";
 const safeText = text || " ";

 const { s, steps, patternLength } = useMemo(() => {
 if (mode === "kmp") return generateKmpSteps(safePattern, safeText);
 else return generateZSteps(safePattern, safeText);
 }, [mode, safePattern, safeText]);

 const [autoPlay, setAutoPlay] = useState(false);
 const [stepIdx, setStepIdx] = useState(0);
 const timer = useRef<NodeJS.Timeout | null>(null);

```

## Z-ФУНКЦИЯ

```

 </button>
 </div>

 <div className="flex items-center gap-2" >
 <div className="bg-slate-800 p-1 flex items-center gap-2" >

 <input value={pattern} onChange={e} />

 </div>
 <div className="bg-slate-800 p-1 flex items-center gap-2" >

 <input value={text} onChange={e} />

 </div>
 </div>
</div>

<div className="bg-slate-950 rounded-xl border-[1px] flex items-center justify-start">
 <div className="flex gap-1.5 px-4 h-full">
 {s.split("").map((char, index) => {
 const isPattern = index < pattern.length;
 const isSeparator = index === pattern.length;

 let borderColor = "border-slate-900/40";
 let bgColor = "bg-slate-900";
 let textColor = isSeparator ? "text-white" : "text-slate-400";

 // KMP Logic
 if (mode === "kmp") {
 if (step.highlight?.include(char)) {
 borderColor = "border-amber-500";
 bgColor = step.match === char ? "bg-amber-500" : "bg-slate-900/40";
 }
 }
 })}
 </div>
</div>

```

```

z-20">
 { /* KMP fingers */
 {mode == "kmp" && step
 <div className="abs
 <span className
 </div>
 }}
 {mode == "kmp" && step
 <div className="abs
 <span className
 </div>
 }}
 {mode == "kmp" && step
 <div className="abs
 <span className
 </div>
 }}

20">
 { /* Z fingers */
 {mode == "z" && step.i
 <div className="abs
 <span className
 </div>
 }}
 {mode == "z" && step.z
 <div className="abs
 <span className
 </div>
 }}
 {mode == "z" && step.z
 <div className="abs
 <span className
 </div>
 }}

e z-20">
 { /* Found flash effect

```

```

 <div className="absolute top-0 right-0" style={{

 </div>
 <div className="text-[10px] text-slate-400">
{steps.length}</div>
 <div className="text-sm text-slate-400">
 {step.desc}
 </div>
 </div>
 </div>

 {/* Why this matters card */}
 <div className="bg-slate-800 p-4 rounded-xl">

 <p className="text-xs text-slate-400">
 Обрати внимание на хитрость с решёткой.
 Мы пишем Шаблон + # + Текст.
 Когда значени {mode === 'kmp' ? "проблема не
о было найдено!
 Пройди шаги вручную, посмотри, как работает
ля пропуска работы (в Z).
 </p>
 </div>

 {/* Code Sneak Peek */}
 <div className="bg-slate-900 border border-slate-800">
 <div className="bg-slate-800 px-4 py-2 border border-slate-400 uppercase">
 Код C++ ({mode === 'kmp' ? 'КМП' : 'ЗКМП'})
 </div>
 <pre className="p-4 text-xs font-mono text-slate-400">
{mode === 'kmp' ? `vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}` : `int l = 0, r = 0;`

```

```

 v: number;
}

const dagNodes: TNode[] = [
 { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
 { id: 1, x: 280, y: 70, label: "1", name: "Дискретка (Дискретная математика)" },
 { id: 2, x: 280, y: 230, label: "2", name: "Линейный алгебра (Линейная алгебра)" },
 { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы (Алгоритмы)" },
 { id: 4, x: 640, y: 150, label: "4", name: "Машинное обучение (Машинное обучение)" },
];

const dagEdges: TEdge[] = [
 { u: 0, v: 1 },
 { u: 0, v: 2 },
 { u: 1, v: 3 },
 { u: 2, v: 3 },
 { u: 3, v: 4 },
];

interface TStep {
 desc: string;
 visited: Set<number>;
 fullyProcessed: Set<number>;
 currentNode: number | null;
 topoList: number[];
 dfsStack: number[];
}

export const TopologicalSortViz: React.FC = () => {
 const [steps, setSteps] = useState<TStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {
 const listSteps: TStep[] = [];
 const visited = new Set<number>();
 const fullyProcessed = new Set<number>();
 const topoList: number[] = [];
 });
};

```

```

 });
 }
}

fullyProcessed.add(u);
dfsStack.pop();
topoList.push(u); // prepend behavior: we write in reverse
recordStep(
 `DFS: закончили обработку '${dagNodes[u].name}'`,
 u,
);
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 recordStep(
 `Запускаем новый обход DFS от нетронутой вершины ${i}`,
 null,
);
 dfs(i);
 }
}

recordStep(
 "Топологическая сортировка завершена! Полученный список вершин: " + topoList,
 null,
);
setSteps(listSteps);
setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
 desc: "Запуск...",
 visited: new Set(),
 fullyProcessed: new Set(),
 currentNode: null,
};
```

```

 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 className="flex items-center justify-center
ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12 >
 {/* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] rela
 <svg className="w-full h-full max-w-[650px]" >
 <defs>
 <marker
 id="dag-arrow"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#4

```



```

{ /* Nodes */
dagNodes.map({node} => {
 const isCurrent = step.currentNode === node.id
 const isVisited = step.visited.has(node.id)
 const isProcessed = step.fullyProcessed.has(node.id)

 let fill = "#1e293b";
 let stroke = "#334155";
 let textColor = "text-slate-400";
 void textColor;

 if (isCurrent) {
 fill = "#312e81";
 stroke = "#818cf8";
 textColor = "text-white font-bold";
 } else if (isProcessed) {
 fill = "#022c22";
 stroke = "#059669";
 textColor = "text-emerald-300";
 } else if (isVisited) {
 fill = "#3730a3";
 stroke = "#6366f1";
 textColor = "text-indigo-200";
 }

 return (
 <g key={node.id}>
 <rect
 x={node.x - 55}
 y={node.y - 22}
 width="110"
 height="44"
 rx="8"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 className="transition-all duration-100"
 />

```

```
</div>
```

```
<div className="flex-1 space-y-4 overflow-y-auto" style={{height: 400px}}>
 {/** Topological List output */}
 <div>

 РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):

 <div className="bg-slate-900/60 p-2.5 rounded" style={{border: 1px solid #333, height: 150px; margin-top: 10px;}}>
 {step.topoList.length === 0 ? (

 ожидаем обработку узлов...

) : (
 <div>
 {step.topoListString}
 </div>
)}
 </div>
 </div>
```

```
{/** Color key */}
<div className="space-y-1">

 ВЕРШИНЫ В DFS:

 <div className="flex items-center gap-2">

 Серые (зашли)

 </div>
 <div className="flex items-center gap-2">

 Черные (готово)

 </div>
</div>
```

```

import { ArticulationPointsViz } from "./ArticulationPo
import BellmanFordViz from "./BellmanFordViz";
import BfsViz from "./BfsViz";
import { DPVisualizer } from "./DPVisualizer";
import { DfsBridgesSimulator } from "./DfsBridgesSimula
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
import { GraphTraversalViz } from "./GraphTraversalViz"
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import MnemonicCards from "./MnemonicCards";
import { PlanarityDemo } from "./PlanarityDemo";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomaton
import { SegmentTreeVisualizer } from "./SegmentTreeVis
import { SplayTreeViz } from "./SplayTreeViz";
import { SplayRotationsViz } from "./SplayRotationsViz"
import { SplayRotationSandbox } from "./SplayRotationSa
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithms"
import { TopologicalSortViz } from "./TopologicalSortVi
import { WaterfallAnimationWidget } from "./WaterfallAn
import ChapterImage from "./ChapterImage";
import { Simulator } from "./Simulator";
import { PrefixSumViz } from "./visualizers/PrefixSumVi
import { SparseTableViz } from "./visualizers/SparseTab

/** Разбор поворота и песочница всегда идут парой: посм
/**
 * Анимация и песочница идут друг под другом, а не в дв
 * колонке дерево сжималось до нечитаемого размера, а п
 * компактности. Между ними – подпись, объясняющая пере
 */
const SplayRotationsPair: React.FC = () => (

```

```

 <QueueViz />
 <BfsViz />
 </div>
),
},
"heap-beam-search": {
 title: "Куча (priority queue)",
 hint: "Добавляйте гипотезы — куча сама держит лучшую",
 Component: HeapViz,
},
"segment-trees": {
 title: "Дерево отрезков",
 hint: "Запросы и обновления на отрезке за $O(\log n)$.",
 Component: SegmentTreeVisualizer,
},
"sparse-table": {
 title: "Разреженная таблица (1D и 2D)",
 hint: "Наведите курсор на любую ячейку — подсветится",
 Component: SparseTableViz,
},
"prefix-sums-2d": {
 title: "Префиксные суммы (1D и 2D)",
 hint: "Наведите на число — увидите, из чего оно сложилось",
 Component: PrefixSumViz,
},
"dynamic-programming": {
 title: "Динамическое программирование",
 hint: "Таблица подзадач заполняется на глазах.",
 Component: DPVisualizer,
},
"splay-tree": {
 title: "Splay-дерево",
 hint: "Покадровый разбор трёх поворотов, песочница",
 Component: () => (
 <div className="space-y-10">
 <SplayRotationsPair />
 <SplayTreeViz />
 </div>
)
},

```

```

 <FloydViz />
 </>
),
 },
 "johnson-algo": { title: "Алгоритм Джонсона", Component: JohnsonAlgo },
 "mst-kruskal": { title: "Краскал и DSU", Component: KruskalSimulator },
 "mst-prima": { title: "Прим", Component: KruskalSimulator },
 "mst-boruvka": { title: "Борувка", Component: KruskalSimulator },
 "string-kmp": { title: "Префикс-функция (КМП)", Component: KMP },
 "string-z-func": { title: "Z-функция", Component: ZFunc },
 "aho-corasick": {
 title: "Ахо-Корасик",
 hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
 Component: () => (
 <div className="space-y-10">
 <WaterfallAnimationWidget />
 <SalmonAutomatonWidget />
 </div>
),
 },
 intro: { title: "Мнемокарточки", Component: MnemonicCards },
 "everyday-basics": {
 title: "Бытовой тренажёр: стек, очередь, куча",
 hint: "Тарелки, очередь в столовой и мешок гипотез",
 Component: Simulator,
 },
};

```

```
export const getViz = (id?: string): VizEntry | undefined;
```

ФАЙЛ 74/88: src/components/WaterfallAnimationWidget.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';

```

```

3: { S: 4 },
4: {},
5: { S: 6 },
6: {},
7: { H: 8 },
8: { E: 9 },
9: {},
};

export const WaterfallAnimationWidget: React.FC = () => {
 // Переключатель режимов: 'training' (Полный BFS обход)
 const [activeMode, setActiveMode] = useState<'training'>('training')

 // === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
 const [textToScan, setTextToScan] = useState<string>('')
 const [currentIndex, setCurrentIndex] = useState<number>(0)
 const [currentNode, setCurrentNode] = useState<number>(0)
 const [isSearchDone, setIsSearchDone] = useState<boolean>(false)
 const [searchLog, setSearchLog] = useState<string>('')
 'Режим поиска: Введите текст и жмите «Следующий шаг»'
);
 const [jumpPath, setJumpPath] = useState<{ from: number, to: number }>({ from: 0, to: 0 })
 const [foundMatches, setFoundMatches] = useState<{ index: number, value: string }[]>([])
 const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(0)

 // === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
 // Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
 const [trainStep, setTrainStep] = useState<number>(0)

 const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
 const clean = e.target.value.toUpperCase().replace(/ /g, '');
 setTextToScan(clean);
 resetSearchSimulation();
 };

 const resetSearchSimulation = () => {
 setCurrentIndex(0);
 setCurrentNode(0);
 };

```

```

 if (originalCurr === 0) {
 logMsg += `В корне нет перехода по '${char}'. Л
 setJumpPath(null);
 } else {
 logMsg += `Поток по '${char}' у вершины #${orig
 ружок по fail-ссылке в #${curr} (${WATERFALL_
 . `;
 setJumpPath({ from: originalCurr, to: nextNode,
 }
 curr = nextNode;
 }

const currentMatches: { index: number; word: string
const matchedWordsForLog: string[] = [];

let temp = curr;
while (temp !== 0) {
 if (WATERFALL_NODES[temp].is_terminal) {
 WATERFALL_NODES[temp].words.forEach((word) :
 currentMatches.push({ index: currentInd
 matchedWordsForLog.push(word);
 });
 }
 temp = WATERFALL_NODES[temp].term_link;
}

if (matchedWordsForLog.length > 0) {
 logMsg += ` БИНГО! Найдено слово: ${matchedWords
 setActiveSearchCodeLine(4);
}

setCurrentNode(curr);
setCurrentIndex(currentIndex + 1);
setSearchLog(logMsg);
if (currentMatches.length > 0) {
 setFoundMatches((prev) => [...prev, ...currentMat
}
};

```

```

 title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
 desc: 'Переходим на Depth 2 к вершине #2 (HE). Родитель :
 а-разведчик плывет в ROOT и ищет ветку по "E".',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'E',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 5,
 title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
 desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
 H" и "S" не подходят. Ветки нет, fail[HI] = ROOT',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'I',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 8,
 title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТАТЬ
 desc: ' ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на D
). Рыба плывет в ROOT и ищет ветку "H". У корня
 суффикс "H"!',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 3,
 title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
 desc: 'Переходим на Depth 3 к #3 (HER). Родитель :
 е подходят. fail[HER] = ROOT.',
 codeLine: 3,

```



```

];

 const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
 const activeFishPosNode = activeMode === 'training' ?
 const targetTrainingNode = WATERFALL_NODES[currentTrainStep];

 return (
 <div className="bg-slate-800/95 rounded-2xl p-6 border-2 border-slate-700">
 {/* Шапка и переключатель режимов */}
 <div className="flex flex-wrap items-center justify-between">
 <div className="flex items-center space-x-3">

 <div>
 <h4 className="text-2xl font-bold text-white">
 Анимация водопада: Полное обучение автомата
 </h4>
 <p className="text-sm text-slate-400">
 Визуализация пошагового построения всех функций
 </p>
 </div>
 </div>
 </div>

 {/* Переключатель режимов */}
 <div className="bg-slate-900 p-1.5 rounded-xl border-2 border-slate-700">
 <button
 onClick={() => setActiveMode('training')}
 className={`flex items-center space-x-1.5 p-2 transition-colors duration-200
 ${activeMode === 'training'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'}
 `}>
 <GitBranch className="w-4 h-4" />
 Режим 1: Полное обучение (BFS от корня)
 </button>
 <button
 onClick={() => setActiveMode('search')}
 className={`flex items-center space-x-1.5 p-2 transition-colors duration-200
 ${activeMode === 'search'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'}
 `}>
 <GitBranch className="w-4 h-4" />
 Режим 2: Поиск по узлам
 </button>
 </div>
 </div>
);
 }
}

```

```

 {idx === trainStep && <span className=
 </button>
 }}}
 </div>
 </div>

 <div className="flex gap-2">
 <button
 onClick={() => setTrainStep((prev) => M
 disabled={trainStep === 0}
 className="flex-1 bg-slate-700 hover:bg
ion-all"
 >
 Назад
 </button>
 <button
 onClick={() => setTrainStep((prev) => M
 disabled={trainStep === TRAINING_STEPS_
 className="flex-1 bg-indigo-600 hover:b
ition-all shadow-lg shadow-indigo-900/50"
 >
 {trainStep === TRAINING_STEPS_INFO.leng
 </button>
 </div>
</div>

```

/\* Блок с кодом построения fail-ссылки и опи

```

<div className="lg:col-span-2 bg-slate-900/90
 >
 <div>
 <h5 className="text-white font-bold mb-3
 <span className="flex items-center gap-
 Код BFS построения fail

 <span className="text-xs bg-indigo-950
 Вершина: #{targetTrainingNode.id} ({t

 </h5>
 </div>

```

```

 <Lightbulb className="w-4 h-4 text-amber-400" />
 </h5>
 <div className="bg-slate-800 p-4 rounded-lg flex items-center justify-between" >
 {currentTrainInfo.desc}
 </div>
 </div>
 </div>
) : (
 /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
 <div className="grid grid-cols-1 lg:grid-cols-3" >
 <div className="bg-slate-900/60 p-5 rounded-xl" >
 <div>
 <h5 className="text-white font-bold mb-2" >
 ✎ Текст для сканирования
 </h5>
 <p className="text-xs text-slate-400 mb-4" >
 Введи текст, чтобы лосось просканировал
 </p>
 <input
 type="text"
 value={textToScan}
 onChange={handleTextChange}
 maxLength={15}
 placeholder="ВВЕДИТЕ ТЕКСТ..."
 className="w-full bg-slate-800 border border-slate-700 focus:outline-none focus:border-blue-500"
 />
 </div>
 </div>
 <div>
 <div className="mb-4 flex flex-wrap gap-1" >

 {textToScan.split('').map((char, idx) =>
 <span
 key={idx}
 className={`w-7 h-8 flex items-center justify-center text-slate-400 font-mono`} >
 {char}

)}

 </div>
 </div>
 </div>
)

```

```

 <div className="bg-slate-950 p-4 rounded-
 <div className={`p-1.5 rounded flex item
border-blue-500 text-blue-300 font-bold' :
 1. curr = state; char = text[i]
 {activeSearchCodeLine === 1 && <span <
нициализация}
 </div>
 <div className={`p-1.5 rounded flex item
border-rose-500 text-rose-300 font-bold an
 2. while (curr !== root && !trie
 {activeSearchCodeLine === 2 && <span <
рыжок по fail!}
 </div>
 <div className={`p-1.5 rounded flex item
border-blue-400 text-blue-200 font-bold' :
 3. curr = trie[curr].get(char)
 {activeSearchCodeLine === 3 && <span <
ем по течению}
 </div>
 <div className={`p-1.5 rounded flex item
l-4 border-emerald-500 text-emerald-300 fon
 4. if (is_terminal[curr]) report
 {activeSearchCodeLine === 4 && <span <
овпадение!}
 </div>
 </div>
 </div>
 </div>

<div>
 <h5 className="text-slate-300 font-bold ml
 <AlertCircle className="w-3.5 h-3.5 tex
 </h5>
 <div className="bg-slate-800 p-3.5 rounder
tems-center">
 {searchLog}
 </div>
</div>
</div>

```

```

 { depth: 3, y: 360, label: 'Depth 3 (HER,
 { depth: 4, y: 460, label: 'Depth 4 (HERS
].map((level) => {
 <g key={`depth-line-${level.depth}`}>
 <line
 x1="20"
 y1={level.y}
 x2="780"
 y2={level.y}
 stroke="#334155"
 strokeWidth="1"
 strokeDasharray="4,4"
 opacity="0.4"
 />
 <rect x="25" y={level.y - 12} width="14"
 />
 <text x="32" y={level.y + 4} fill="#94a5b8"
 {level.label}
 </text>
 </g>
 }}}

```

```

{/* Рендер прямых переходов (потоков воды) */
Object.entries(TRIE_TRANSITIONS).map(([fromId, toId]) => {
 const fromNode = WATERFALL_NODES[Number(fromId)];
 return Object.entries(children).map(([childId, child]) => {
 const toNode = WATERFALL_NODES[toId];

```

```

 // Подсветка в режиме поиска
 let isHighlighted = activeMode === 'search' ||
 type === 'normal';

```

```

 // Подсветка в режиме обучения (когда ищем совпадения)
 let isTrainingExamined = activeMode === 'training' ||
 let isMatchBranch = isTrainingExamined && childId === toId;

```

```

 return (
 <g key={`edge-${fromNode.id}-${toNode.id}`}>

```

```
}}}
```

```

{/* Рендер fail-ссылок (пунктирные водопады)
Object.values(WATERFALL_NODES).map(({node}) => {
 if (node.id === 0) return null;
 const targetNode = WATERFALL_NODES[node.fail];

 // В режиме обучения показываем fail-ссылку
 // Найдем индекс шага, на котором обработали node
 const stepIdxForNode = TRAINING_STEPS_INFO[node.step];
 if (activeMode === 'training' && trainStep === stepIdxForNode) {

 let isHighlighted = activeMode === 'search' || activeMode === 'fail';
 let isJustEstablished = activeMode === 'training' && !targetNode.fail;

 // Вычисляем изогнутую кривую для красоты
 const dx = targetNode.x - node.x;
 const dy = targetNode.y - node.y;
 const cx = node.x + dx / 2 + (node.id % 2 === 0 ? -10 : 10);
 const cy = node.y + dy / 2 - 30;

 return (
 <g key={`fail-${node.id}`}>
 <path
 d={`M ${node.x} ${node.y} Q ${cx} ${cy} ${targetNode.x} ${targetNode.y}`
 fill="none"
 stroke={isHighlighted ? '#f43f5e' : '#ccc'}
 strokeWidth={isHighlighted || isJustEstablished ? 2 : 1}
 strokeDasharray="6,6"
 className={isHighlighted || isJustEstablished ? 'highlighted' : ''}
 opacity={isHighlighted || isJustEstablished ? 1 : 0.5}
 />
 {isJustEstablished && (
 <text x={cx} y={cy - 10} fill="#10b98f">
 fail[{node.label}] = {targetNode.label}
 </text>
)}
 </g>
);
 }
});

```

```

 fontSize={isCurrent || isTargetChild ? 12 : 10}
 fontWeight="bold"
 textAnchor="middle"
 >
 {node.label}
 </text>

 {/* Метка искомой вершины в режиме обучения */}
 {isTargetChild && {
 <g transform={`translate(${node.x}, ${node.y})`} >
 <rect x="-35" y="-12" width="70" height="24" fill="#8b572a" />
 <text x="0" y="2" fontSize="10" fontColor="white">
 ИЩЕМ FAIL
 </text>
 </g>
 }}

 {/* Индикатор словарных слов */}
 {hasWords && {
 <g transform={`translate(${node.x} + 10, ${node.y})`} >
 <circle cx="0" cy="0" r="9" fill="#8b572a" />
 <text x="0" y="3" fontSize="10" fontColor="white">
 *
 </text>
 </g>
 }}
 </g>
 </g>
 </g>
 </g>
 </g>

 {/* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */}
 <g>
 {/* Круг-подложка под рыбу */}
 <circle
 cx={activeFishPosNode.x + 15}
 cy={activeFishPosNode.y - 25}
 r="18"
 fill={activeMode === 'training' ? '#8b572a' : '#8b572a'}
 />
 </g>

```

```

 -sm font-bold shadow animate-[scaleUp_0.3s_ease-out]
 >
 <CheckCircle2 className="w-4 h-4 text-emerald-500" />
 {match.word}

 Индекс {match.index}

 </div>
)})
 </div>
}
</div>
);
};
```

## РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

ФАЙЛ 75/88: src/components/visualizers/PrefixSumViz.tsx

КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 330 | РАЗМЕР: 10240 байт

```

import React, { useMemo, useState } from "react";

/**
 * Префиксные суммы: 1D и 2D.
 *
 * Главная фишка – наведение на число:
 * • в 1D наводим на P[i] – подсвечивается кусок массива
 * • в 1D наводим на a[i] – видно, в какие префиксы он входит
 * • в 2D наводим на ячейку – подсвечивается прямоугольник
 * а при выбранном запросе показывается формула включения-исключения
 */
```



```

 });
 };

 /* ----- 1D -----
const Prefix1D: React.FC = () => {
 const pref = useMemo(
 () => A1.reduce<number[]>((acc, v) => [...acc, acc[0] + v],
 []
);

 /** Что сейчас под курсором: префикс P[i] или элемент a[i]
const [hover, setHover] = useState<{ kind: "pref" | "a"; i: number }>({ kind: "pref", i: 0 });
const [range, setRange] = useState<{ l: number; r: number }>({ l: 0, r: 0 });

const covered =
 hover?.kind === "pref"
 ? { from: 0, to: hover.i - 1 }
 : hover?.kind === "a"
 ? { from: hover.i, to: hover.i }
 : null;

const sum = pref[range.r + 1] - pref[range.l];

return (
 <div className="space-y-5">
 <div className="overflow-x-auto pb-1">
 <div className="inline-block min-w-full">
 /* индексы */
 <div className="flex gap-1.5 mb-1 pl-[52px]">
 {A1.map((_, i) => (
 <div key={i} className={` ${cellBase} !h-5`}>
 {i}
 </div>
))}
 </div>

 /* массив a */
 <div className="flex gap-1.5 items-center mb-1">

```

```

 active
 ? "bg-emerald-500 text-white ring
 : isR
 ? "bg-emerald-700 text-white"
 : isL
 ? "bg-rose-700 text-white"
 : "bg-slate-900 text-slate-300"
 }` }
 title={`P[${i}] = сумма a[0..${i} - 1]
 >
 {v}
 </div>
);
 }}}
</div>
</div>
</div>

{/* Пояснение под курсором */}
<div className="min-h-[62px] bg-slate-900 border
 {hover?.kind === "pref" ? {
 hover.i === 0 ? {
 <p className="text-slate-300">
 <b className="text-emerald-400">P[0] = 0<
 отрезка, начинающегося с нуля.
 </p>
 } : {
 <p className="text-slate-300">
 <b className="text-emerald-400">
 P[{hover.i}] = {pref[hover.i]}
 {" "}
 – это сумма всего, что набежало от начала

 a[0..{hover.i} - 1] = {A1.slice(0, hove

 </p>
 }
 } : hover?.kind === "a" ? {

```

```

const m = A2[0].length;

const P = useMemo(() => {
 const p = Array.from({ length: n + 1 }, () => Array(
 for (let i = 1; i <= n; i++) {
 for (let j = 1; j <= m; j++) {
 p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i]
 }
 }
), [n, m]);

const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });

const D = P[rect.r1][rect.c1];
const B = P[rect.r1][rect.c2 + 1];
const Cc = P[rect.r2 + 1][rect.c1];
const Aa = P[rect.r2 + 1][rect.c2 + 1];
const total = Aa - B - Cc + D;

const cornerOf = (i: number, j: number) => {
 if (i === rect.r2 + 1 && j === rect.c2 + 1) return "A";
 if (i === rect.r1 && j === rect.c2 + 1) return "B";
 if (i === rect.r2 + 1 && j === rect.c1) return "C";
 if (i === rect.r1 && j === rect.c1) return "D";
 return null;
};

return (
 <div className="space-y-5">
 <div className="grid gap-5 lg:grid-cols-2">
 {/* Исходная матрица */}
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">Исходная матрица A2</div>
 <div className="inline-block">
 {A2.map((row, i) => (
 <div key={i} className="flex gap-1.5 mb-1"

```

```

 ? "bg-rose-600 text-white"
 : "bg-slate-900 border border
return (
 <div
 key={j}
 onMouseEnter={() => setHover({ i,
 onMouseLeave={() => setHover(null
 className={`${cellBase} cursor-he
 isHover ? "bg-amber-500 text-wh
 }`}
 >
 {v}
 {corner && !isHover && (
 <span className="absolute -top-
ht">
 {corner}

)}
 </div>
);
}}
</div>
)}}
</div>
</div>
</div>

<div className="min-h-[54px] bg-slate-900 border
{hover ? (
 <p className="text-slate-300">
 <b className="text-amber-400">
 P[{hover.i}][{hover.j}] = {P[hover.i][hov
 {" "}
 — сумма всего прямоугольника от левого верх
 </p>
) : (
 <p className="text-slate-500">Наведите курсор
)}}

```

```

const st1D: number[][] = Array.from({ length: N }, () =>
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j) - 1][j - 1]);
 }
}
```

// Data for 2D Matrix (Sparse Table 2D)

```
const val2D = [
 [45, 12, 88, 34, 11, 76, 23, 90],
 [67, 19, 44, 55, 33, 21, 65, 87],
 [14, 51, 99, 13, 22, 64, 43, 76],
 [89, 32, 54, 71, 15, 88, 29, 60],
 [25, 41, 16, 92, 9, 17, 56, 31],
 [59, 18, 77, 24, 61, 82, 35, 12],
 [73, 8, 38, 85, 47, 95, 19, 58],
 [39, 81, 62, 28, 51, 42, 85, 14]
];
```

```
const ST2D: number[][][][] = Array.from({length: 8}, () =>
 Array.from({length: 8}, () =>
 Array.from({length: 4}, () =>
 Array(4).fill(0)
)
)
);
```

```
for (let r = 0; r < 8; r++) {
 for (let c = 0; c < 8; c++) {
 ST2D[r][c][0][0] = val2D[r][c];
 }
}
```

```
for (let kx = 0; kx <= 3; kx++) {
 for (let ky = 0; ky <= 3; ky++) {
 if (kx === 0 && ky === 0) continue;
 for (let r = 0; r + (1 << kx) <= 8; r++) {
```

```

 onClick={() => setTab("2d_query")}
 className={`px-3 sm:px-4 py-2 rounded-lg text-
 se-600 text-white" : "bg-slate-800 text-sla
 >
 3. Запрос 2D (Формула)
 </button>
 </div>

 <AnimatePresence mode="wait">
 {tab === "1d" ? (
 <Viz1D key="1d" />
) : tab === "2d_build" ? (
 <Viz2DBuild key="2d_build" />
) : (
 <Viz2DQuery key="2d_query" />
)}
 </AnimatePresence>
</div>
);
};

const Viz1D: React.FC = () => {
 const [step, setStep] = useState(0);
 const [hover, setHover] = useState<{ i: number; j: number}>({ i: 0, j: 0 });

 // Total steps: we animate computing each element for
 const computeSteps: { i: number; j: number }[] = [];
 for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 computeSteps.push({ i, j });
 }
 }

 const maxStep = computeSteps.length;
 const covered = hover ? { from: hover.i, to: hover.i + 1 } : { from: 0, to: 0 };

 return (
 <div className="flex flex-col gap-5">

```

```

 return (
 <div key={idx} className="flex flex-col i
 <div
 className={`flex items-center justify
w,44px)}] h-[clamp(26px,8vw,44px)] text-[cla
 inCover ? "bg-indigo-500 text-white
 }` }
 >
 {v}
 </div>
 <span className="text-[9px] text-slate-
 </div>
);
 }}}
 </div>
 <div className="mt-2 text-xs min-h-[36px]">
 {hover && covered ? (
 <p className="text-slate-300">
 <b className="text-sky-400">
 ST[{hover.i}][{hover.j}] = {stID[hover.i
 {" "}
 – это минимум на отрезке{" "}

 [{covered.from}, {covered.to}]
 {" "}
 длины $2^{\{hover.j\}}$ = {1 << hover.j}:{" "}

 min({arrID.slice(covered.from, covered.

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор (или тапните) на любое чи
 </p>
)}
 </div>
 </div>

```

```

// Active computing state
let isActive = false;
let isSrc1 = false;
let isSrc2 = false;

if (step > 0 && step <= maxStep) {
 const currentStep = computeSteps[step];
 if (currentStep.i === i && currentStep.j === j) {
 isActive = true;
 if (currentStep.j === j + 1 && currentStep.i === i) {
 isSrc1 = true;
 }
 if (
 currentStep.j === j + 1 &&
 currentStep.i === i - (1 << (currentStep.i - i))
) {
 isSrc2 = true; // wait, no. Src2
 }
 // Let's re-eval exact source:
 // We are asking if THIS cell [i][j]
 if (currentStep.j - 1 === j) {
 if (currentStep.i === i) isSrc1 = true;
 if (currentStep.i + (1 << (currentStep.i - i)) === i) {
 isSrc2 = true;
 }
 }
 }
}

return (
 <div key={j} className="relative flex" >
 {!isValid ? (
 <div className="w-[clamp(30px,90px,100px)] h-[clamp(30px,90px,100px)]" >
) : (
 <motion.div
 onMouseEnter={() => isVisible = true}
 onMouseLeave={() => setHover(false)}
 onClick={() => isVisible && setHover(true)}
 initial={{ opacity: 0, scale: 0.8 }}
 animate={{
 opacity: isVisible ? 1 : 0,
 scale: isVisible ? 1 : 0.8,
 }}
 >
)}
 </div>
)

```



```

 strokeWidth="3"
 />
 </motion.svg>
)}
</div>
);
}}
</React.Fragment>
)}}
</div>
</div>
</div>

{/* Formula helper text */}
<div className="bg-slate-900 border border-slate-
 {step > 0 && step <= maxStep ? (
 (() => {
 const c = computeSteps[step - 1];
 const half = 1 << (c.j - 1);
 return (
 <p className="text-lg text-slate-300">

 ST[{c.i}][{c.j}]
 {" "}
 = min(
 <span className="text-emerald-400 font-l
 ST[{c.i}][{c.j - 1}]

 ,
 <span className="text-amber-400 font-bo
 ST[{c.i + half}][{c.j - 1}]

) → min(
 {st1

 {st1D[c.i + half][c.j - 1]}

) ={" "}

```

```

 key={step}
 onClick={() => { setK(step); setHover(null);
 className={`px-4 py-2 rounded-md text-sm text-slate-400 hover:text-white hover:bg-slate-800`}
 >
 {step === 0 ? "Оригинал (1x1)" : `Шаг ${step}`}
 </button>
)}}
 </div>

 <div className="flex flex-col items-center gap-4 overflow-y-scroll" >
 <div className="flex flex-col items-center gap-4" >
 { [...Array(k + 1)].map((_, i) => k - i).map((i) => {
 const stepL = 1 << step;
 const stepSize = 8 - stepL + 1;
 const isCurr = step === k;

 return (
 <React.Fragment key={step}>
 {idx > 0 && <div className="text-slate-400" >
 <div className={`flex flex-col items-center gap-4`} >
 <h4 className={`font-bold mb-3 flex items-center`} >
 Матрица блоков {stepL}x{stepL}

 {stepL}
 </h4>
 <div className="max-w-full overflow-hidden" >
 <div className={`grid gap-[2px] p-[10px] border-indigo-900/40 shadow-[0_0_25px_rgba(99,102,241,0.5)]`} >
 <div className="grid grid-cols: repeat(${stepSize}, max-content);" >
 {Array.from({length: stepSize * stepSize}, (_, i) => {
 const r = Math.floor(idx / stepSize);
 const c = idx % stepSize;

 let highlightClass = isCurr ? 'highlight' : '';
 let zIndex = isCurr ? 'z-10' : 'z-0';
 })}
 </div>
 </div>
 </div>
 </div>
 </div>
 </React.Fragment>
);
 })}
 </div>
 </div>

```



---

```

 return (
 <>
 <div className="flex item
 <span className="flex
shadow-[0_0_8px_#f43f5e]"></div> ST[{activeC
 <span className="text
tiveCell.r][activeCell.c][k-1][k-1]}
 </div>
 <div className="flex item
 <span className="flex
shadow-[0_0_8px_#3b82f6]"></div> ST[{activeC
 <span className="text
tiveCell.r][activeCell.c + prevL][k-1][k-1]
 </div>
 <div className="flex item
 <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{a
 <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
 </div>
 <div className="flex item
 <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
 <span className="text
[activeCell.r + prevL][activeCell.c + prevL
 </div>
 </>
)
 })()}
</div>

 <div className="pt-3 mt-3 border-t
) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][k][k]}
 </div>
 <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
 </div>

```

```

<div className="grid grid-cols-[repeat(
ative shadow-inner w-max max-w-full">
 {Array.from({length: 64}).map((_,
 const r = Math.floor(idx / 8);
 const c = idx % 8;
 const isInQuery = r >= r1 && r
 return (
 <div key={idx} className={`
y-center rounded text-[clamp(9px,2.4vw,12px)
0 border-rose-500/50 border scale-105 z-10'
 {val2D[r][c]}
 </div>
)
)}
)}

```

```

<div
 className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
 style={{
 top: `${(r1 / 8) * 100}%`,
 left: `${(c1 / 8) * 100}%`,
 marginTop: '8px',
 marginLeft: '8px',
 height: 'calc(' + (H/8*100) +
 width: 'calc(' + (W/8*100) +
 }}
/>

```

```

{/* Показываем 4 угла в самой матрице */
<div className="absolute pointer-e
shadow-[0_0_10px_#f43f5e]" style={{ top: `
lc(`${(Ly / 8) * 100}% - 16px}`, height: `ca
 <div className="absolute pointer-e
shadow-[0_0_10px_#3b82f6]" style={{ top: `
width: `calc(`${(Ly / 8) * 100}% - 16px}`, l
 <div className="absolute pointer-e
ld-400 shadow-[0 0 10px #10b981]" style={{

```

Чтобы покрыть весь прямоугольник, мы  
 Name="font-mono bg-[#0a0f1e] px-1 rounded b  
 роса. (Цветные прямоугольники слева).

</p>

```
<div className="bg-[#0a0f1e] rounded-xl
text-slate-300 shadow-inner overflow-x-auto
 <div className="absolute right-3 top-3
 // 1.
 int<
ext-amber-300">1); <span className="
 int<
ext-amber-300">1); <span className="
 int<
"text-slate-500">// Выс: {Lx}

 int<
"text-slate-500">// Шир: {Ly}
<b
```

```
 // 2.
 int<
 В запроса)

```

```
 r1][<span className="text-white bg-r
```

```
 во, чтобы правый край уперся в c2)<b
 r1][<span className="text-white bg-b
```

```
 px, чтобы нижний край уперся в r2)<b
 nded">r2 - Lx + 1][<span className="
```

```
 и влево от c2)


```

```

 r-slate-700 text-white px-1 font-bold rounded
 </div>
)
 }
 }
</div>
</div>
</div>

 <div className="mt-auto bg-slate-950 font
)]">
 <div className="text-center font-bold
wrap bg-[#0a0f1e] py-3 px-2 rounded-lg border
 <span className="text-slate-400 font

 <span className="text-blue-400 m
n>
 <span className="text-emerald-40
span>
 <span className="text-amber-400 r
 <span className="text-slate-4
 <span className="ml-3 text-emerald-
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
 </div>
 </div>

 </div>
 </div>
)
}
```

---

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

---

```

 }
 }

function setTheme(theme) {
 const body = document.body;
 body.classList.remove('theme-dark', 'theme-light', 'theme-terminal');
 body.classList.add('theme-' + theme);

 // Just some visual feedback on buttons
 document.querySelectorAll('[id^="theme-"]')
 .forEach(btn => {
 btn.classList.remove('outline', 'outline-dark');
 });

 if (theme === 'dark') document.getElementById('btn-theme-dark').classList.add('text-white transition-colors');
 if (theme === 'light') document.getElementById('btn-theme-light').classList.add('text-white transition-colors');
 if (theme === 'terminal') document.getElementById('btn-theme-terminal').classList.add('text-white transition-colors');

 document.getElementById('btn-theme-dark').classList.add('text-white transition-colors');
 document.getElementById('btn-theme-light').classList.add('text-white transition-colors');
 document.getElementById('btn-theme-terminal').classList.add('text-white transition-colors');
 document.getElementById('btn-theme-notebook').classList.add('text-white transition-colors');
 document.getElementById('btn-theme-terminal').classList.add('text-white transition-colors');
}

// Initialize theme and mode
setTheme('dark');

// Setup intersection observer for TOC highlighting
document.addEventListener('DOMContentLoaded', () => {
 const mobileMenuBtn = document.getElementById('mobile-menu-btn');
 const mobileDrawer = document.getElementById('mobile-drawer');
 const closeDrawerBtn = document.getElementById('close-drawer-btn');
 const drawerOverlay = document.getElementById('drawer-overlay');

```



```
 });

 const observerOptions = {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: 0
 };

 const observer = new IntersectionObserver((
 entries.forEach(entry => {
 if (entry.isIntersecting && !document
 // Highlight active TOC link lo
 const id = entry.target.id;
 document.querySelectorAll('aside
 if (link.getAttribute('href'
 link.classList.add('bg-
 } else {
 link.classList.remove('l
 }
 }));
 }
 }));
 }, observerOptions);

 document.querySelectorAll('section[id^="que
 observer.observe(section);
 });
 });
</script>
</body>
</html>
```

```
/* Global formula scroll settings */
.formula-scroll {
 overflow-x: auto;
 overflow-y: hidden;
 -webkit-overflow-scrolling: touch;
 scrollbar-width: none; /* Firefox */
 -ms-overflow-style: none; /* IE and Edge */
}
.formula-scroll::-webkit-scrollbar {
 display: none;
}

mjsx-container {
 max-width: 100% !important;
}
mjsx-container[display="true"] {
 overflow-x: auto;
 overflow-y: hidden;
 max-width: 100%;
 scrollbar-width: none;
 -ms-overflow-style: none;
 -webkit-overflow-scrolling: touch;
}
mjsx-container[display="true"]::-webkit-scrollbar
 display: none;
}

@media (max-width: 640px) {
 html { font-size: 13px; }

 .uno-card { width: 40px; height: 60px; font
 .uno-card::before, .uno-card::after { font-
 .uno-card::before { top: 1px; left: 2px; }
 .uno-card::after { bottom: 1px; right: 2px;

 .uno-card.mini { width: 28px !important; he
 us: 4px; }
 .uno-card.mini::before, .uno-card.mini::aft
```

```

body.theme-light .bg-slate-800 { background-color: #1e293b; }
body.theme-light .bg-slate-700, body.theme-light .text-slate-800 { color: #f1f3f4; font-weight: bold; }
body.theme-light .text-slate-200, body.theme-light .text-slate-300 { color: #f1f3f4; }
body.theme-light .text-slate-400 { color: #475569; }
body.theme-light .border-slate-600, body.theme-light .border-slate-700 { border-color: #334155; }
body.theme-light .border-slate-500 { border-color: #475569; }
body.theme-light #top-controls { background-color: #f1f3f4; }

body.theme-notebook {
 background-color: #f8f8f8 !important; /* light gray background */
 color: #3b2818 !important; /* brown ink */
 background-image: repeating-linear-gradient(45deg, transparent, transparent 2px, #3b2818 2px, #3b2818 4px);
 background-attachment: local !important;
 font-family: "Comic Sans MS", "Caveat", "Serif";
}
body.theme-notebook .bg-slate-900, body.theme-notebook .bg-slate-700 {
 background-color: rgba(255, 255, 255, 0.7);
 border-color: #cbd5e1 !important;
 box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
}
body.theme-notebook .text-slate-200, body.theme-notebook .text-slate-300 { color: #f1f3f4; font-weight: bold; }
body.theme-notebook .text-slate-400 { color: #475569; }
body.theme-notebook .border-slate-600, body.theme-notebook .border-slate-700 { border-color: #334155; }
body.theme-notebook .border-slate-500 { border-color: #475569; }
body.theme-notebook .font-mono { font-family: "Monospace"; }
body.theme-notebook #top-controls { background-color: #f1f3f4; }

/* CREEPY BLINKING */
@keyframes creepy-blink {
 0%, 95%, 98%, 100% { transform: scaleY(1); }
 96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
 transform-origin: center;

```

```

 </div>
 </div>
</div>

<!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
<svg width="0" height="0" class="absolute">
 <defs>
 <marker id="arrow-yellow" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#fcd34d"
 </marker>
 <marker id="arrow-orange" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#f9731d"
 </marker>
 <marker id="arrow-pink" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#f472b0"
 </marker>
 <marker id="arrow-cyan" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4bf"
 </marker>
 <marker id="arrow-indigo" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8"
 </marker>
 <marker id="arrow-lime" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d"
 </marker>
 <marker id="arrow-green" markerWidth="6" ma
 <path d="M0,0 L6,3 L0,6 Z" fill="#4ade88"
 </marker>

 <!-- ЛЕГО БЛОКИ -->
 <g id="lego-yellow">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
 </g>
 <g id="lego-red">
 <rect x="0" y="0" width="40" height="20"

```

```
 <svg xmlns="http://www.w3.org/2000/svg" class="lg:w-10 lg:h-10">
 <path stroke-linecap="round" stroke-linejoin="round" stroke="black" stroke-width="2" d="M6 9l9 9" />
 </svg>
 </button>
```

```
<!-- Затемнение фона на мобилках при открытом меню -->
<div id="drawer-overlay" class="lg:hidden fixed inset-0 bg-slate-500 bg-opacity-50">
```

```
<div class="max-w-7xl mx-auto mt-10 px-4 flex flex-col">
```

```
<!-- Левое меню (Оглавление) -->
```

```
<aside id="mobile-drawer" class="fixed inset-y-0 left-0 w-full lg:w-72 lg:shrink-0 lg:z-auto
 translate-x-full transition-transform duration-300 ease-in-out
 w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto">
 <div class="h-full flex flex-col p-6 overflow-hidden">
 <div class="flex justify-between items-center">
 <h3 class="font-black text-transparent bg-clip-text bg-gradient-to-r from-blue-500 to-blue-300">
 Содержание
 </h3>
 <button id="close-drawer-btn" class="lg:hidden text-slate-500">
 <svg xmlns="http://www.w3.org/2000/svg" class="h-6 w-6">
 <path stroke-linecap="round" stroke-linejoin="round" stroke="black" stroke-width="2" d="M6 9l9 9" />
 </svg>
 </button>
 </div>
 <nav class="space-y-3 text-sm font-medium">
```

```
 <div>
 <details class="group" open>
 <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-300">
 1. Алгебраические структуры
 </summary>
 <div class="pl-6 space-y-2 text-sm">

 1.1. Группы

 1.1.1. Группы

 </div>
 </details>
 </div>
 </nav>
 </div>
</aside>
```

er-amber-500 pl-3 font-bold text-amber-300">

4. Степени и корни

</a>

</summary>

<div class="pl-6 space-y-2 text

<a href="#q4-def" class="block l

<a href="#q4-moivre" class="blo

<a href="#q4-root" class="block

<a href="#q4-count" class="bloc

</div>

</details>

<details class="group">

<summary class="list-none curso

<a href="#question-5" class:

er-fuchsia-500 pl-3 font-bold text-fuchsia-

5. Группа корней из единицы

</a>

</summary>

<div class="pl-6 space-y-2 text

<a href="#q5-def" class="block l

<a href="#q5-group" class="bloc

<a href="#q5-prim" class="block

<a href="#q5-crit" class="block

<a href="#q5-cyclic" class="blo

</div>

</details>

<details class="group">

<summary class="list-none curso

<a href="#question-6" class:

er-lime-500 pl-3 font-bold text-lime-300">

6. Делимость, НОД и Евклид

</a>

</summary>

<div class="pl-6 space-y-2 text

<a href="#q6-1" class="block ho



```
<div class="pl-6 space-y-2 text
 <a href="#q13-1" class="blo
 <a href="#q13-2" class="blo
 <a href="#q13-3" class="blo
</div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-14" clas
der-fuchsia-500 pl-3 font-bold text-fuchsia
 14. Кратность корня. Ко

 </summary>
 <div class="pl-6 mt-2 space-y-2
 <a href="#q14-1" class="blo
 <a href="#q14-2" class="blo
 <a href="#q14-3" class="blo
 deg)
 <a href="#q14-4" class="blo
ition-colors border border-rose-500/30"> K
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-15" clas
der-cyan-500 pl-3 font-bold text-cyan-400">
 15. Производная. Критер

 </summary>
 <div class="pl-6 mt-2 space-y-2
 <a href="#q15-1" class="blo
 <a href="#q15-2" class="blo
 <a href="#q15-3" class="blo
 <a href="#q15-4" class="blo
 </div>
</details>
```



```
 <span class="font-bold text-fuc

 <a href="#proof-algorithms" class="l
-red-500 pl-3 mt-4 text-red-500 font-bold ml
 ▲ Ликбез: Алгоритмы выживания

</nav>
<script>
 document.querySelectorAll('details.
 detail.addEventListener('toggle
 if (detail.open) {
 document.querySelectorA
 if (other !== detail
 other.removeAtt
 }
 });
 }
 });
 });

 // Scroll spy logic
 document.addEventListener('DOMConten
 const sections = document.query
 ="q5-"], div[id^="q6-"], div[id^="q7-"], di
 v[id^="q13-"], div[id^="q14-"], div[id^="q1
 let isScrolling = false;

 document.querySelectorAll('.grou
 link.addEventListener('clie
 isScrolling = true;
 setTimeout(() => isScro
 });
 });

 const observer = new Intersectio
 if (isScrolling) return;
```

```

 });

 function fallbackModal(contentRaw,
 const modal = document.createElement('div');
 modal.style.position = 'fixed';
 modal.style.top = '0';
 modal.style.left = '0';
 modal.style.width = '100vw';
 modal.style.height = '100vh';
 modal.style.backgroundColor = 'black';
 modal.style.zIndex = '99999';
 modal.style.display = 'flex';
 modal.style.flexDirection = 'column-reverse';
 modal.style.alignItems = 'center';
 modal.style.justifyContent = 'center';

 const box = document.createElement('div');
 box.style.width = '80%';
 box.style.height = '80%';
 box.style.backgroundColor = '#1a1a1a';
 box.style.padding = '20px';
 box.style.borderRadius = '10px';
 box.style.display = 'flex';
 box.style.flexDirection = 'column-reverse';
 box.style.color = '#fff';

 const header = document.createElement('div');
 header.innerText = 'Экспорт в PDF';
 header.style.marginBottom = '10px';

 const subHeader = document.createElement('div');
 subHeader.innerText = 'Из-за проблем с совместимостью
некоторые веб-студии в новой вкладке (через меню)';
 subHeader.style.marginBottom = '10px';
 subHeader.style.color = '#f87171';
 subHeader.style.fontSize = '0.9em';

 const textarea = document.createElement('div');

```

```

 closeBtn.style.borderRadius = '50%';
 closeBtn.style.cursor = 'pointer';
 closeBtn.onclick = () => modal.close();
 }

 box.appendChild(header);
 box.appendChild(subHeader);
 if (title !== 'PDF') {
 box.appendChild(textarea);
 btnContainer.appendChild(closeBtn);
 }
 btnContainer.appendChild(closeBtn);
 box.appendChild(btnContainer);
 modal.appendChild(box);
 document.body.appendChild(modal);
}

function downloadMD() {
 let clone = document.querySelector('#md').cloneNode(true);

 // Smart markdown parsing based on DOM
 // Smart markdown parsing based on DOM
 clone.querySelectorAll('h2').forEach(el => {
 clone.querySelectorAll('h3').forEach(el => {
 clone.querySelectorAll('.analog').forEach(el => {
 clone.querySelectorAll('li').forEach(el => {
 clone.querySelectorAll('svg').forEach(el => {
 let caption = '';
 if (el.nextElementSibling && el.nextElementSibling.tagName === 'img') {
 caption = el.nextElementSibling.alt;
 }
 el.textContent = '\n\n[Иллюстрация: ' + caption + ']';
 });
 });
 });
 });
 });
 clone.querySelectorAll('.img-caption').forEach(el => {
 let text = clone.innerHTML;
 text = text.replace(/\n{3,}/g, '\n\n');
 });
}

```



```

</script>

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И ...
<div class="mt-8 flex flex-col gap-4 bo
 <!-- Кнопки Конвертации / Экспорта
 <div class="flex flex-col gap-2">
 <div class="text-xs text-slate-
 <button onclick="downloadWord()
ont-bold flex items-center justify-center g
 Word
 </button>
 <a href="algebra.pdf" target="_
-sm font-bold flex items-center justify-cen
 PDF

 <button onclick="downloadMD()"
-b bold flex items-center justify-center gap-
 Markdown
 </button>
 <button onclick="downloadHTML()
t-sm font-bold flex items-center justify-cen
 HTML
 </button>
 </div>

 <!-- Смена темы -->
 <div>
 <div class="text-xs text-slate-
 <div class="flex items-center g
 <button id="theme-dark" onc
justify-center text-amber-300 hover:text-wh
line-amber-500" title="Темная тема"> </butt
 <button id="theme-light" on
r justify-center text-amber-50 hover:text-w
">* </button>
 <button id="theme-terminal"

```

```

15.
17.
18.
19.
20.
21.
22.

</div>
</header>

<!-- ===== WTF CONTAINER =====>
<div id="wtf-container" class="hidden">
 <h2 class="text-3xl font-black text-cen

 <div class="overflow-x-auto w-full pb-4">
 <div class="grid grid-cols-3 gap-4">
 <!-- Column headers -->
 <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
 <div class="bg-orange-900/40 bo
ольцо полиномов</div>
 <div class="bg-cyan-900/40 bord
сные числа</div>

 <!-- Row 1: База -->
 <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans

```

```

 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-11" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-5" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans.
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 5: Корни / Уравнения -->
 <div class="hidden md:block"></div>
 <a href="#question-13" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans.
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 6: Кратность -->
 <div class="hidden md:block"></div>
 <a href="#question-14" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <div class="hidden md:block"></div>

 <!-- Row 7: Производная -->
 <div class="hidden md:block"></div>

```

```
border-l-4 border-fuchsia-500 hover:bg-slate-700
 <div class="text-xs text-fuchsia-500">
 <div class="font-bold text-slate-700">

```

```
<!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ -->
<div class="col-span-1 md:col-span-3"
border-b border-red-500/30 pb-2">Кластер 4:
```

```
 <a href="#proof-algorithms" onclick="
:col-span-3 bg-red-900/20 p-4 border-l-4 border-r-4
 <div class="text-xs text-red-500">
 <div class="font-bold text-slate-700">

</div>
</div>
```

```
<!-- ЛИКБЕЗ SECTION INSERTED HERE -->
<section id="proof-algorithms" class="mb-6"
 <div class="flex items-center mb-6">
 <div class="bg-red-500/20 p-3 rounded"
 <svg class="w-8 h-8 text-red-500"
rg/2000/svg">
 <path stroke-linecap="round"
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
 </svg>
 </div>
 <h2 class="text-3xl pr-4 border-r-4">
</div>
```

```
 <div class="prose prose-invert max-w-3xl">
 <p class="text-lg">
 Хватит тупить. Доказательств
есь для того, чтобы жать на кнопки. Вот жес
 </p>
```

```
 <h2 class="text-2xl font-bold border-top-2px solid black">
(Меметика)</h2>
```



```


 </div>

 <!-- Индукция -->
 <div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
 <h3 class="text-xl text-yellow-400">Индукция
 <p class="text-sm text-slate-400">
 <ul class="list-disc pl-5" style="list-style-type: none;">

 валось.

 Просто перепиши формулу с буквой k .

 усок k , замени его на жареного карася из

 </div>

 <!-- Двойная делимость -->
 <div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
 <h3 class="text-xl text-orange-400">Двойная делимость
 <p class="text-sm text-slate-400">
 <ul class="list-disc pl-5" style="list-style-type: none;">

 вверх по уравнениям).

 рху вниз, показывая, что Δr_i

 х. Твой d больше.

 </div>

 <h2 class="text-2xl font-bold" style="color: #f1c40f;">
 ритмы экзекуции)</h2>

 <div class="grid grid-cols-1 md:grid-cols-2" style="background-color: #f1c40f; padding: 10px;">
 <div class="bg-slate-800 p-6" style="background-color: #1a202c; color: white;">
 <h4 class="text-red-400">

```

```

<div>
 <h4 class="text-cyan">
 <p class="text-sm text-cyan">
 <code class="text-x
 </div>

<li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm text-cyan">
 x двух, потом последних.

 <code class="text-x
 </div>

 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm text-cyan">
 есть сложение – есть ноль. Если есть умножение,
 <code class="text-x
 </div>

 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm text-cyan">
 авь их драться. Умножение всегда врывается и
 <code class="text-x
 </div>


```

```
* Файл получается автономным: стили защиты внутрь, инт
* его можно открыть с флешки, отправить в мессенджер и
* Живые React-визуализации в статику не переносятся —
* подставляется заметка со ссылкой на онлайн-версию, а
* которые в приложении всплывают по наведению, собираю
* в конце документа.
*/
```

```
/** Собирает CSS страницы: сначала пробуем правила, при
async function collectCss(): Promise<string> {
 const parts: string[] = [];
 for (const sheet of Array.from(document.styleSheets))
 try {
 const rules = (sheet as CSSStyleSheet).cssRules;
 parts.push(Array.from(rules).map((r) => r.cssText
 } catch {
 const href = (sheet as CSSStyleSheet).href;
 if (!href) continue;
 try {
 const res = await fetch(href);
 if (res.ok) parts.push(await res.text());
 } catch {
 /* чужой домен — пропускаем */
 }
 }
 }
 return parts.join("\n");
}
```

```
const EXTRA_CSS = `
/* — правки для автономного файла — */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family
a,term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; paddi
.export-head { border-bottom: 1px solid #1e293b; paddin
.export-note { background: #0f172a; border: 1px solid #
 border-radius: .5rem; padding: .85rem 1rem; font-size
```

```

 if (!id) return;
 if (!ids.includes(id)) ids.push(id);
 const link = document.createElement("a");
 link.className = "term-hint";
 link.href = `#term-${id}`;
 link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
 link.textContent = el.textContent ?? "";
 el.replaceWith(link);
 });

 // <details> в файле лучше раскрыть: иначе код и разб
 host.querySelectorAll("details").forEach((d) => d.set

 return { body: host.innerHTML, termIds: ids };
}

function glossarySection(termIds: string[]): string {
 if (!termIds.length) return "";
 const items = termIds
 .map((id) => {
 const e = GLOSSARY_BY_ID.get(id);
 if (!e) return "";
 return `<div class="export-term" id="term-${escape
<h3>${escapeHtml(e.title)}</h3>
<p>${escapeHtml(e.short)}</p>
${e.formal ? `<p class="formal">${escapeHtml(e.formal
${e.complexity ? `<p class="cx">Сложность: ${escapeHt
</div>`;
 })
 .join("\n");

 return `<section class="export-gloss">
<h2>Словарик терминов из этой темы</h2>
<p style="font-size:.8rem;color:#94a3b8;margin:0 0 1r
 ны сюда.</p>
 ${items}
</section>`;
}

```

```

function triggerDownload(html: string, filename: string) {
 const blob = new Blob([html], { type: "text/html;charset=utf-8" });
 const url = URL.createObjectURL(blob);
 const a = document.createElement("a");
 a.href = url;
 a.download = filename;
 document.body.appendChild(a);
 a.click();
 a.remove();
 setTimeout(() => URL.revokeObjectURL(url), 2000);
}
```

```
/** Имя файла: «04-splay-derevo.html» – без кириллицы и пробелов */
function safeName(title: string, id: string): string {
 const num = title.match(/^s*(\d+)/)?.[1];
 const base = id.replace(/[^a-z0-9-]+/gi, "-").toLowerCase();
 return `${num ? String(num).padStart(2, "0") + "-" : ""}${base}.html`;
}
```

```
/** Выгрузить одну главу. */
export async function downloadChapterHtml(chapter: Chapter) {
 const css = await collectCss();
 const { body, termIds } = prepareBody(chapter.content);
 const html = wrap({
 title: chapter.title,
 subtitle: chapter.category || "Универсальное пособие",
 css,
 body,
 gloss: glossarySection(termIds),
 origin: window.location.origin,
 });
 triggerDownload(html, safeName(chapter.title, chapter.id));
}
```

```
/** Выгрузить всё пособие одним файлом со сквозным оглавлением */
export async function downloadBookHtml(chapters: Chapter[]) {
 const css = await collectCss();
 const allTerms: string[] = [];
 const allContent: string[] = [];
 for (const chapter of chapters) {
 const { body, termIds } = prepareBody(chapter.content);
 allTerms.push(...termIds);
 allContent.push(body);
 }
 const html = wrap({
 title: "Универсальное пособие",
 subtitle: "Содержание",
 css,
 body: allContent.join("\n"),
 gloss: glossarySection(allTerms),
 origin: window.location.origin,
 });
 triggerDownload(html, "book.html");
}
```

ФАЙЛ 81/88: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМ

```
/**
 * Аудит покрытия глав пособия: у каких тем нет «объясн
 * (блок «Аналогия») и/или 2D-визуализации (интерактивн
 *
 * node scripts/audit-coverage.mjs # отчёт
 * node scripts/audit-coverage.mjs --md # markd
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
 entryPoints: [path.join(ROOT, "src/data/content.ts")],
 bundle: true,
 format: "esm",
 platform: "node",
 outfile: bundle,
 logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/c
const registryBody = registry.slice(registry.indexOf("V
const vizIds = new Set([...registryBody.matchAll(/^\s{2
```

```

console.log(`Глав в пособии: ${rows.length}\n`);
const dump = (title, list) => {
 console.log(`${title} (${list.length}):`);
 list.forEach((r) => console.log(` * ${r.title} [$
 console.log();
};
dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
dump("НЕТ 2D (аналогия есть)", gapViz);
console.log(`Пропущенные номера билетов 1-24: ${missi
console.log(`Визуализаторы без главы: ${orphanViz.join
}

```

-----  
ФАЙЛ 82/88: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```

/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 * node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
 entryPoints: ["src/data/content.ts", "src/data/glossa
 bundle: true, format: "esm", platform: "node",
 outdir: "tmp/audit-terms", outExtension: { ".js": ".m
 external: ["react", "react-dom", "motion", "lucide-re
});

const { chapters } = await import(path.join(ROOT, "tmp/
const { GLOSSARY LABELS, GLOSSARY } = await import(path

```

```
 if (ids.size === 0) bad.push(c.title);
 console.log(
 String(ids.size).padStart(3), "|", String(highlight
 c.title.slice(0, 46).padEnd(47), [...ids].slice(0,
);
 }
 console.log("\nГлав без единой подсказки:", bad.length,
 console.log("Терминов в глоссарии:", GLOSSARY.length, "
 console.log("Не встретились:", GLOSSARY.filter((g) => !
```

---

ФАЙЛ 83/88: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРӨК: 64 | РАЗМ

---

```
/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pa
 * в единый документ public/export/full_code_all_pages.p
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir,

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
 if (!fs.existsSync(PAGES_DIR)) {
 throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)
 }

 const pages = fs
 .readdirSync(PAGES_DIR)
```



```
main().catch((err) => {
 console.error("Ошибка сборки PDF:", err.message);
 process.exit(1);
});
```

ФАЙЛ 84/88: scripts/export/collect-code.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 196 | РАЗМ:

```
/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕ исходный код репозитория в один текст
 * public/export/full_code_all_pages.txt
 *
 * Список файлов берётся из git (чтобы ничего не забыть
 * с фоллбэком на обход файловой системы, если git недо
 */
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,

/** Расширения, которые считаем «кодом» и включаем в да
const CODE_EXT = new Set([
 ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
 ".css", ".scss", ".html", ".json", ".md", ".yml", ".y
]);

/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",

/** Человечитаемые категории для оглавления. */
const CATEGORIES = [
 [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"]
];
```

```

files = [];
const walk = (dir) => {
 for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
 const abs = path.join(dir, e.name);
 const rel = path.relative(ROOT, abs).split(path.sep);
 if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;
 if (e.isDirectory()) walk(abs);
 else files.push(rel);
 }
};
walk(ROOT);

return files
 .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d)))
 .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
 .filter((rel) => CODE_EXT.has(path.extname(rel)))
 .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
 .sort((a, b) => {
 const ia = ORDER.indexOf(categoryOf(a));
 const ib = ORDER.indexOf(categoryOf(b));
 return ia !== ib ? ia - ib : a.localeCompare(b);
 }));
}

```

/\*\* Достаём заголовки глав из данных пособия (без запуска)

```

function extractChapters(files) {
 const chapters = [];
 const seen = new Set();
 for (const rel of files.filter((f) => f.startsWith("chapters/"))) {
 const src = fs.readFileSync(path.join(ROOT, rel), "utf8");
 const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"([^"]+)"/g;
 let m;
 while ((m = re.exec(src)) !== null) {
 const [, id, title] = m;
 if (seen.has(id)) continue;
 seen.add(id);
 chapters.push({ id, title, file: rel });
 }
 }
}

```

```

push(HR);
push(" ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДНИКОВ");
push(HR);
let current = null;
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 push();
 push(`    # ${cat}`);
 }
 push(`    ${String(i + 1).padStart(3, " ")}: ${rel}`);
});
push();
push();

current = null;
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 push(HR);
 push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
 push(HR);
 push();
 }
 const content = fs.readFileSync(path.join(ROOT, rel), "utf-8");
 const lines = content.split("\n");
 push(SUBHR);
 push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
 push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} | СЛОВА: ${words.length}`);
 push(SUBHR);
 push();
 push(content.replace(/\n+$/, ""));
 push();
 push();
});

```

Универсальное пособие • полный исходный код

```

export const OUT_DIR = path.join(ROOT, "public", "export");
/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают) */
export const PAGES_DIR = path.join(ROOT, "tmp", "export");

export const TXT_PATH = path.join(OUT_DIR, "full_code_and_pages.txt");
export const PDF_PATH = path.join(OUT_DIR, "full_code_and_pages.pdf");

/** Параметры вёрстки страницы (подобраны под A4 @150dpi) */
export const PAGE = {
 /** DPI раstra. Можно переопределить: EXPORT_DENSITY=100 */
 density: Number(process.env.EXPORT_DENSITY ?? 150),
 /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT_GRAYSCALE=1 */
 monochrome: process.env.EXPORT_GRAYSCALE !== "1",
 size: "A4",
 font: "DejaVu-Sans-Mono",
 fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
 pointsize: 8,
 /** Максимум символов в строке (далее – жёсткий перенос) */
 cols: 138,
 /** Строк содержимого на странице (+2 служебные: шапка, подвал) */
 rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
 fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
 if (bytes < 1024) return `${bytes} B`;
 if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;
 return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
 const cmd = process.env.IMAGEMAGICK_CMD || "magick";
 if (process.env.IMAGEMAGICK_VERSION) {
 const version = process.env.IMAGEMAGICK_VERSION;
 if (version.startsWith("7")) return cmd;
 if (version.startsWith("6")) return "convert";
 }
 return cmd;
}
```

```

/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
 const expanded = line.replace(/\t/g, " ").replace(
 if (expanded.length <= PAGE.cols) return [expanded];
 const parts = [];
 const indent = " ".repeat(Math.min((expanded.match(/
 let rest = expanded;
 let first = true;
 while (rest.length > 0) {
 const width = first ? PAGE.cols : PAGE.cols - indent;
 parts.push((first ? "" : indent) + rest.slice(0, wi
 rest = rest.slice(width);
 first = false;
 }
 return parts;
}

function paginate(txt) {
 const source = txt.replace(/\r\n/g, "\n").split("\n")
 const pages = [];
 for (let i = 0; i < source.length; i += PAGE.rows) {
 pages.push(source.slice(i, i + PAGE.rows));
 }
 return pages;
}

async function main() {
 if (!fs.existsSync(TXT_PATH)) {
 throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)
 }

 const im = imageMagickCmd();
 const pages = paginate(fs.readFileSync(TXT_PATH, "utf8

 fs.rmSync(PAGES_DIR, { recursive: true, force: true }
 ensureDir(PAGES_DIR);

 const total = pages.length;
```

```
 process.stdout.write(`      отрендерено ${done}`);
 }
 }
 }

 await Promise.all(Array.from({ length: concurrency },
 process.stdout.write("\n"));

 const missing = jobs.filter((j) => !fs.existsSync(j.path));
 if (missing.length) throw new Error(`Не отрендерены с ${missing.length} страниц`);

 const bytes = jobs.reduce((s, j) => s + fs.statSync(j.path).size, 0);
 console.log("[2/3] txt -> png");
 console.log(`      страниц: ${total}`);
 console.log(`      выход:   ${path.relative(ROOT, PAGE_PATH)}`);
}

main().catch((err) => {
 console.error("Ошибка рендера страниц:", err.message);
 process.exit(1);
});
```

=====

РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

=====

-----

ФАЙЛ 87/88: [.github/workflows/deploy-pages.yml](#)

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 61 | РАЗМЕР: 1.3 КБ

-----

name: Deploy app to GitHub Pages

on:

push:

branches:

- main

```
- name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist
```

```
deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
```

---

ФАЙЛ 88/88: .github/workflows/rebuild-pdf.yml

КАТЕГОРИЯ: CI / автоматизация | СТРӨК: 128 | РАЗМЕР: 4.0 КБ

---

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
#
|
|→ page-0001.png ... page-NNNN.png (
#
|
|→ full_code_all_page
```

```
Готовые TXT и PDF коммитаются обратно в ветку (их отда
промежуточные PNG сохраняются как artifacts запуска.
```

```
on:
 push:
```

cache: npm

- name: Install ImageMagick + DejaVu fonts  
run: |  
    sudo apt-get update -qq  
    sudo apt-get install -y --no-install-recommends  
    # На Ubuntu политика ImageMagick иногда запрещает экспорт в текстовый формат  
    # разрешаем то, что нужно пайплайну (text -> pdf)  
    for policy in /etc/ImageMagick-6/policy.xml /usr/lib/ImageMagick-6/policy.xml; do  
        if [ -f "\$policy" ]; then  
            sudo sed -i 's/rights="none" pattern="\{PDF,PS,PSD,SVG,SVGZ,TIFF,WMF,XPS\}/rights="write,write-replace"/g' "\$policy"  
        fi  
    done  
    convert -version
- name: Install dependencies  
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt  
run: npm run export:txt
- name: Step 2 – txt → png  
env:  
    EXPORT\_DENSITY: \${GITHUB\_EVENT\_INPUTS.density:-300}  
run: npm run export:png
- name: Step 3 – png → pdf  
run: npm run export:pdf
- name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
- name: Summary  
run: |  
    {  
        echo "### Экспорт пересобран"  
        echo ""



Универсальное пособие • полный исходный код

.....  
=====

КОНЕЦ ДОКУМЕНТА • 88 файлов • сгенерировано 11.09.2026,

=====